

计算机组成原理

王鸿鹏
计算机科学与技术学院
wanghp@hit.edu.cn

第五章 流水线处理器

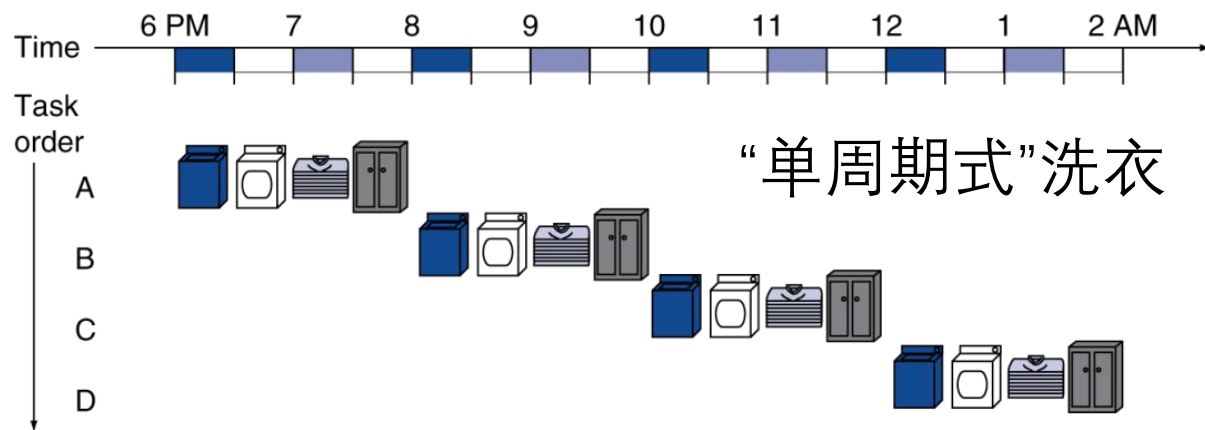
- 流水线概述
- 流水线数据通路和控制
- 解决数据冒险的方法：前递与停顿
- 控制冒险
- 例外

单周期处理器的性能问题

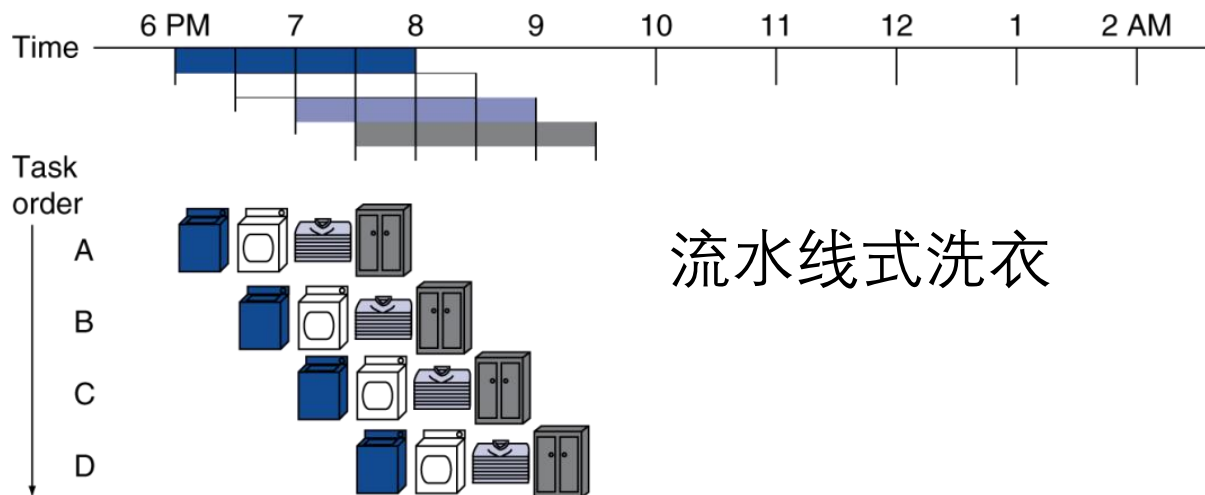
- 处理器中的关键路径决定了时钟周期：单周期设计中，时钟周期对于每条指令必须相等
 - 例如load指令，经历了完整的五个步骤：指令存储器、寄存器堆、ALU、数据存储器、寄存器堆
- 违反了设计思想
 - 无法加速经常性事件：不能使用那些 缩短常用指令执行时间而不改变最坏情况 的实现技术
- 可通过流水线来提高性能

生活中的流水线

- 假设洗衣包括**四个**步骤：洗衣机中**洗衣**、烘干机中**烘干**、**叠衣服**、**收纳**到柜子中，每个步骤0.5小时。



- 洗衣任务分4段，**加速比**
 $= 2*4/(2+0.5*3) \approx 2.3$



- 洗衣任务分n段，加速比
 $= 2n/(2+0.5*(n-1))$
 $\approx 4 = \text{流水线中的步骤数}$

RISC-V 指令执行的五个阶段

- IF (Instruction Fetch): 从指令存储器中取出指令
- ID (Instruction Decode): 读寄存器堆并译码指令
- EX (EXecute): ALU执行操作或计算地址
- MEM (MEMory access): 访问数据存储器（如有必要）
- WB (Write Back register): 将结果写入寄存器（如有必要）
- 让五个阶段重叠执行**可提高效率**

流水线性能分析

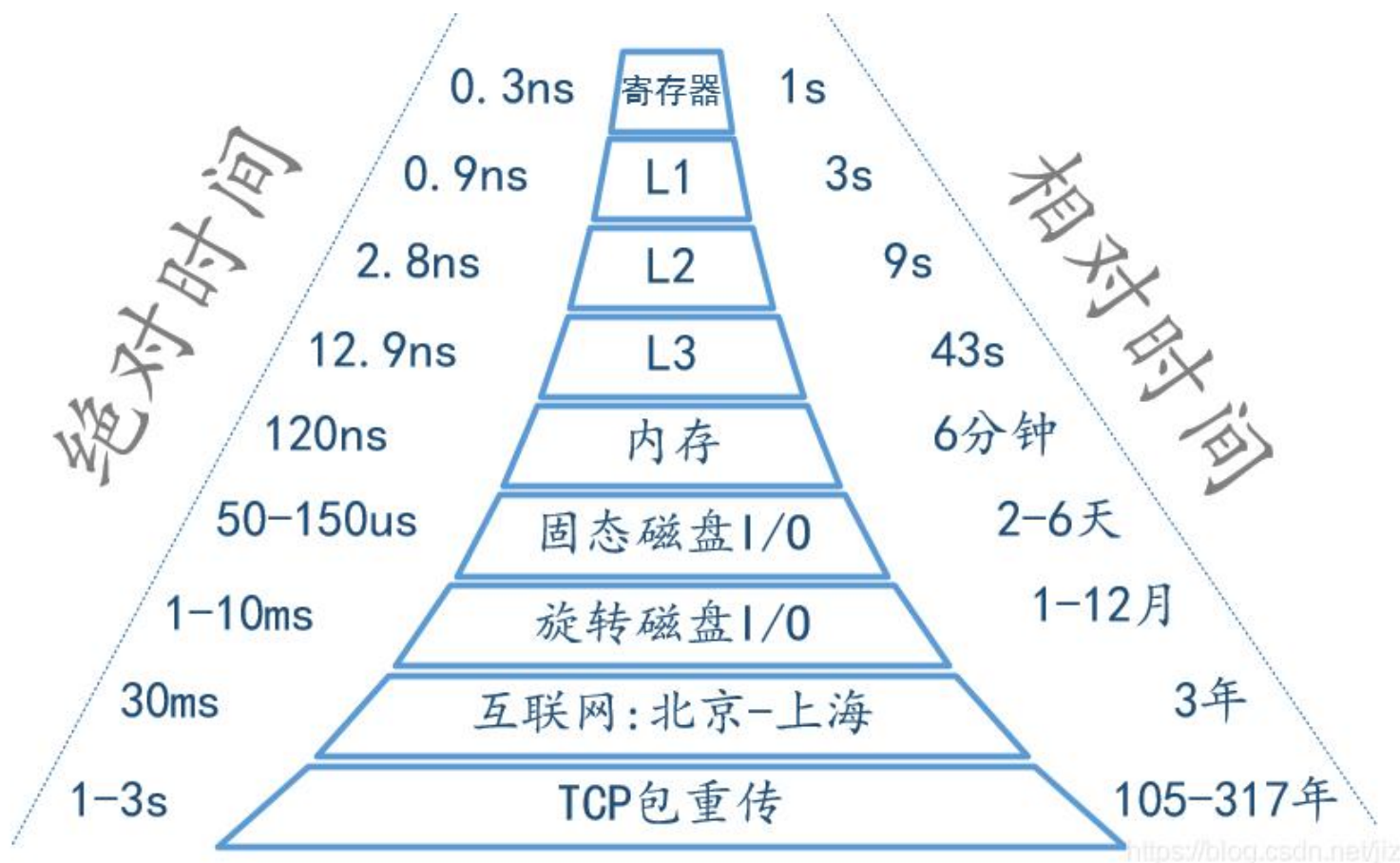
- 假设各个阶段的耗时：
 - 寄存器堆的读或写为100ps
 - 其他阶段为200ps
- 比较流水线指令执行与单周期指令执行的平均执行时间

指令类型	取指令	读寄存器	ALU 操作	数据存取	写寄存器	单周期总时间
Load类指令	200ps	100 ps	200ps	200ps	100 ps	800ps
Store类指令	200ps	100 ps	200ps	200ps		700ps
R型指令	200ps	100 ps	200ps		100 ps	600ps
beq	200ps	100 ps	200ps			500ps

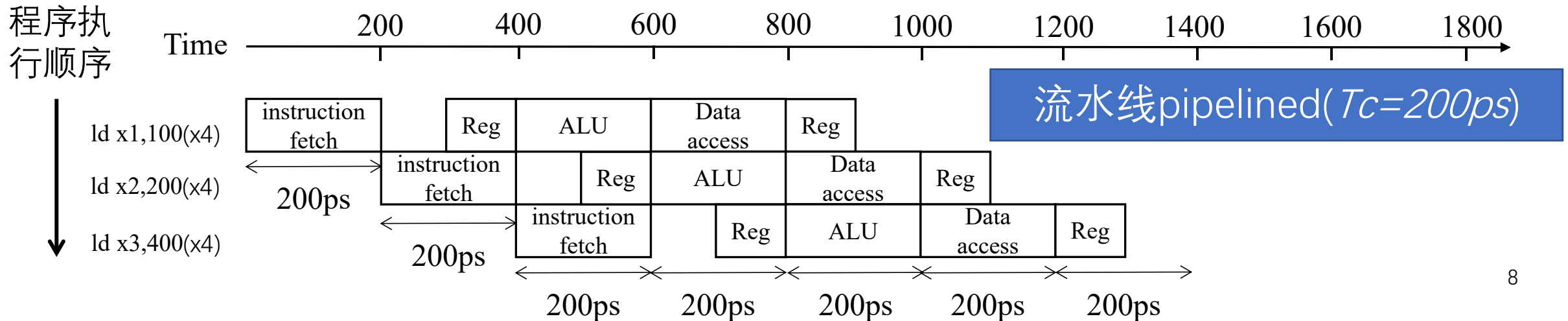
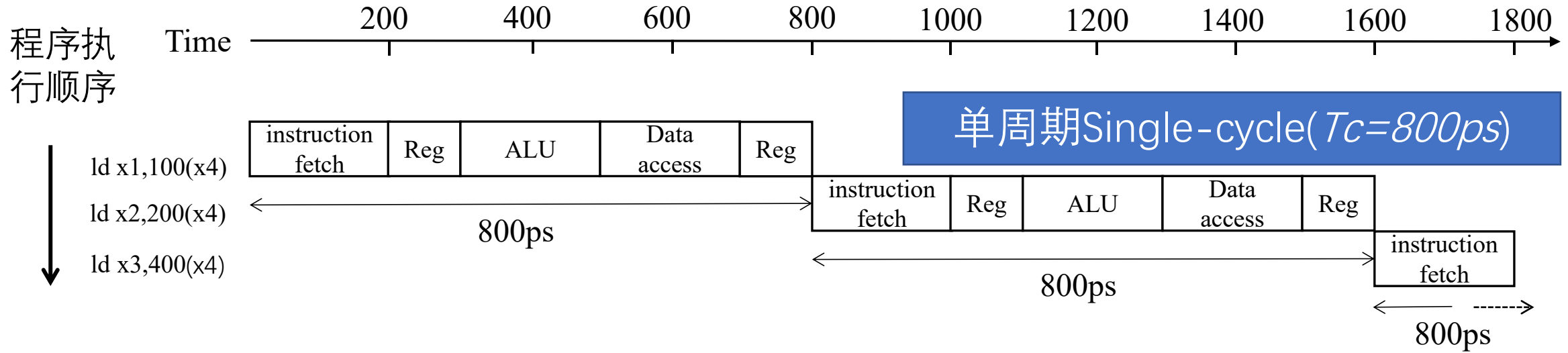
计算机各种延时示意图

性能之巅：洞悉系统、企业与云计算

<https://blog.csdn.net/jizhu4873/article/details/84341884>



单周期CPU vs 流水线CPU



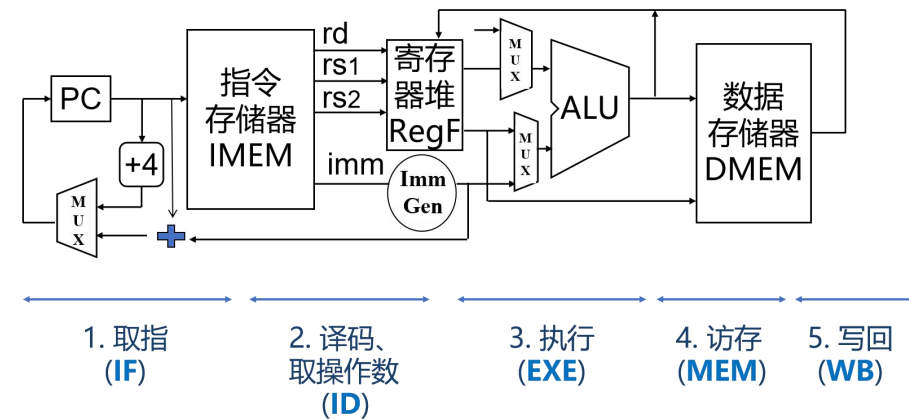
流水线加速比

- 如果流水线各阶段操作平衡
 - 例如：所有阶段需要相同的时间则：指令执行时间（流水线）
 $\approx$ 指令执行时间（非流水线） / 流水线级数
- 若各阶段不完全平衡，加速比会变小
- 通过**提高指令吞吐率**来提高性能
 - **单个指令的执行时间没有减少，而且还有可能增加。**

面向流水线的指令系统设计（RISC-V）

- 所有RISC-V指令长度相同，都是32 bits
 - 简化了流水线取指令和指令译码
 - x86: 1-17 bytes 变长指令不利于流水线实现

- 只有6种指令格式，格式整齐
 - 能在一个阶段内完成译码和读寄存器



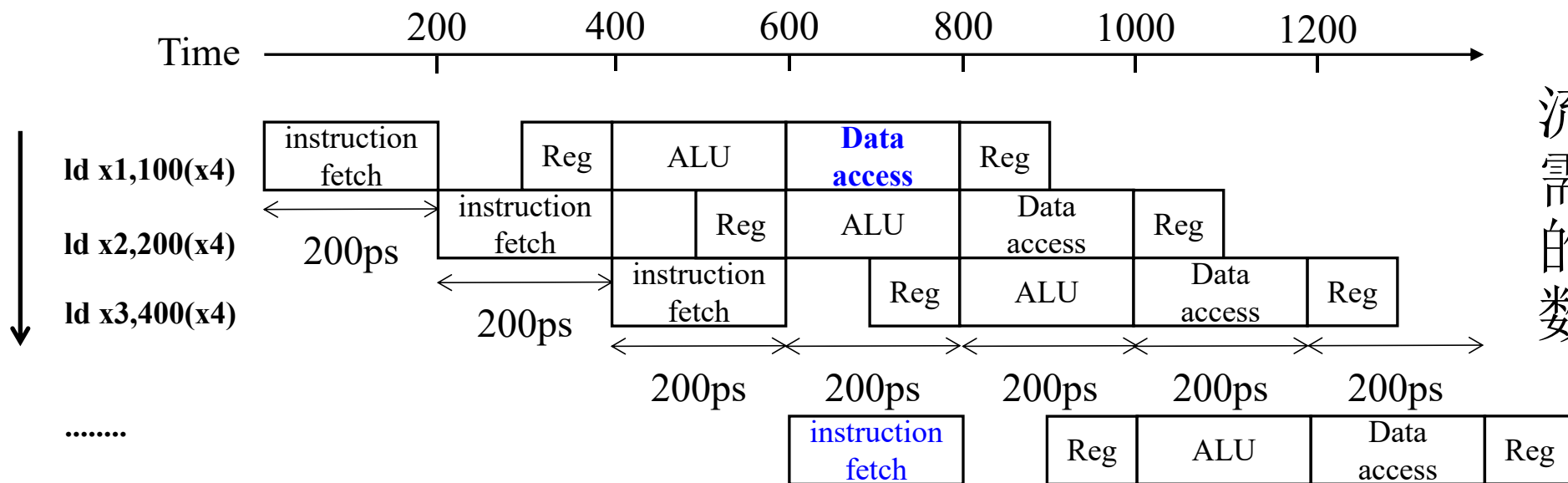
- 存储器操作只出现在load/store指令中
 - 1. 取指 (IF)
 - 2. 译码、取操作数 (ID)
 - 3. 执行 (EXE)
 - 4. 访存 (MEM)
- 利用执行阶段来计算存储器地址，然后在下一阶段访问存储器

流水线冒险(hazard)

- 冒险：有时在下一个时钟周期中无法**正确**执行相应指令
- 结构冒险
 - 因**缺乏硬件支持**而导致指令不能在预定时钟周期内正确执行。
- 数据冒险
 - 因**无法提供指令执行所需数据**而导致指令不能在预期的时钟周期内正确执行。
- 控制冒险（分支冒险）
 - 由于**取到的指令并不是所需要的**，或者**指令地址的流向不是所预期的**，导致**正确的指令无法在正确的时钟周期内执行**。

结构冒险

- 硬件资源不支持多条指令在同一时钟周期执行
- 假设下图所示流水线结构只有一个存储器(指令和数据共用)
 - load/store需要访问存储器
 - 如果有第四条指令, 则第一条指令从存储器取数据的同时, 第四条指令从同一存储器取指令, 流水线发生结构冒险

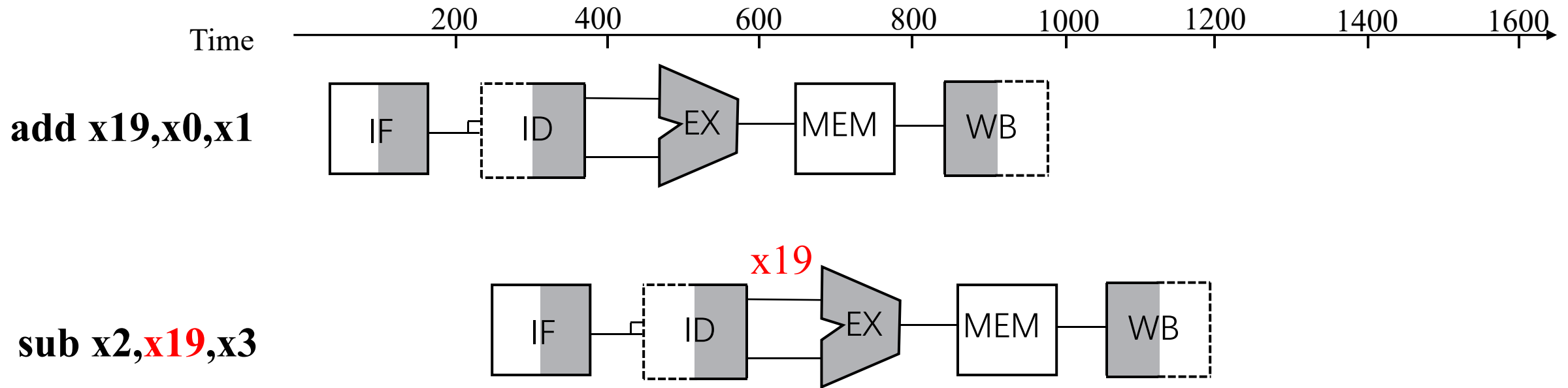


流水线数据通路
需要可独立访问
的指令存储器和
数据存储器

数据冒险

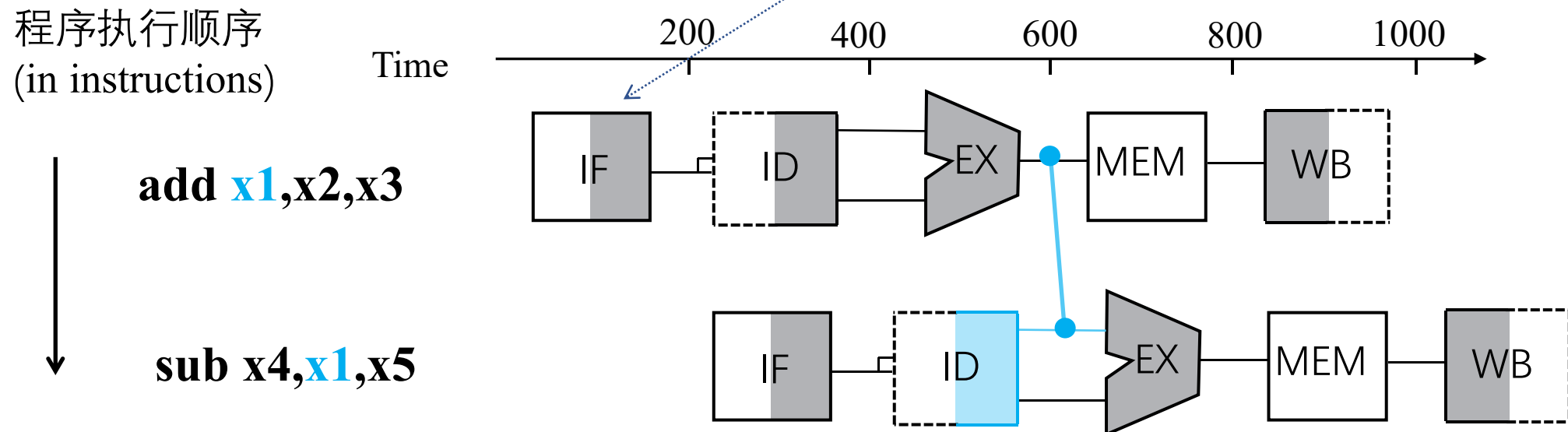
- 一条指令依赖于前面一条**尚在流水线中**的指令运行结果

add **x19**, x0, x1
sub x2, **x19**, x3

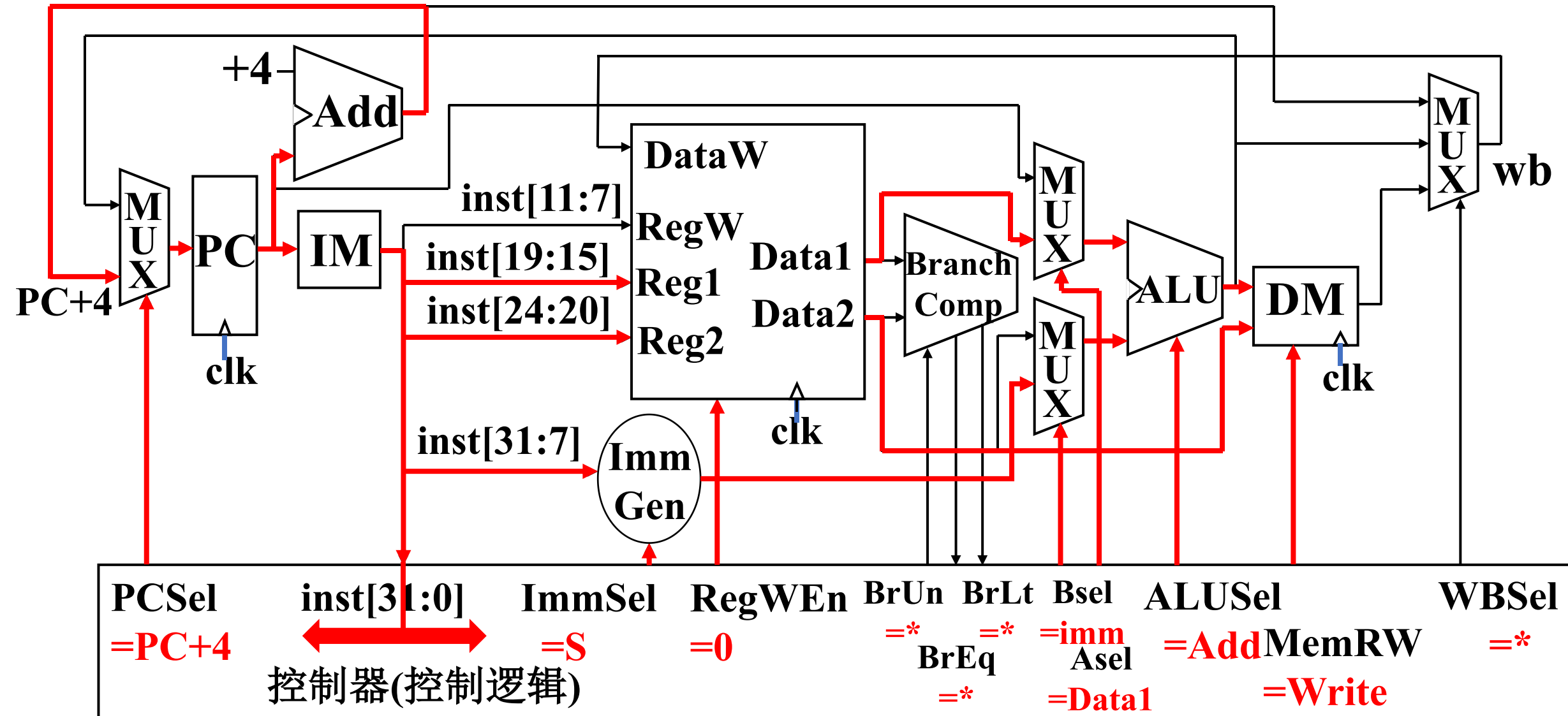


一种解决数据冒险的方法—前递forwarding

- ALU计算出结果，立刻传数据给后续指令使用，而不是等到数据到达寄存器或存储器
- 需要向内部资源添加额外的硬件
 - 寄存器堆或存储器右半部分为阴影：**读**寄存器堆或存储器
 - 寄存器堆或存储器左半部分为阴影，**写**寄存器堆或存储器

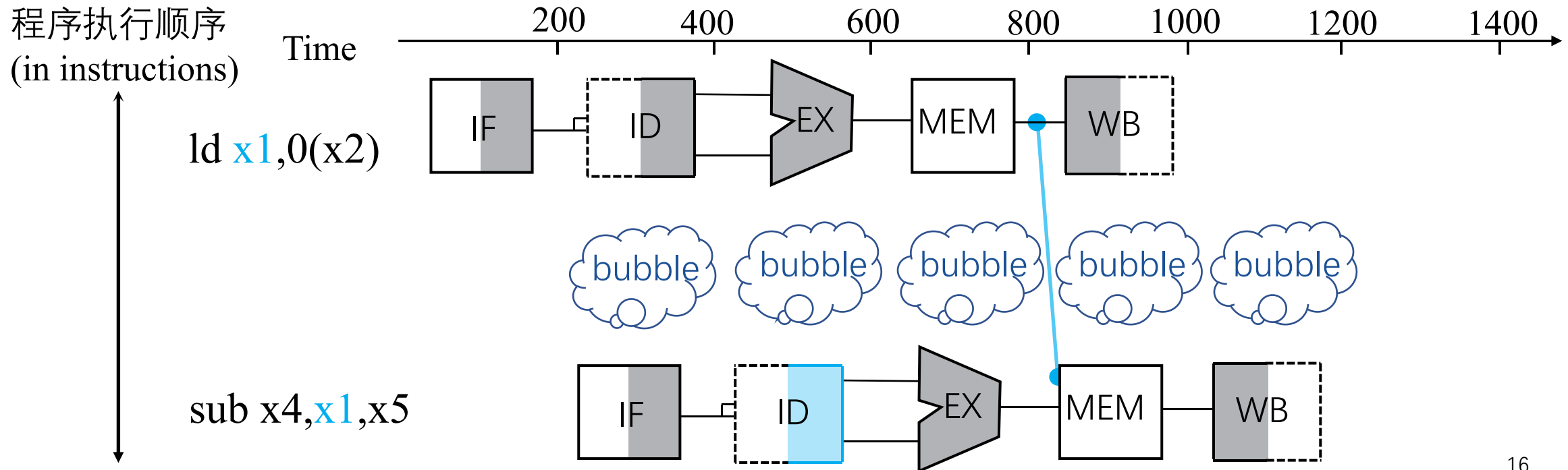


sd指令的执行



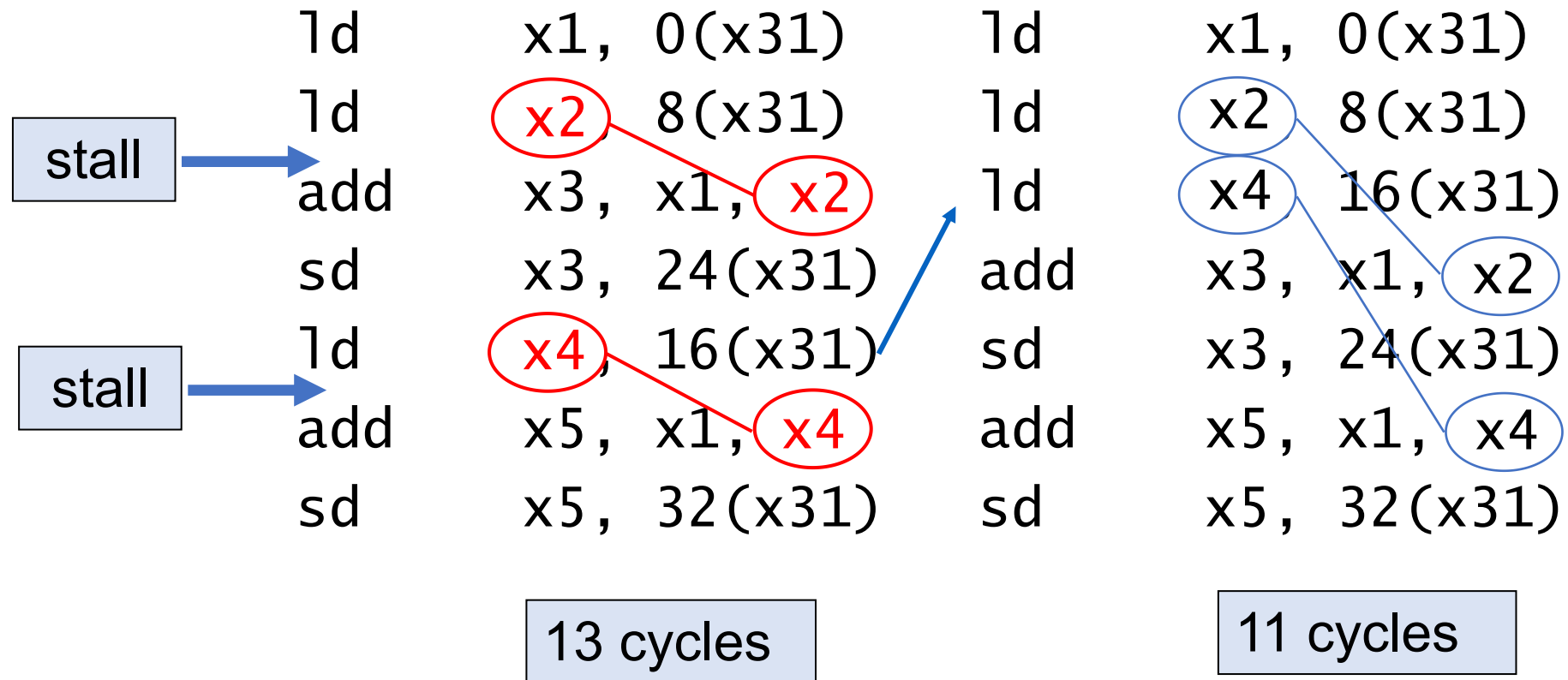
载入-使用型数据冒险(load-use data hazard)

- 前递不能避免所有的流水线停顿
 - 当载入指令要取的数据还没被取回，而后续指令却需要该数据
 - 一种解决方案：流水线停顿(stall)，也称为气泡(bubble)，为了解决冒险而实施的一种阻塞



重排代码以避免流水线停顿

- C code: $a = b + e$; $c = b + f$;
- 假设流水线CPU中有前递机制

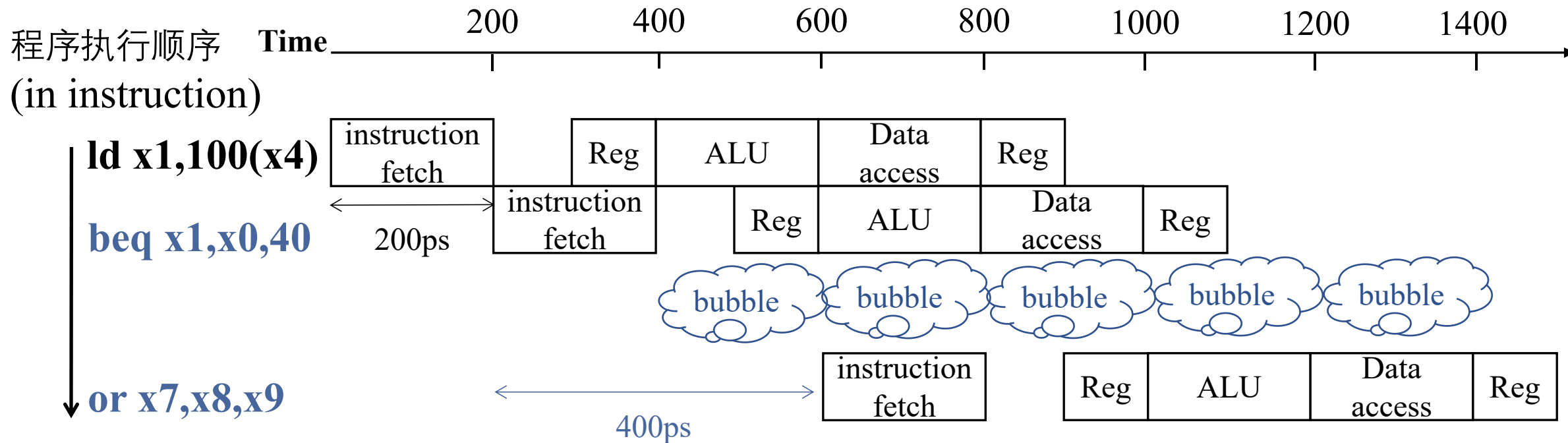


控制冒险

- 分支决定了控制流
 - 分支指令之后的IF阶段依赖于分支的结果
 - 流水线无法保证永远取到正确的指令
 - 此时，分支指令仍在ID阶段
- 在RISC-V流水线设计中
 - 尽可能早地完成寄存器比较、分支目标地址计算
 - 添加硬件，使得以上计算能在ID阶段完成

1 每遇到条件分支指令就停顿的流水线

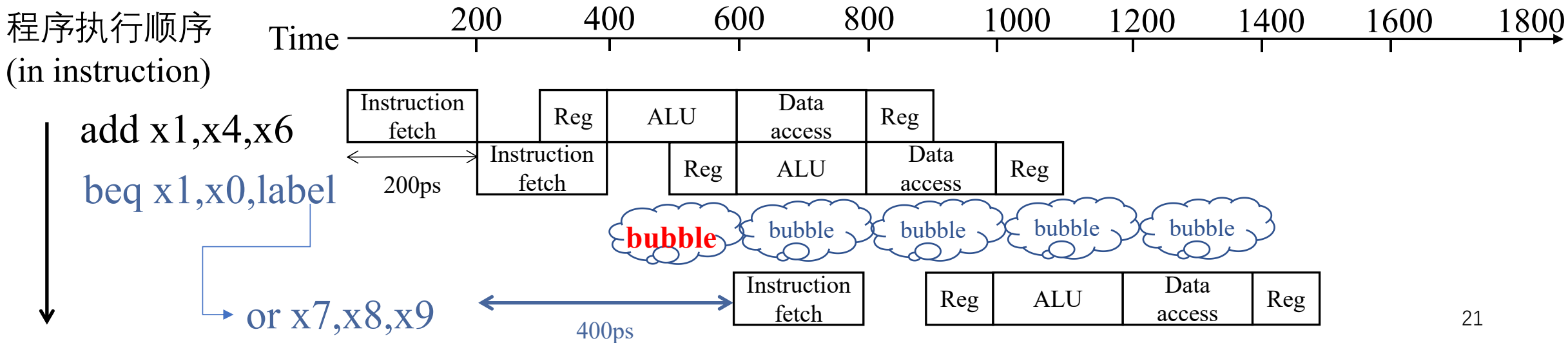
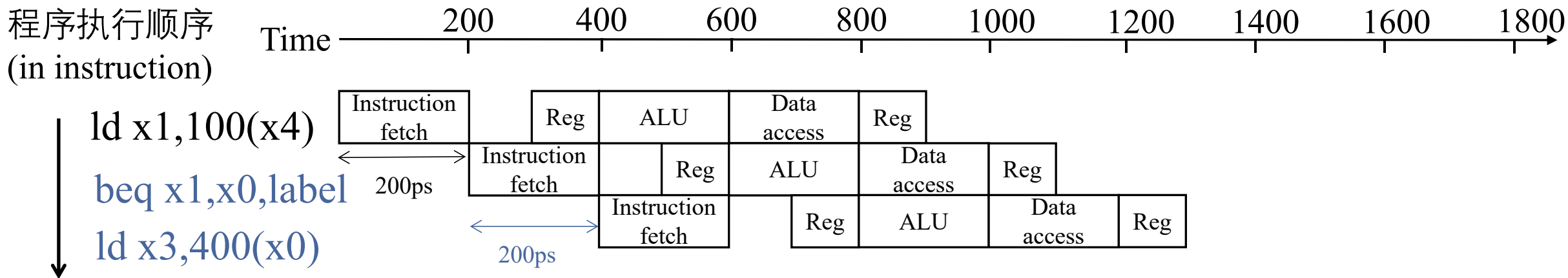
- 在取下一条指令之前，等待分支结果
 - 假设beq指令的ID阶段得到分支结果(需要额外的硬件支持)



2 分支预测

- 对较长流水线而言，通常无法在第二阶段解决分支指令问题
 - 每个条件分支都停顿带来的代价太大，不可接受
- 采用**预测**来处理条件分支
 - 只有预测错误时，才引发停顿
- 可以总是预测分支指令不跳转
 - 在分支指令之后，立即取指，流水线全速前进
 - 分支指令发生跳转时，流水线才会停顿

总是预测条件分支不发生跳转



更成熟的分支预测

- 静态分支预测
 - 根据典型的分支行为来决定是否跳转
 - 例如：循环+条件指令产生的分支
 - 在计算机程序中，循环底部是条件分支指令，并会跳转回循环顶部
 - 经常发生分支并向回跳转，所以可预测发生分支并跳到靠前的地址处
- 动态分支预测
 - 根据每个分支指令的行为进行预测
 - 一种常用实现方式：**保存**每个条件分支是否发生分支的**历史记录**
 - **假设未来的行为会延续这个趋势**
 - 当预测错误时，需要停顿、重新取指令，并更新分支历史记录

流水线总结

- 流水线通过提高吞吐率来提升性能
 - 并行执行多条指令
 - 每条指令拥有相同的延迟
 - 延迟：执行单条指令的时间
- 会受到冒险的影响
 - 结构冒险、数据冒险、控制冒险
- 指令集的设计影响到流水线实现的复杂度
 - （RISC-V）指令长度相同、指令格式整齐、访存操作只发生在load/store指令中

下面的程序在流水线处理器中执行时，程序 [填空1] 无需前递和停顿，程序 [填空2] 需要停顿、 [填空3] 可使用前递避免停顿。

程序1

```
ld x10, 0(x5)
add x11, x10, x10
```

程序2

```
add x11, x10, x10
addi x12, x10, 5
addi x14, x11, 5
```

程序3

```
addi x12, x10, 5
addi x14, x10, 5
addi x16, x10, 7
addi x13, x10, 9
addi x17, x10, 8
addi x15, x10, 15
```

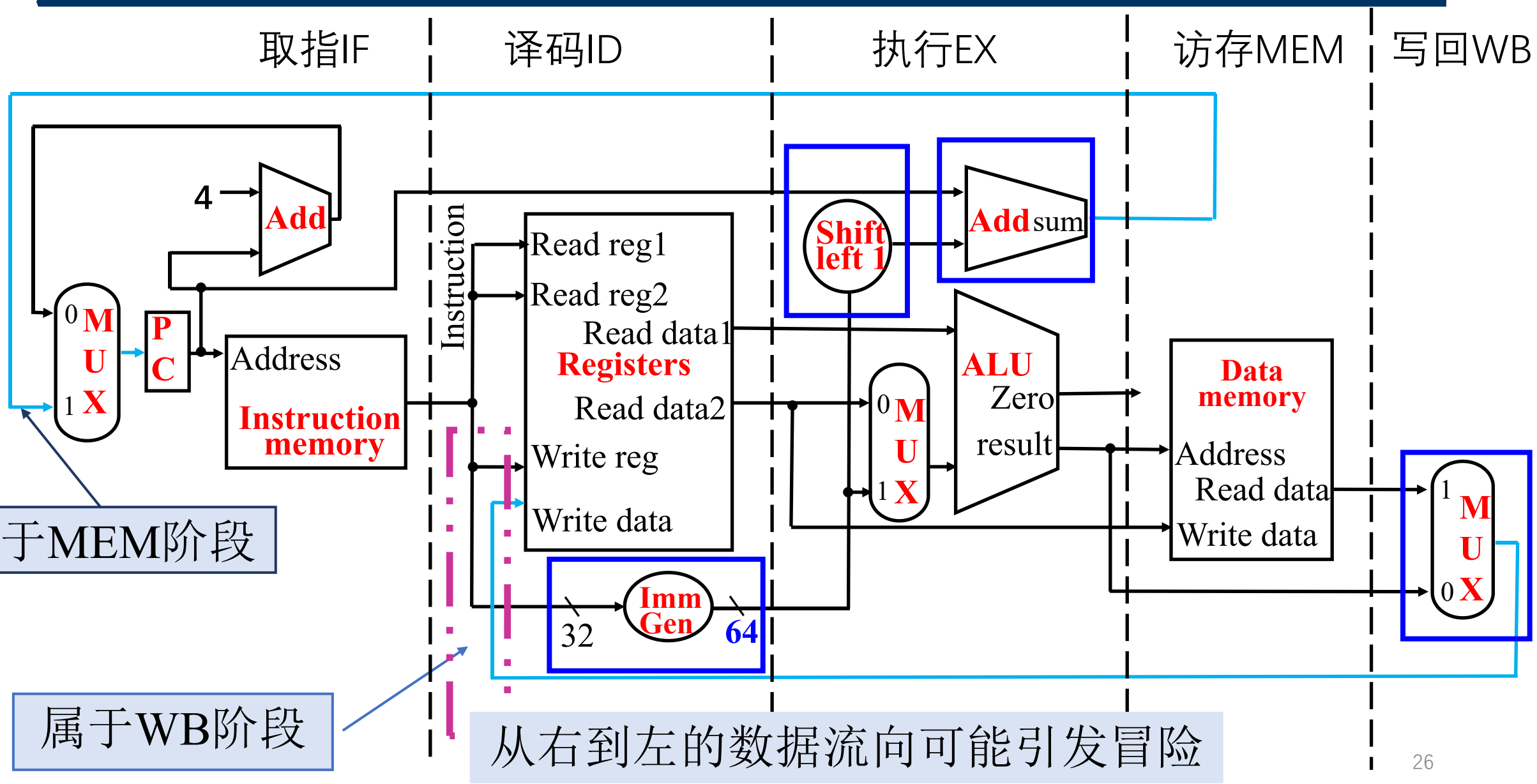
正常使用填空题需3.0以上版本雨课堂

作答

第五章

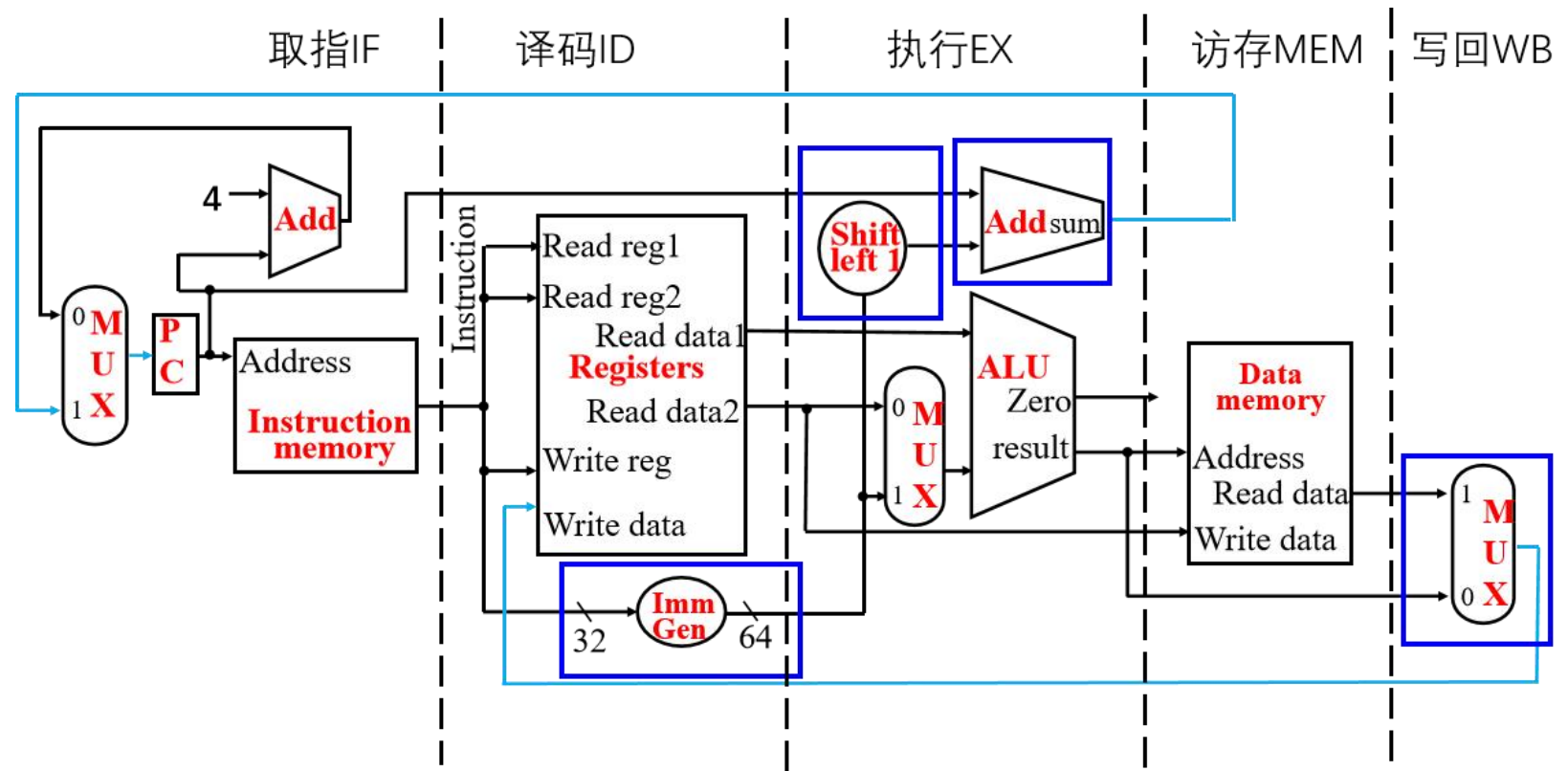
- 流水线概述
- 流水线数据通路和控制
- 解决数据冒险的方法：前递与停顿
- 控制冒险
- 例外

构建RISC-V流水线数据通路



RISC-V流水线数据通路中的特殊情况

- 写回阶段：将结果写回位于数据通路中段的寄存器堆
- 下一条指令PC的选择：自增PC值 or 后面阶段的分支地址

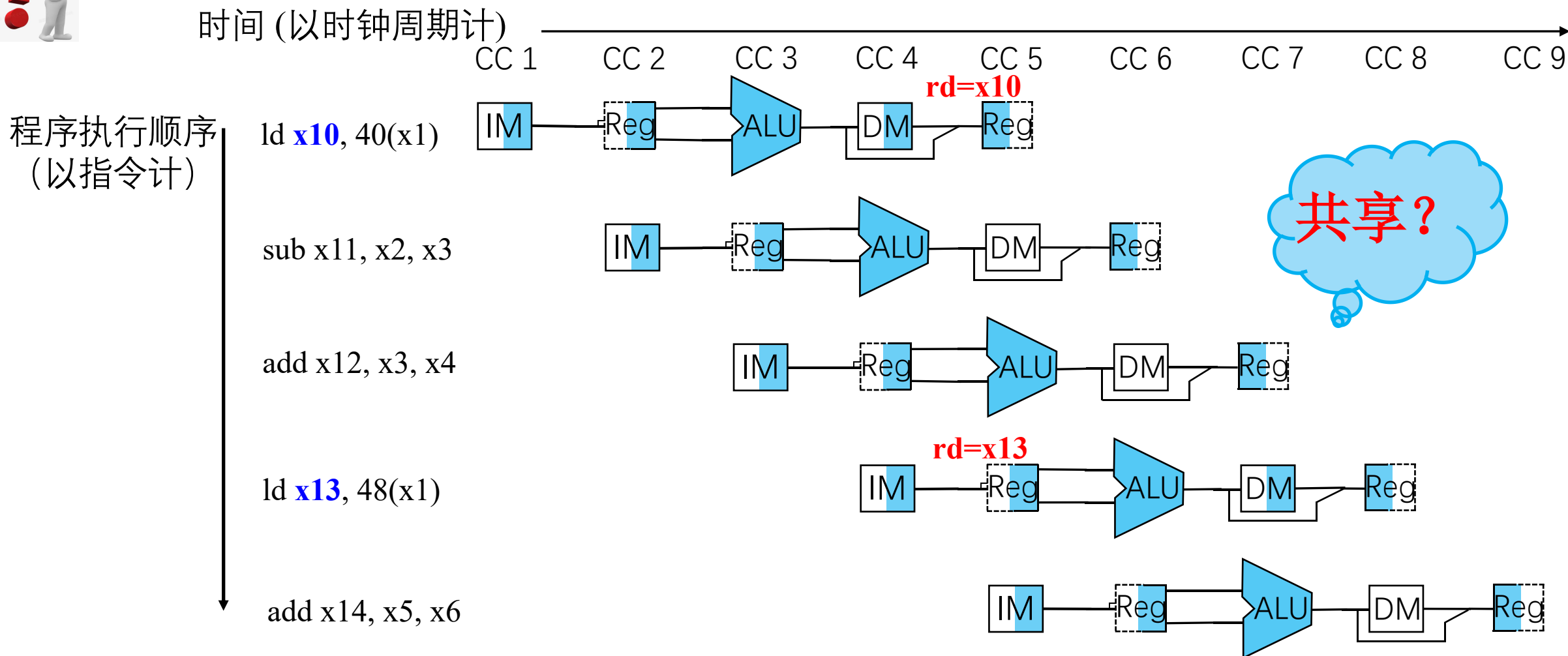


从右到左的数据流向可能引发冒险

流水线数据通路如何执行？

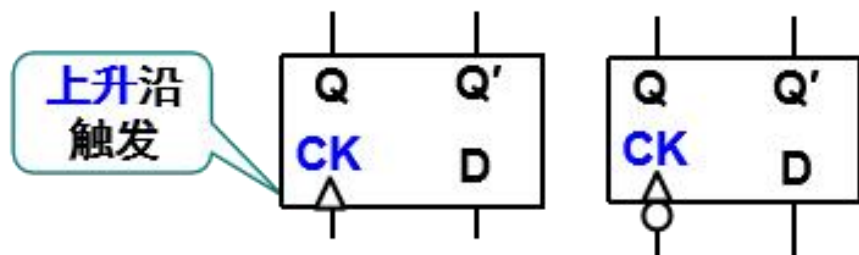


- 每一条指令都有一条独立的数据通路？



边沿触发器——D触发器

(1) 逻辑符号



(2) 功能表（上升沿为例）

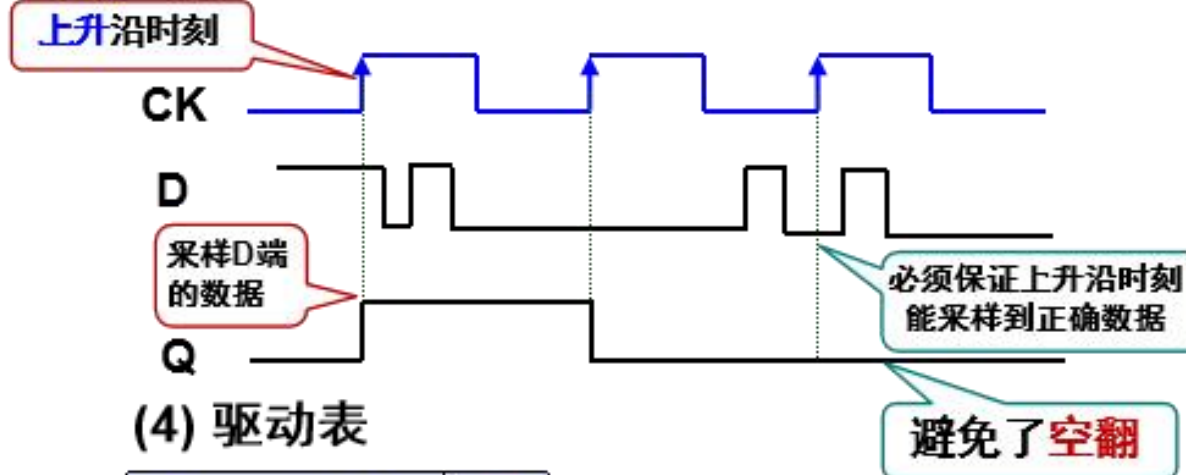
时钟端 CK	输入端 D	现态 Q_n	次态 Q_{n+1}
↑	0	0	0
↑	0	1	0
↑	1	0	1
↑	1	1	1

(3) 次态方程

$$Q^{n+1} = D$$

时钟触发器的特点

- ◆ 由时钟脉冲确定状态转换的时刻(即何时转换?)
- ◆ 由输入信号确定触发器状态转换的方向(即如何转换?)



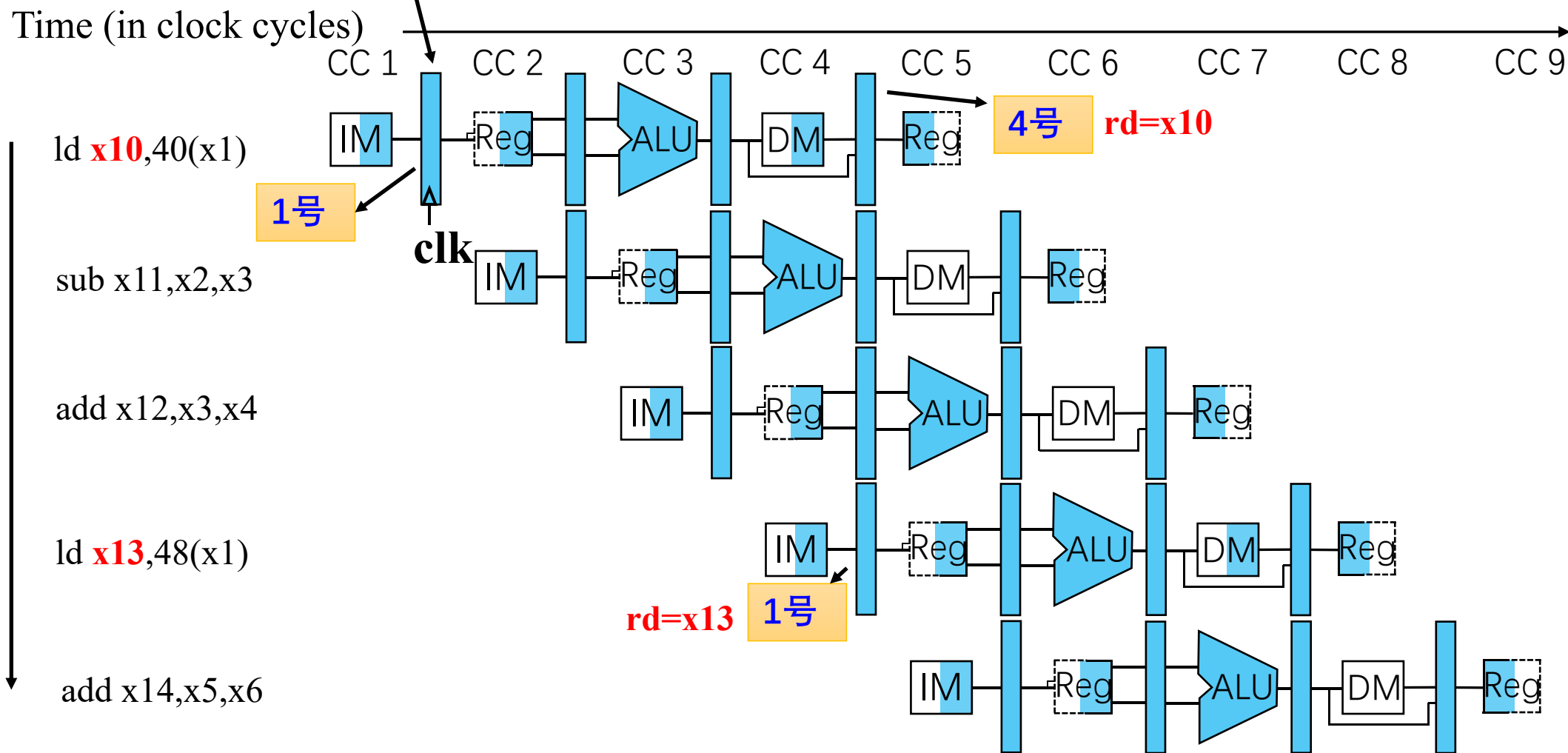
(4) 驱动表

Q_n	$\rightarrow$	Q_{n+1}	D
0	$\rightarrow$	0	0
0	$\rightarrow$	1	1
1	$\rightarrow$	0	0
1	$\rightarrow$	1	1

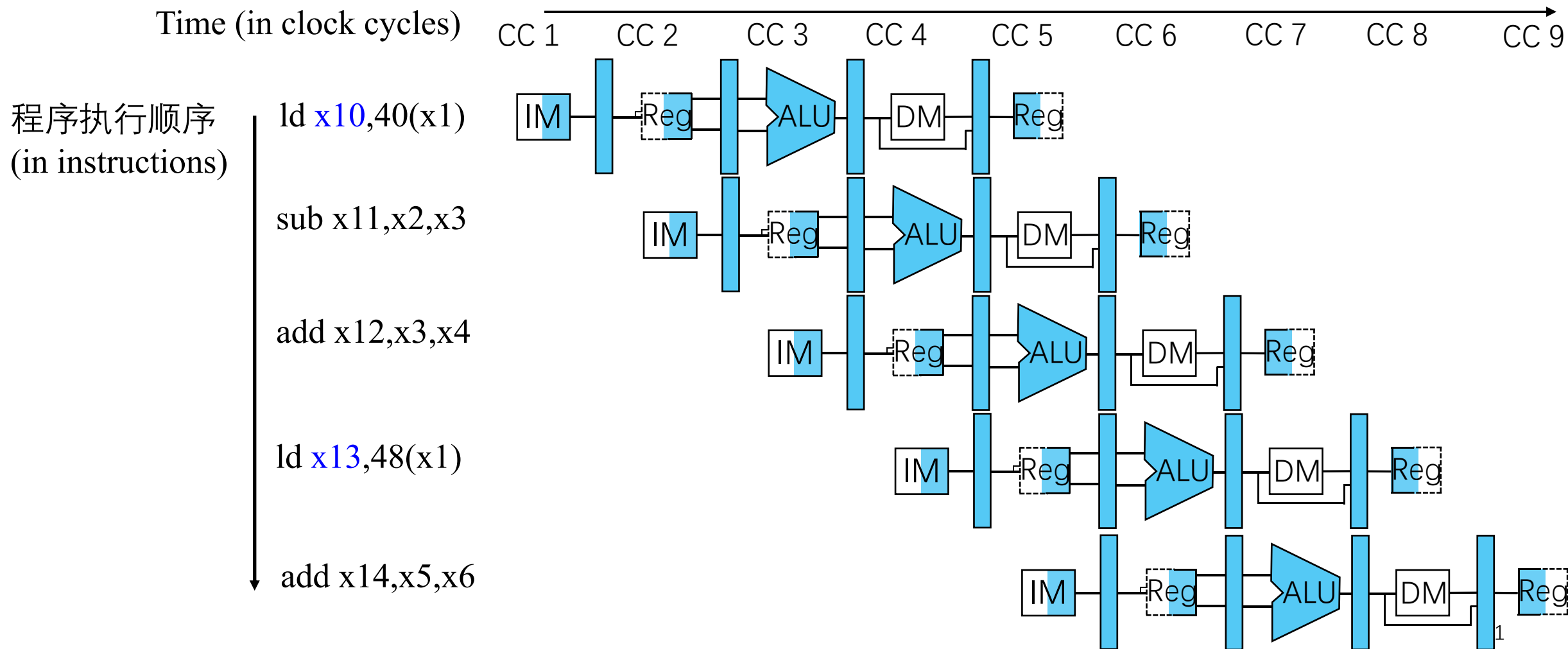
D触发器的特点:

最简单, 应用最广

采用流水线寄存器的流水线数据通路操作

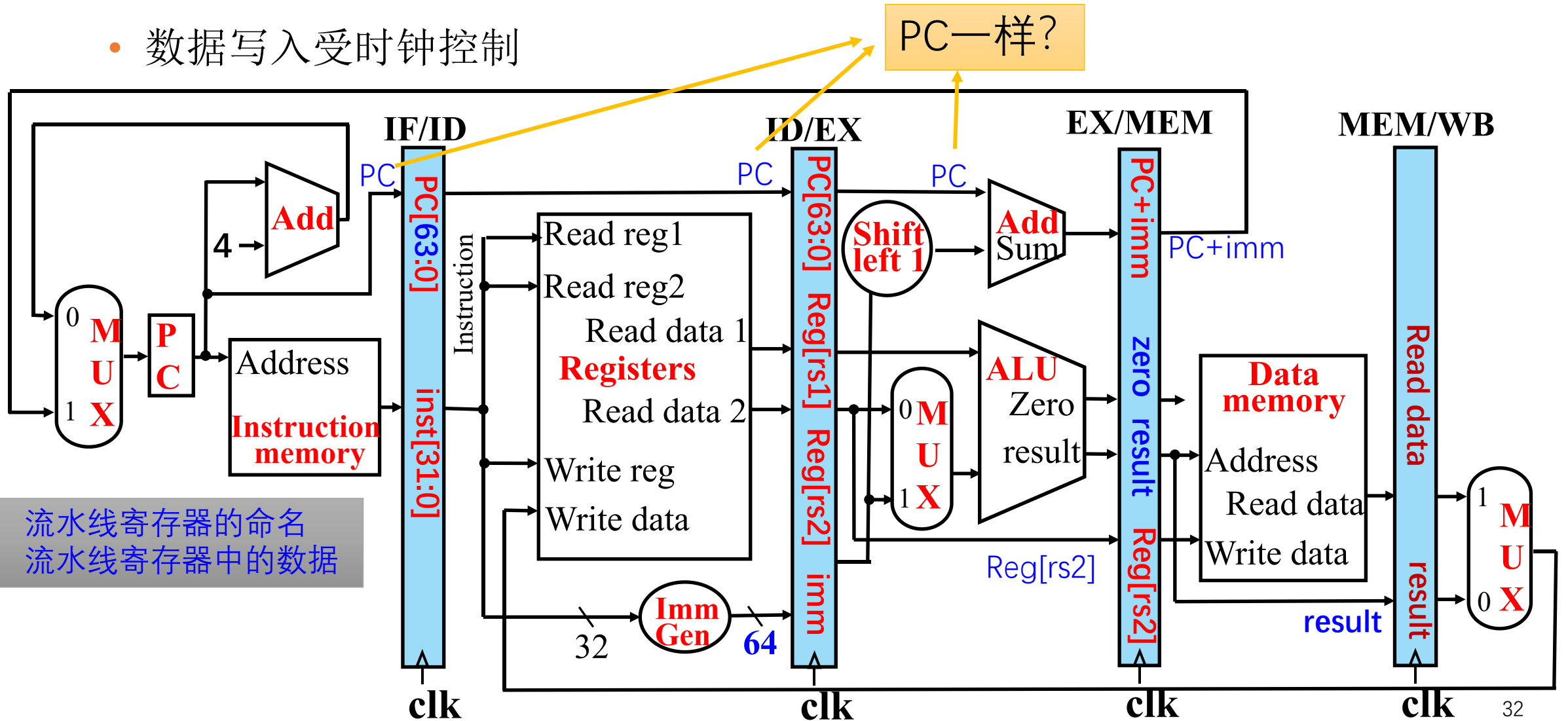


在流水线数据通路中观察指令的流动



加入流水线寄存器（Load类指令）

- 流水线寄存器：保存前一个阶段产生的信息
 - 数据写入受时钟控制

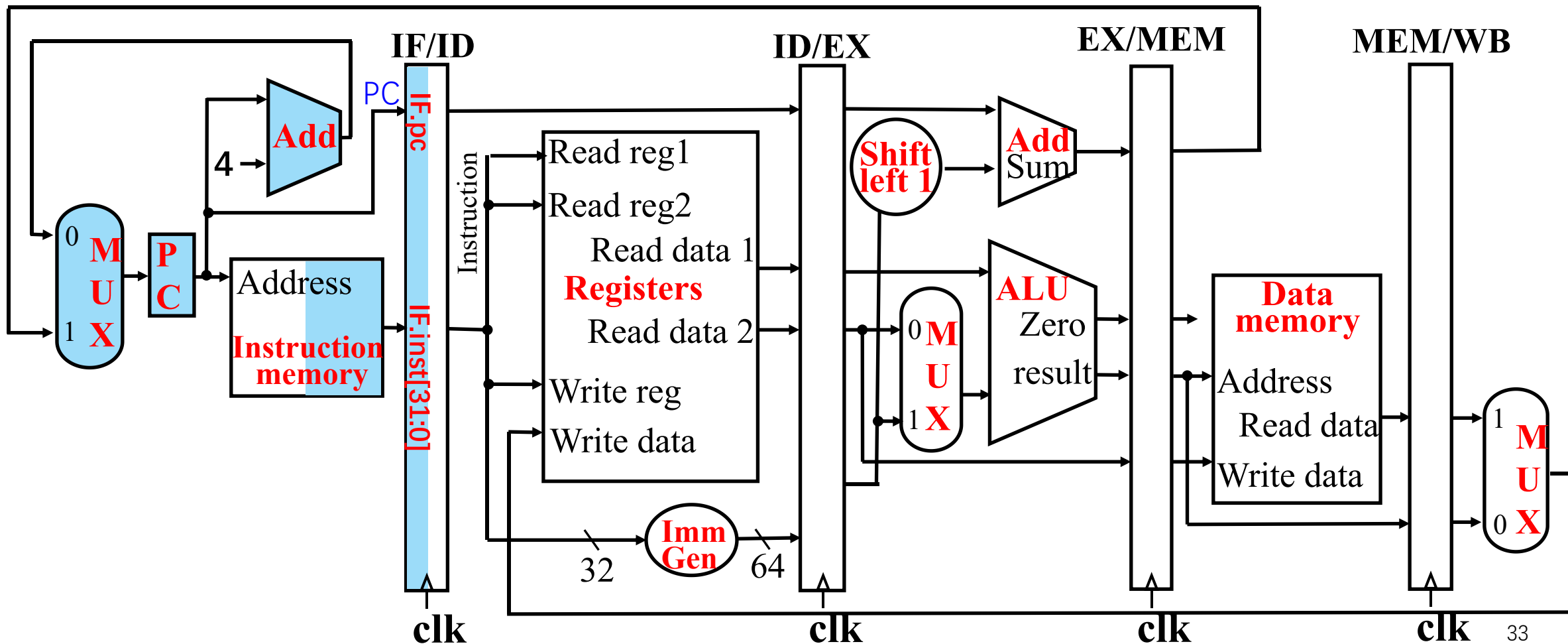


IF for Load, Store, ...

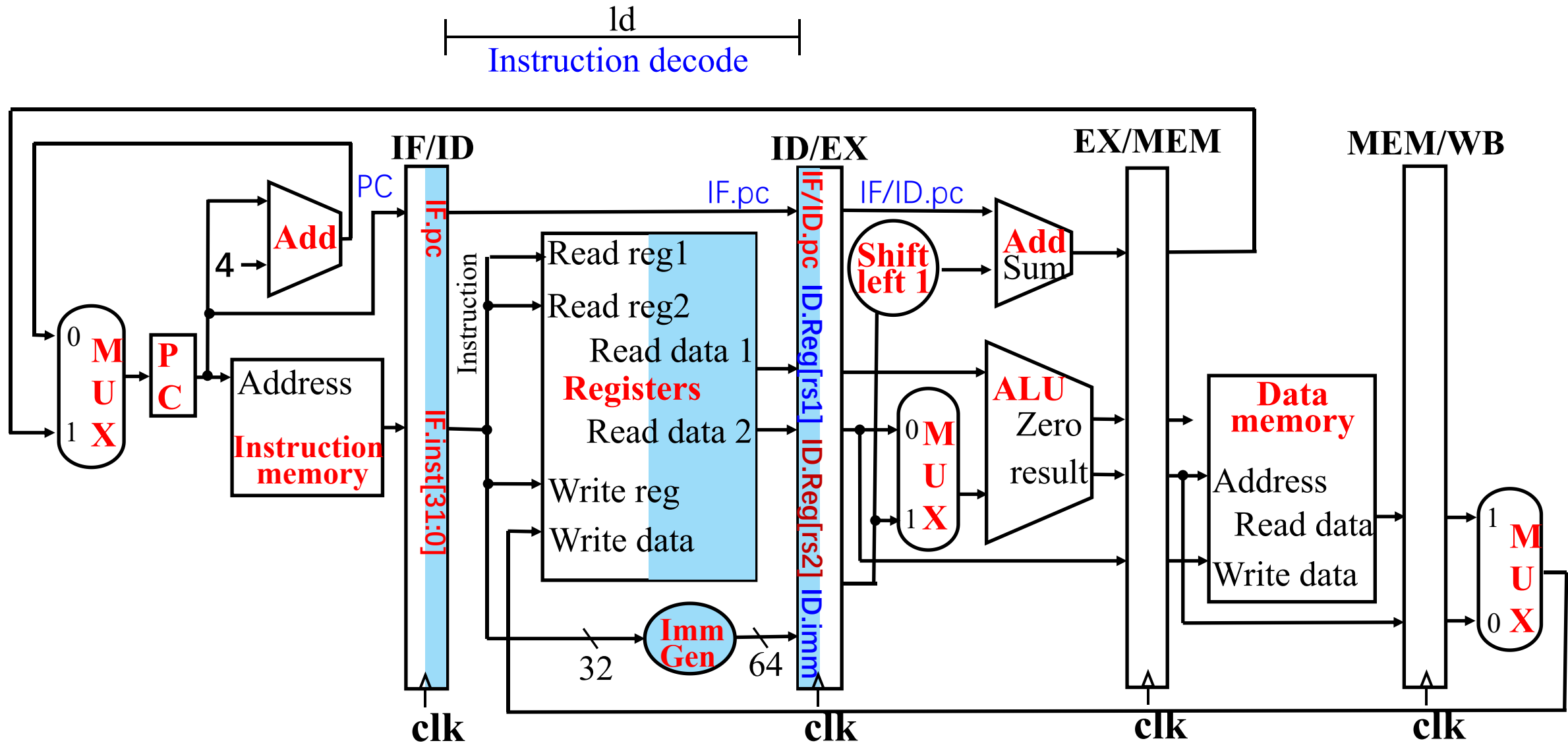
观察load和store指令的操作

ld
Instruction fetch

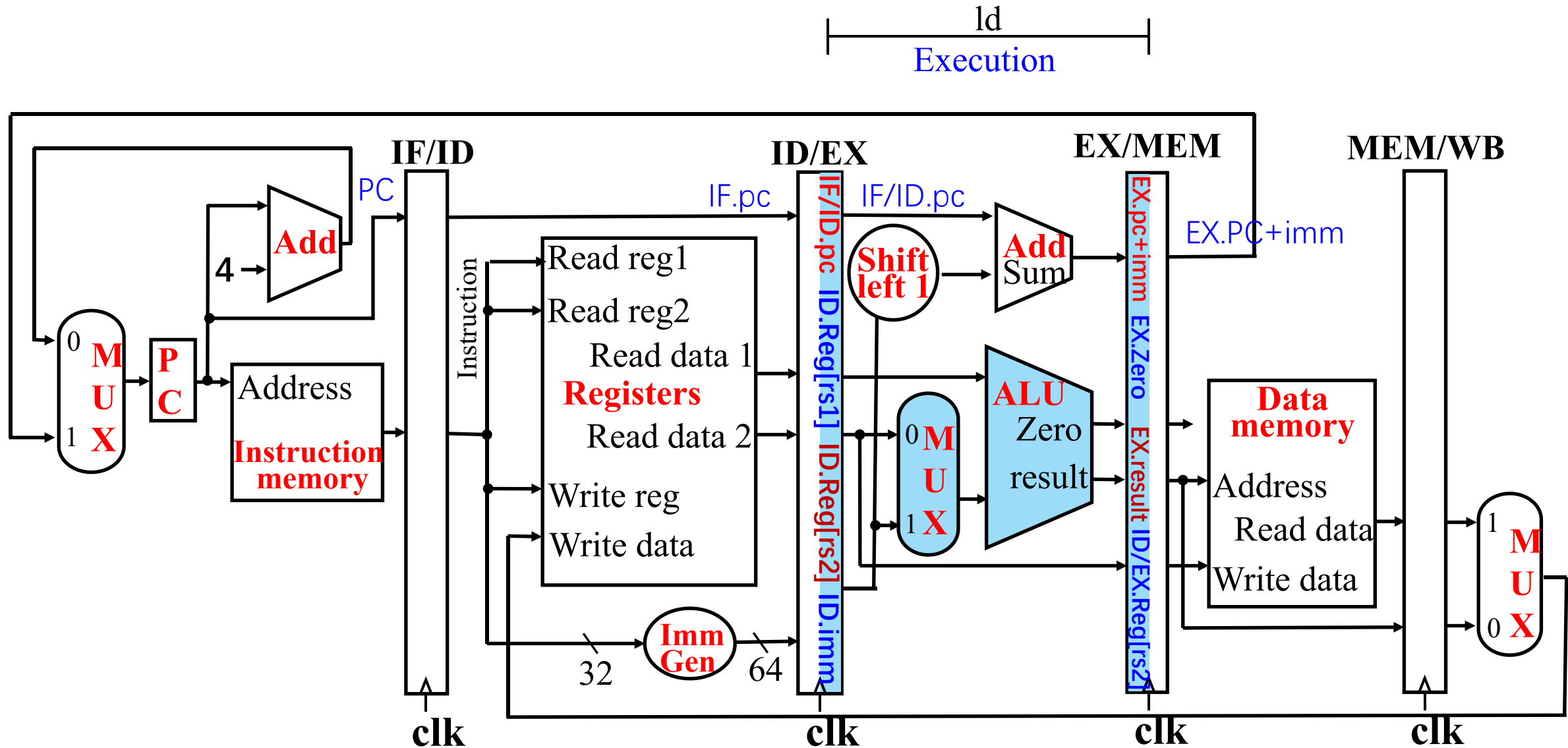
右半部分高亮: 寄存器或存储器被读取
左半部分高亮: 寄存器或存储器被写入



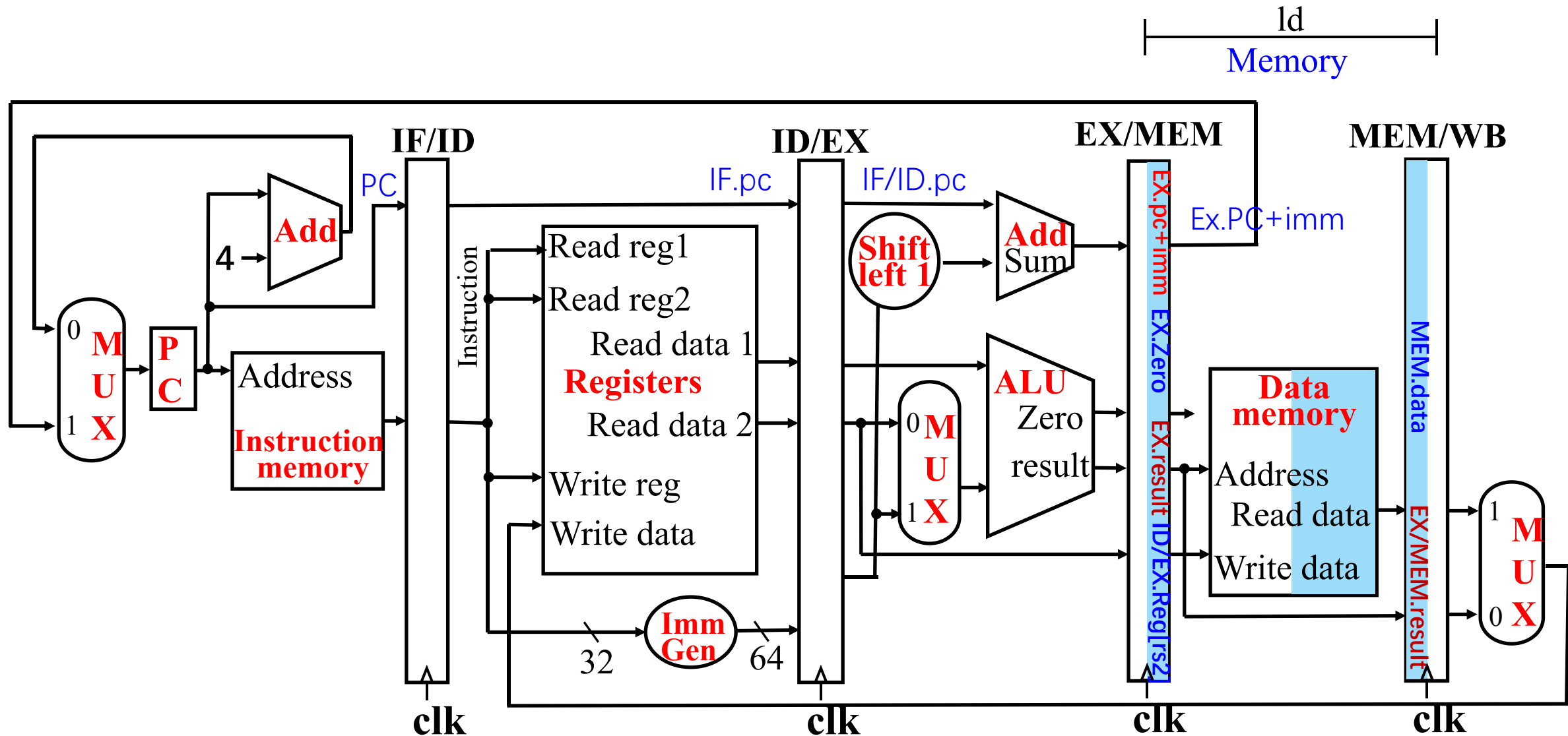
ID for Load, Store, ...



EX for Load



MEM for Load



WB for Load

ld x10, 40(x1)
 sub x11, x2, x3
 add x12, x3, x4
 ld x13, 48(x1)
 add x14, x5, x6

add x14, x5, x6

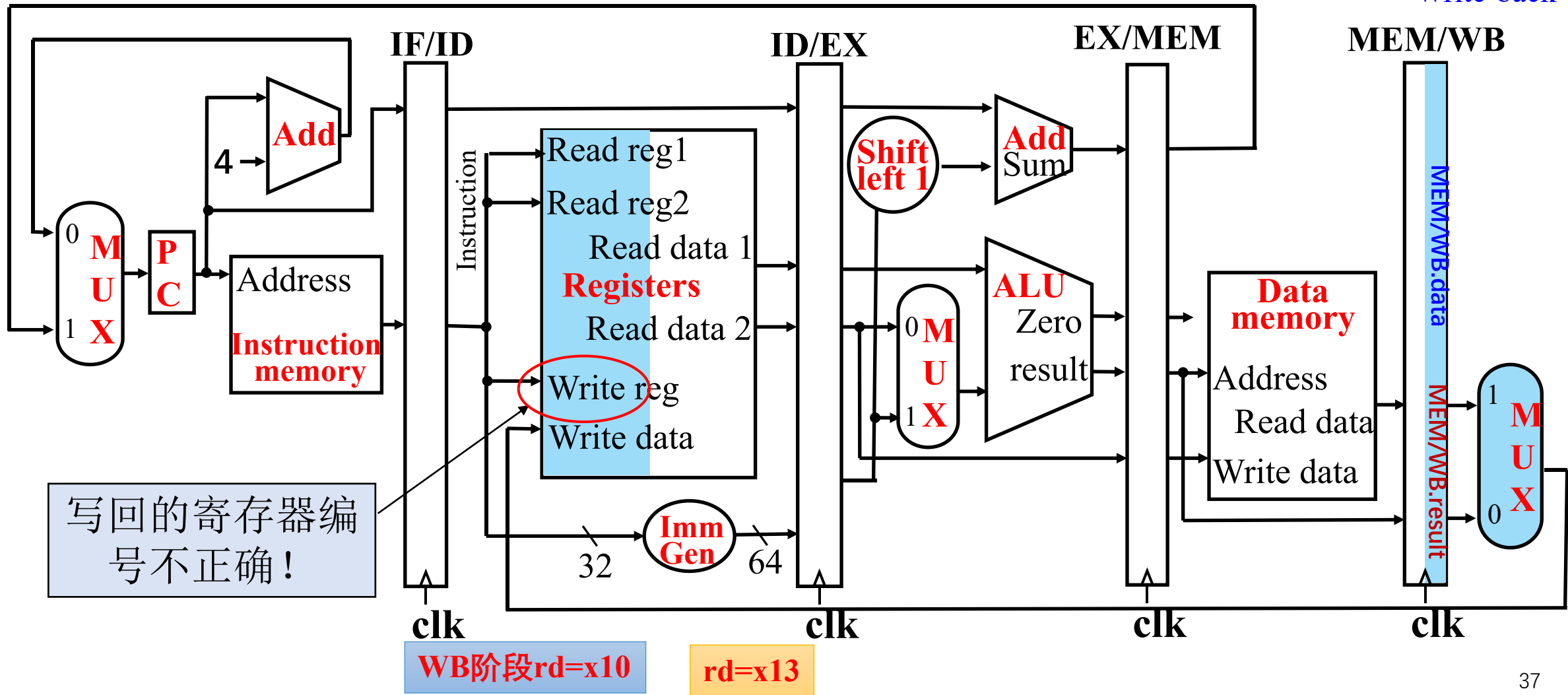
ld x13, 48(x1)

add x12, x3, x4

sub x11, x2, x3

ld x10, 40(x1)

ld
 Write-back



更正之后的数据通路

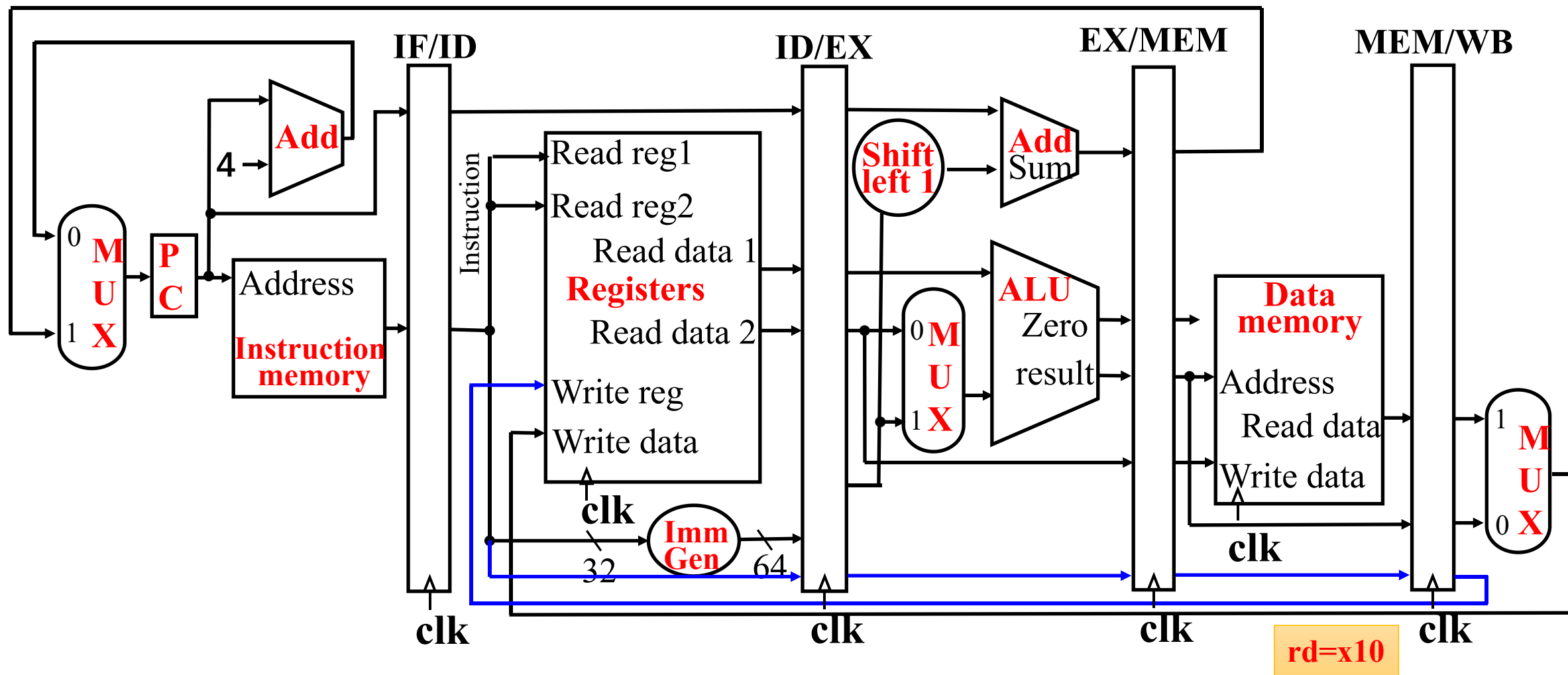
```
add x14,x5,x6
```

ld **x13**, 48(x1)

```
add x12, x3, x4
```

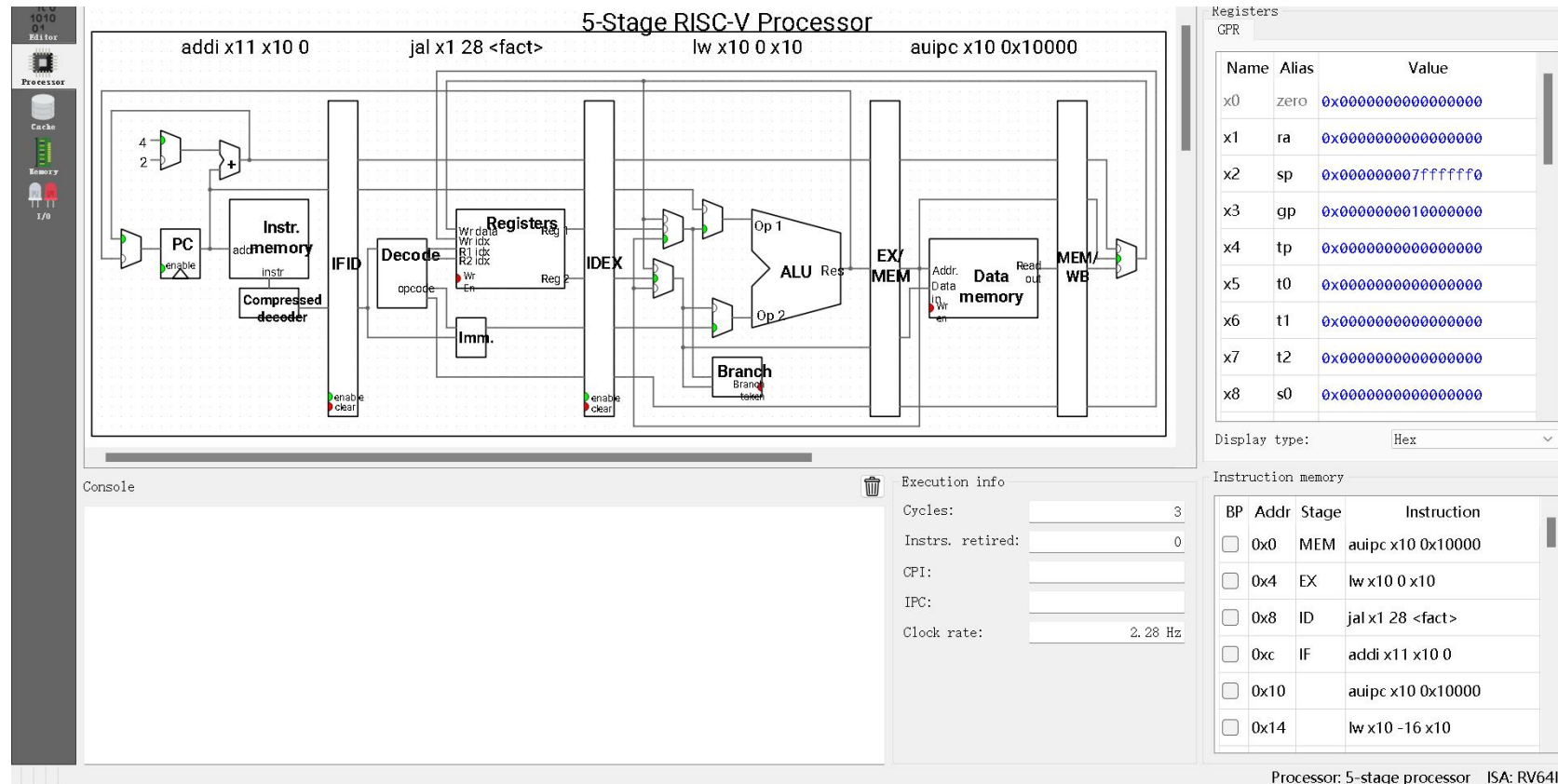
```
sub x11, x2, x3
```

ld **x10**, 40(x1)

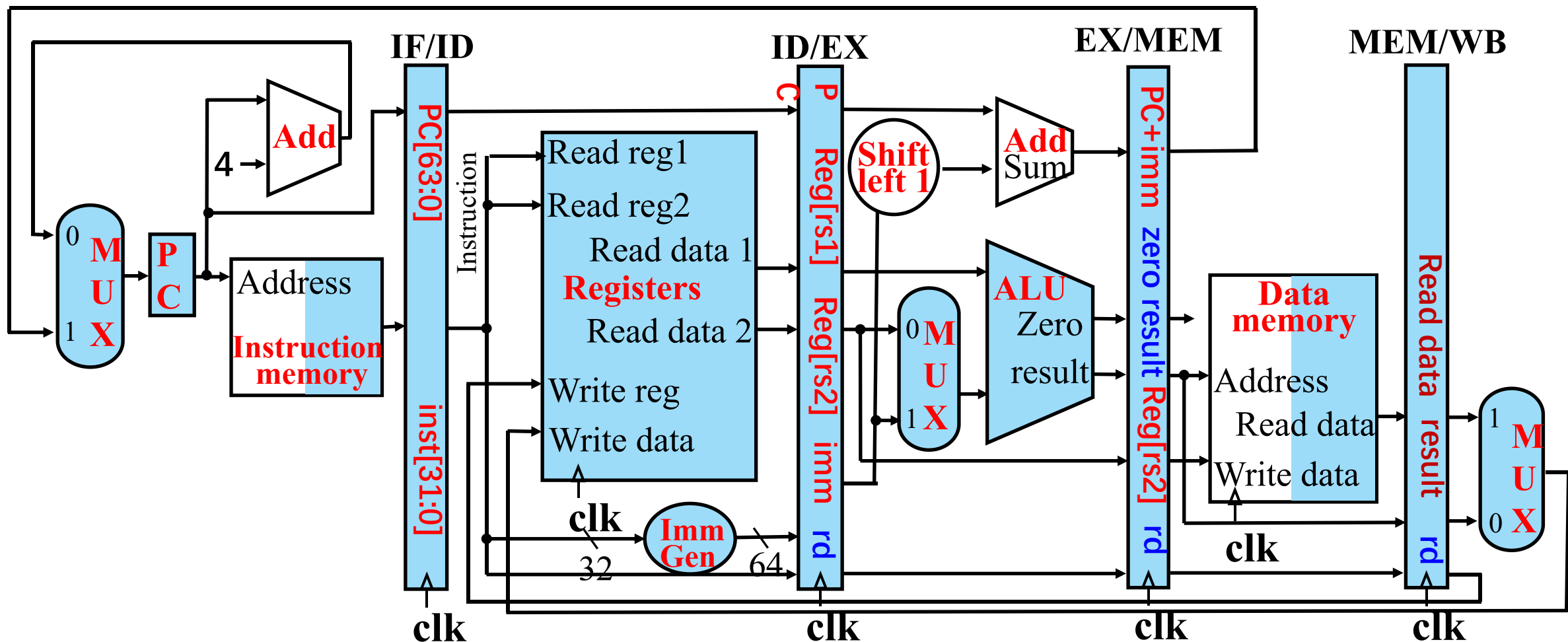


A processor simulator——Ripes

Ripes: a graphical **processor simulator** and **assembly code editor** built for the RISC-V instruction set architecture
(https://gitee.com/shzhou_admin/Ripes)

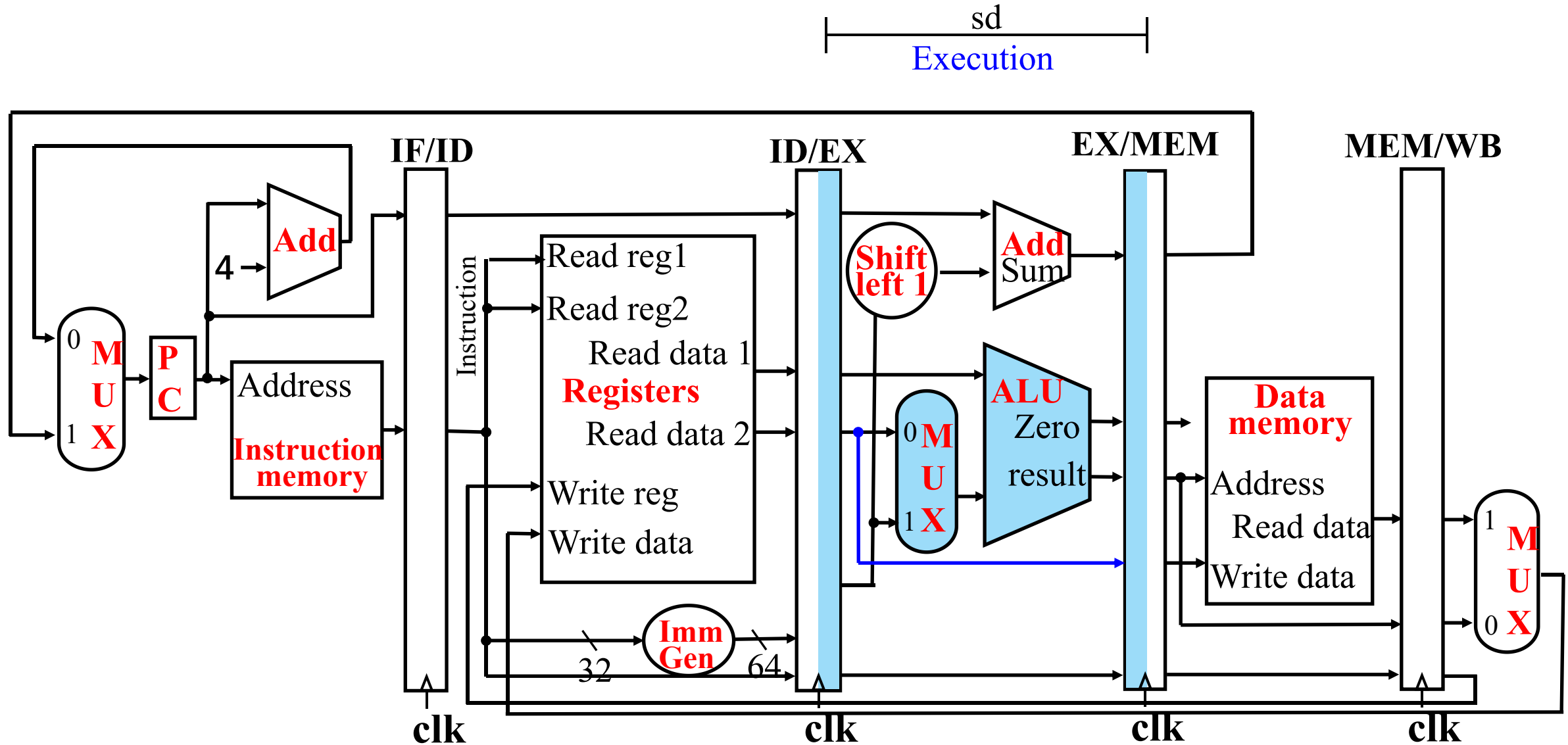


用于加载指令的全部五个阶段流水线

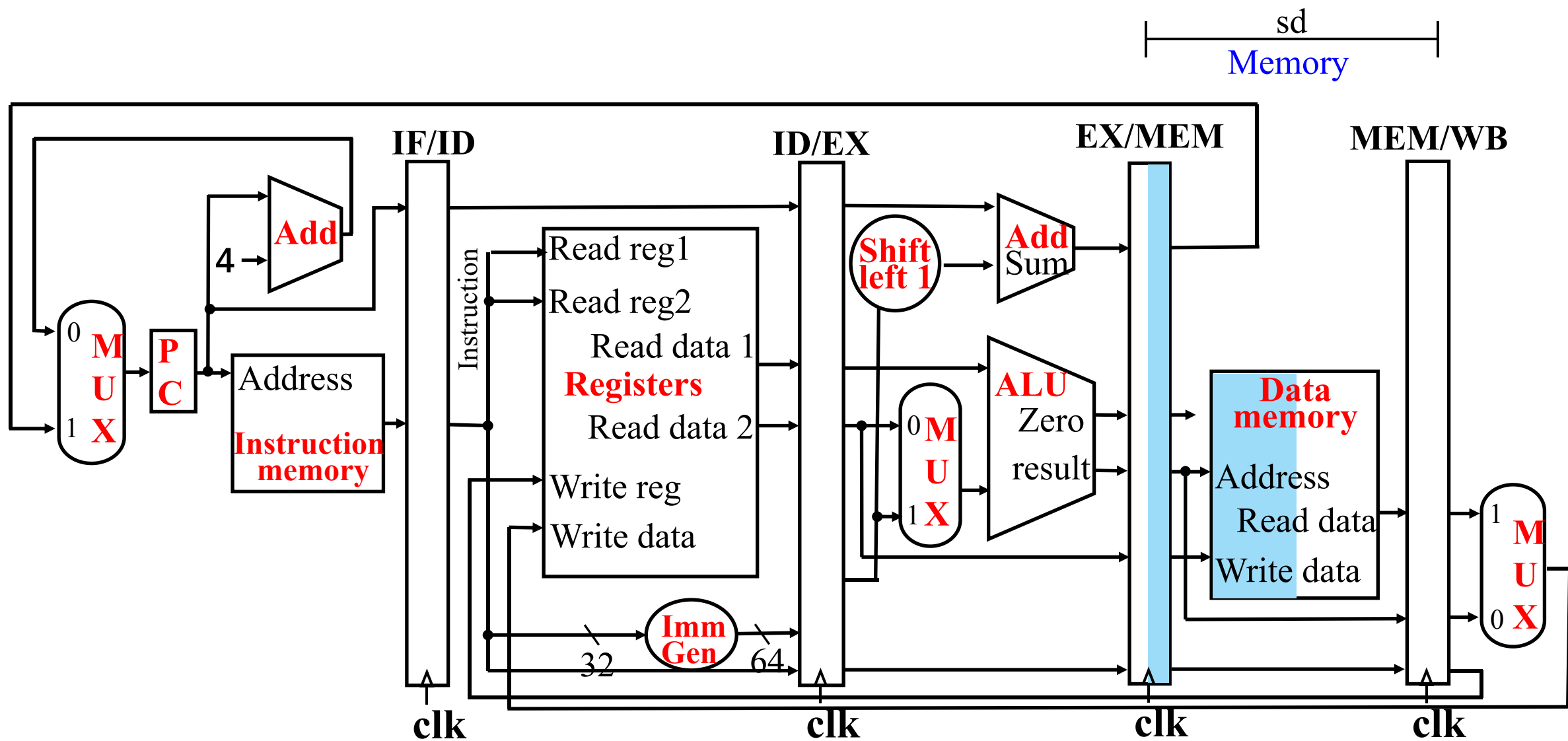


- 若要将相关信息从之前的流水线阶段传递到后续的流水线阶段，须将他们放置在**流水线寄存器**中。
- 在流水线通路设计中的每一个逻辑部件，只能在单个流水线阶段中被使用，否则会发生结构冒险。

EX for Store



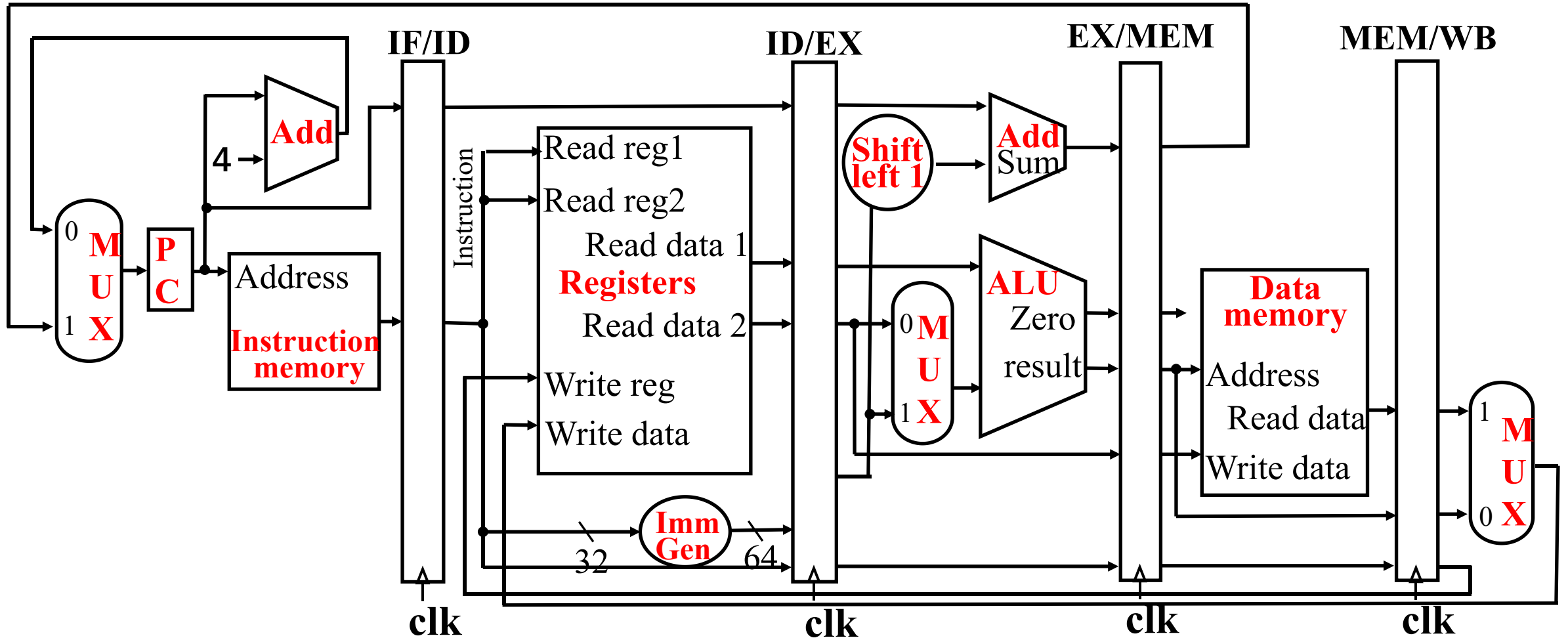
MEM for Store



WB for Store

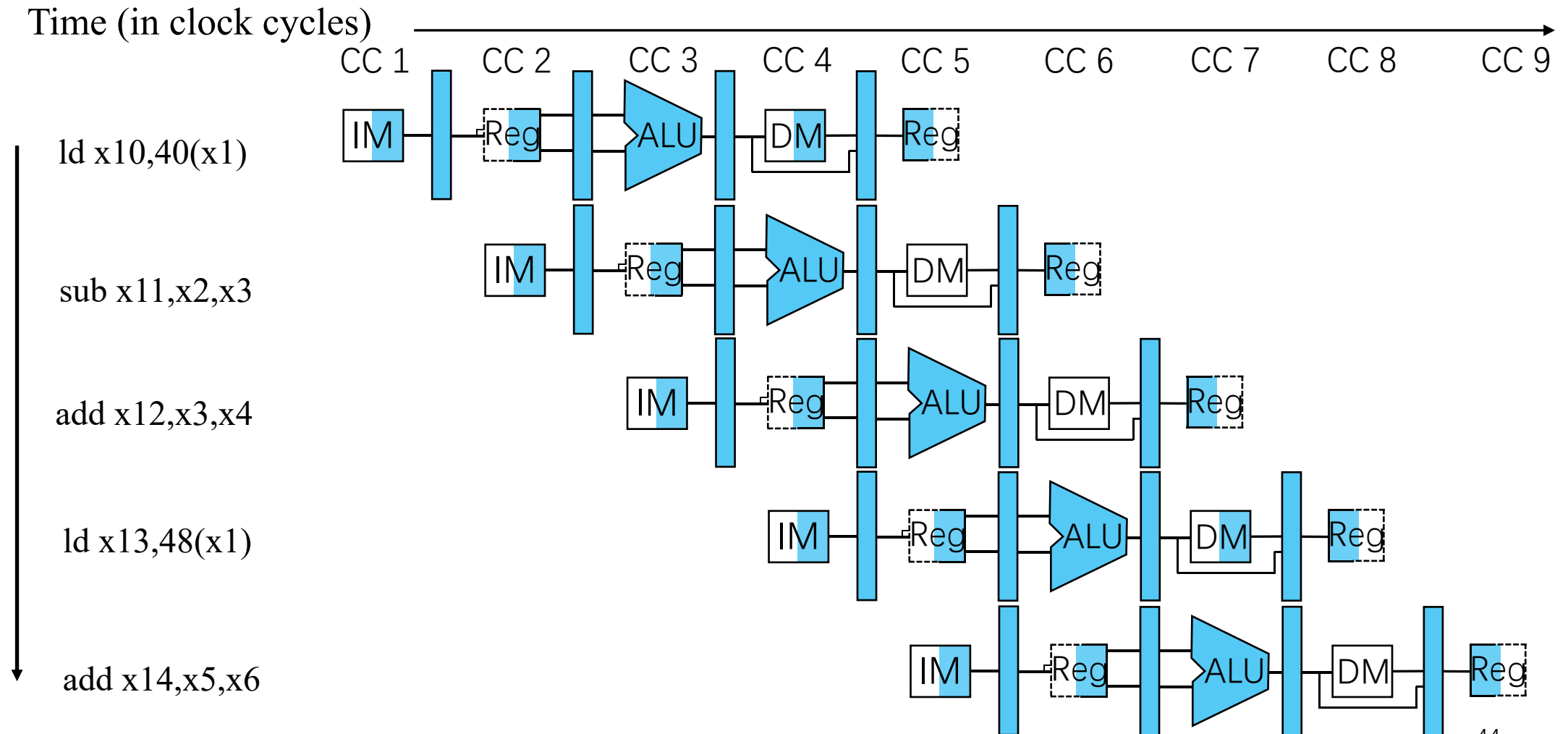
存储指令在“写回阶段”不执行任何操作。

sd
Write-back



多时钟周期流水线图

- 显示了每个流水线阶段中使用的**物理资源**



多时钟周期流水线图

用矩形块来命名每个流水线阶段

Time (in clock cycles)

CC1

CC2

CC3

CC4

CC5

CC6

CC7

CC8

CC9

ld x10,40(x1)

sub x11,x2,x3

add x12,x3,x4

ld x13,48(x1)

add x14,x5,x6

取指令

译码

执行

访存

写回

取指令

译码

执行

访存

写回

取指令

译码

执行

访存

写回

取指令

译码

执行

访存

写回

取指令

译码

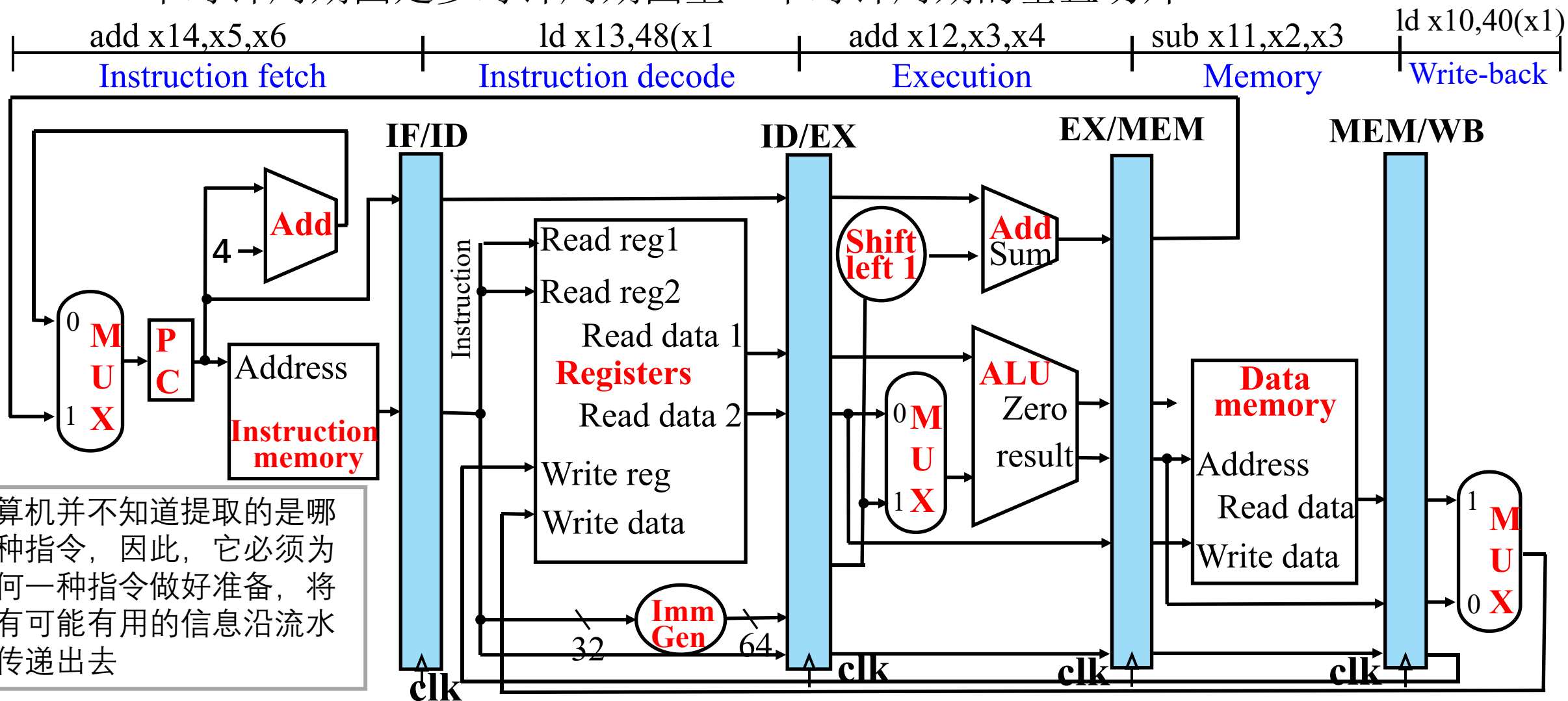
执行

访存

写回

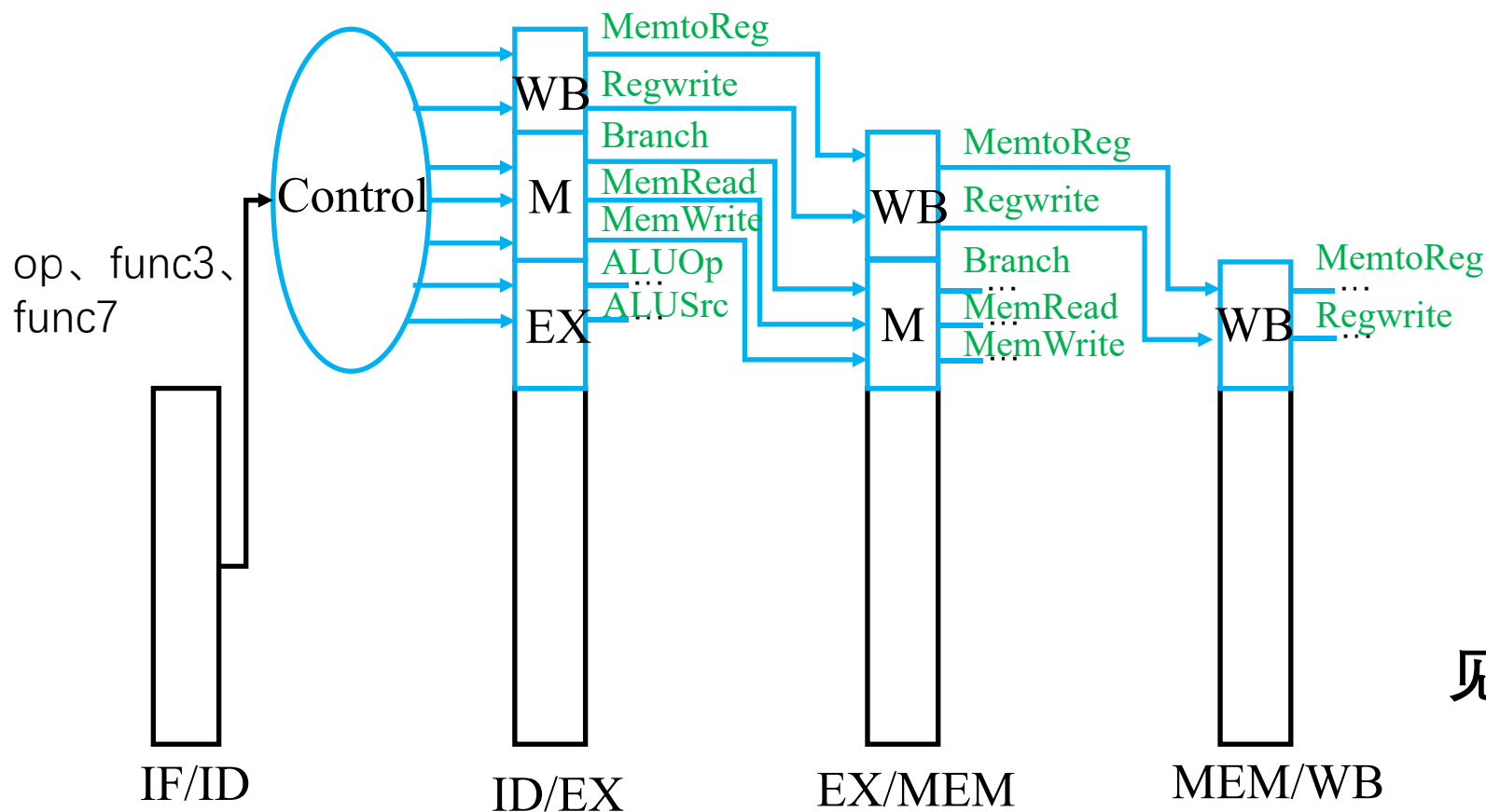
一个时钟周期的流水线图

- 显示了给定周期中流水线的状态
 - 单时钟周期图是多时钟周期图里一个时钟周期的垂直切片



流水线控制

- 控制信号从指令中产生，与单周期实现相似
- 根据流水线阶段将控制线分组
 - IF和ID阶段没有需要特别控制的内容



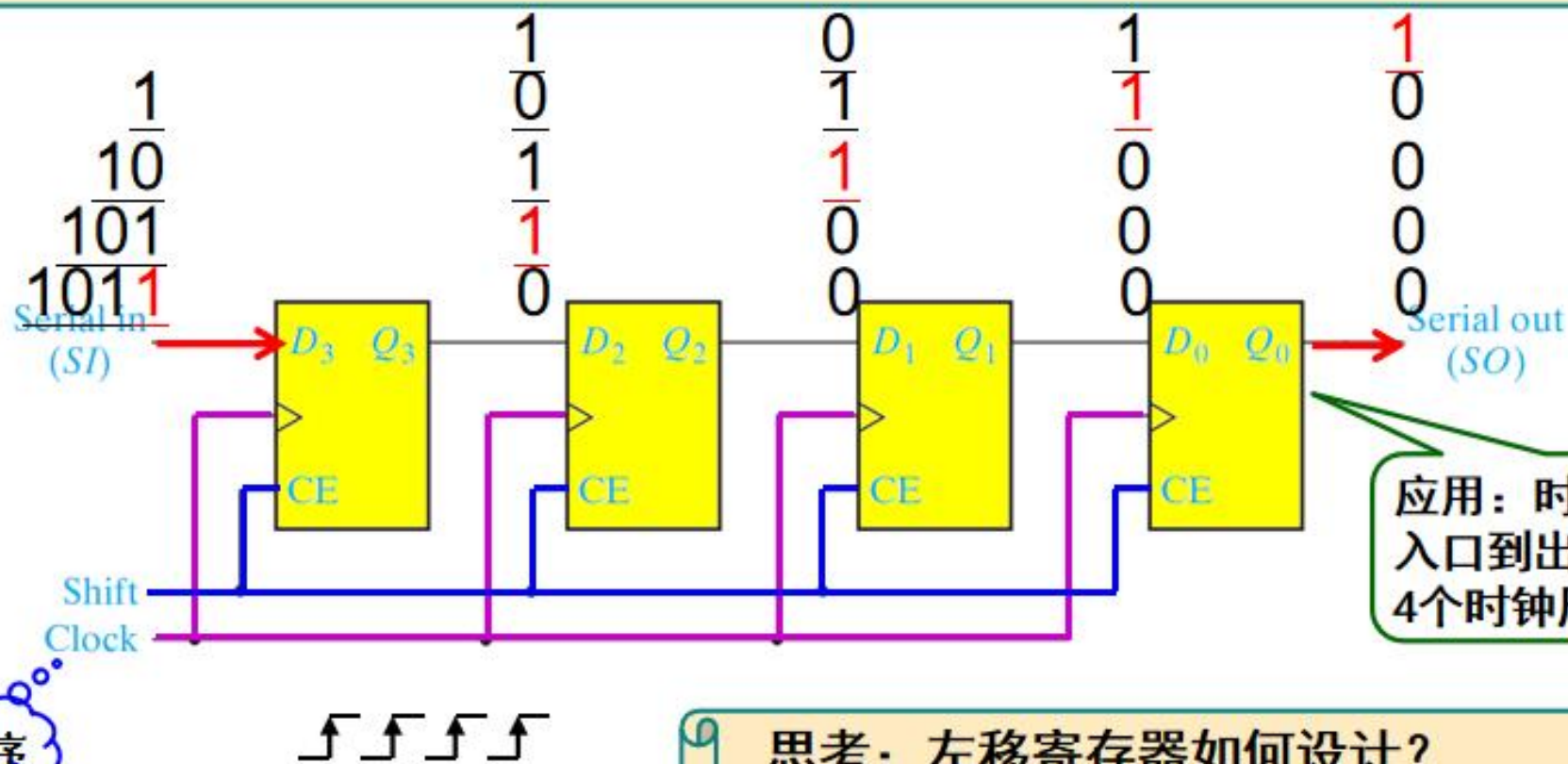
- MemtoReg选择ALU或数据存储器值写入到寄存器堆
- ALUSrc: ALU前的选择器控制信号，用于选择Reg[rs2]或立即数
- Branch: beq指令的控制信号，相等则跳转（备注：这里的数据通路对于分支指令仅实现了beq指令）

见黑书P206-207页的图表

右移寄存器 (Right-Shift Register)

1 串行输入/串行输出 (Serial in/Serial out)

■ 串行输出：移位路径上最后一个触发器的输出作为整个电路的输出



右移方式下：
数据从串行输入端送入，应该先送最低位

应用：时间延迟，
入口到出口，滞后
4个时钟周期

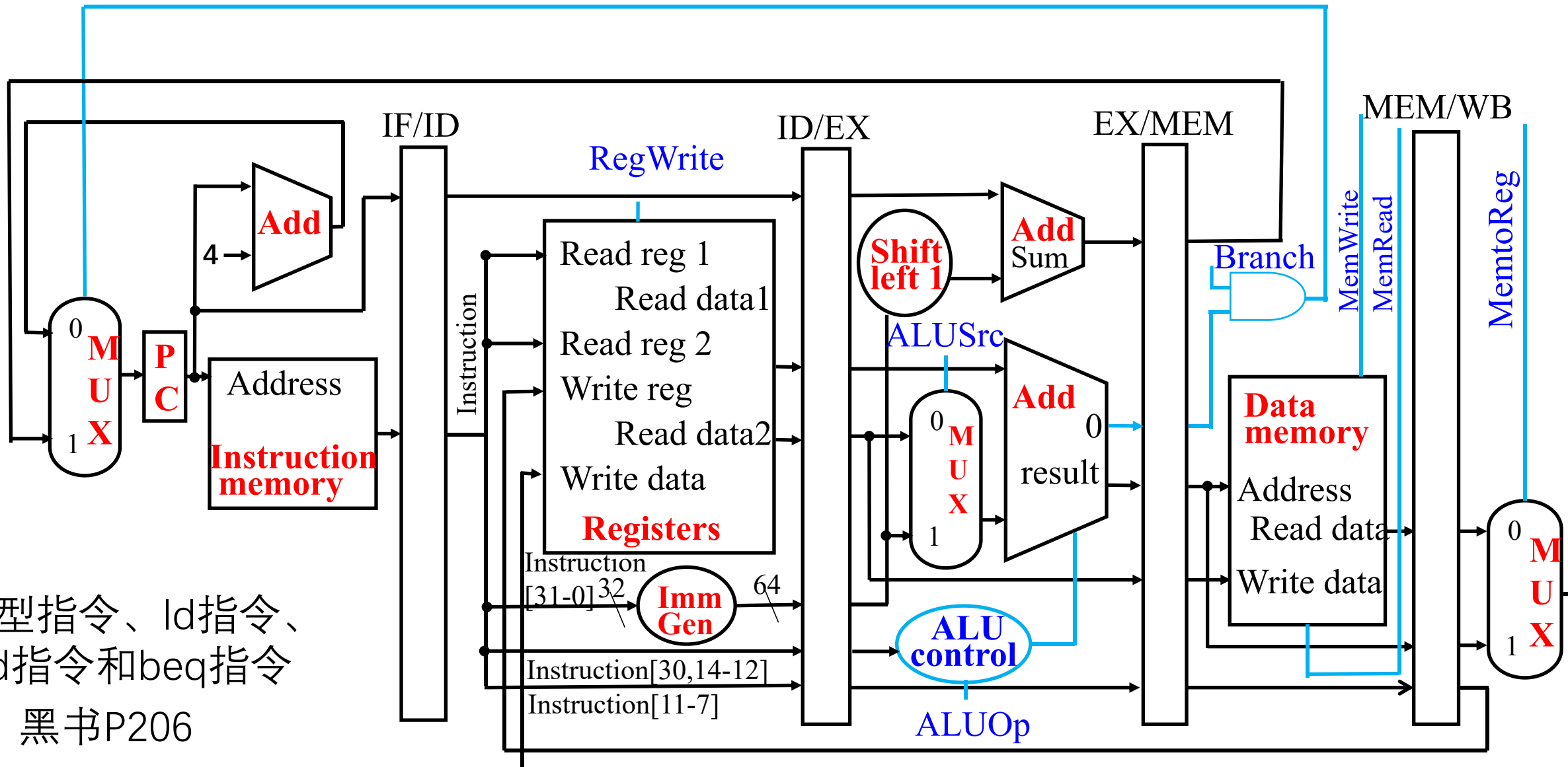
思考：左移寄存器如何设计？
左移方式下从串行输入端送入数据，
应该先送最低位还是最高位？

示例：后三个流水线阶段的控制线

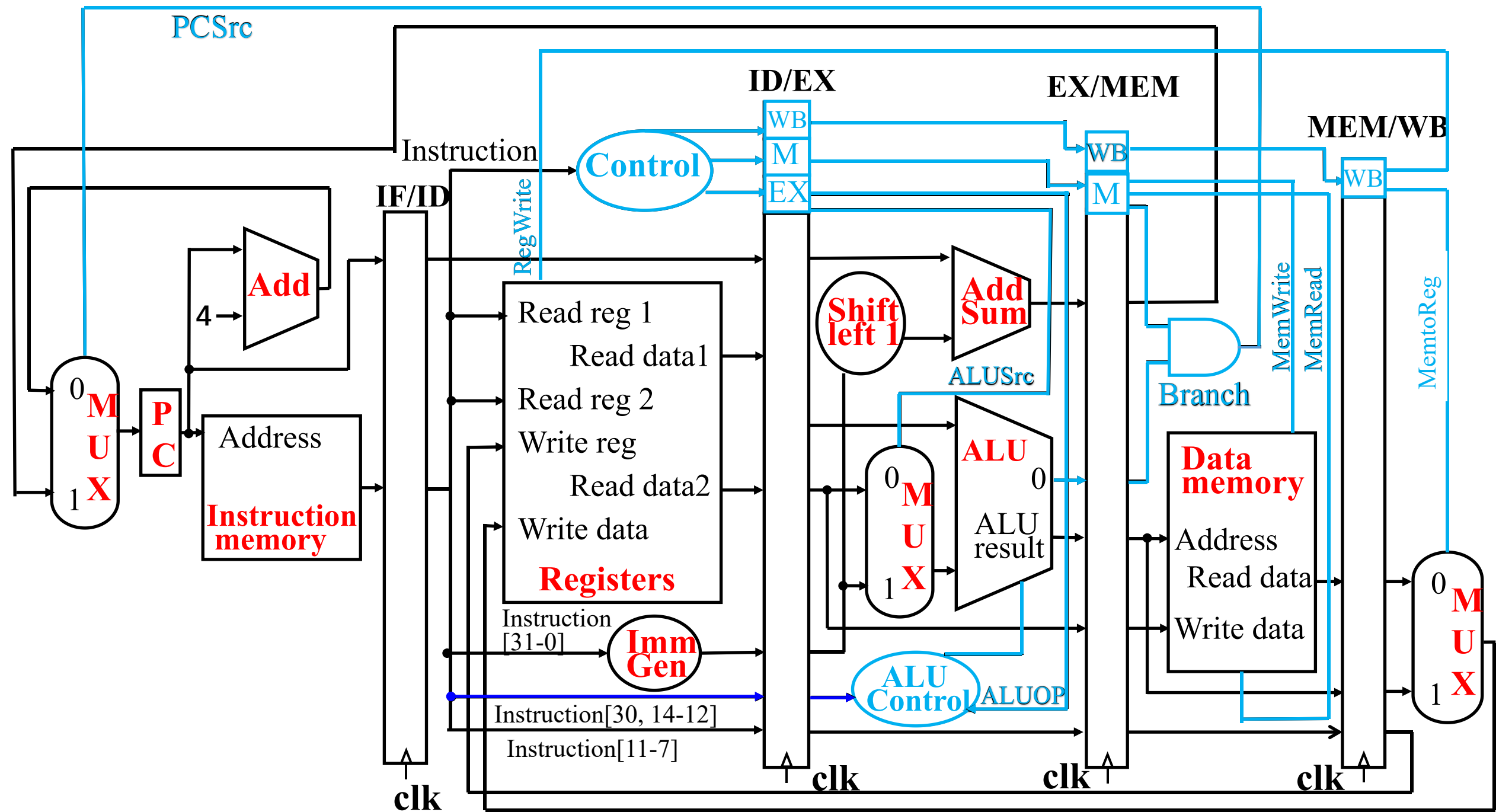
指令	EX阶段的控制线		MEM阶段的控制线			WB阶段的控制线	
	ALUOp	ALUSrc	Branch	Mem-Read	Mem-Write	RegWrite	MemtoReg
R-format	10	0	0	0	0	1	0
ld	00	1	0	1	0	1	1
sd	00	1	0	0	1	0	X
beq	01	0	1	0	0	0	X

简单的流水线控制

PCSrc



R型指令、ld指令、
sd指令和beq指令
黑书P206



回顾

- 流水线通过提高吞吐率来提升性能
 - 并行执行多条指令
 - 每条指令拥有相同的延迟
 - 延迟：执行单条指令的时间
- 流水线会受到冒险的影响
 - 结构冒险、数据冒险、控制冒险
- 指令集的设计影响到流水线实现的复杂度
 - 指令长度相同、指令格式整齐、存储操作只发生在load/store指令中（RISC-V）

第五章

- 流水线概述
- 流水线数据通路和控制
- 解决数据冒险的方法：前递与停顿
- 控制冒险
- 例外

ALU相关指令中的数据冒险

- 考虑以下指令序列:

sub x2, x1, x3

x2: 10 (before)

and x12, x2, x5

x2: -20 (after)

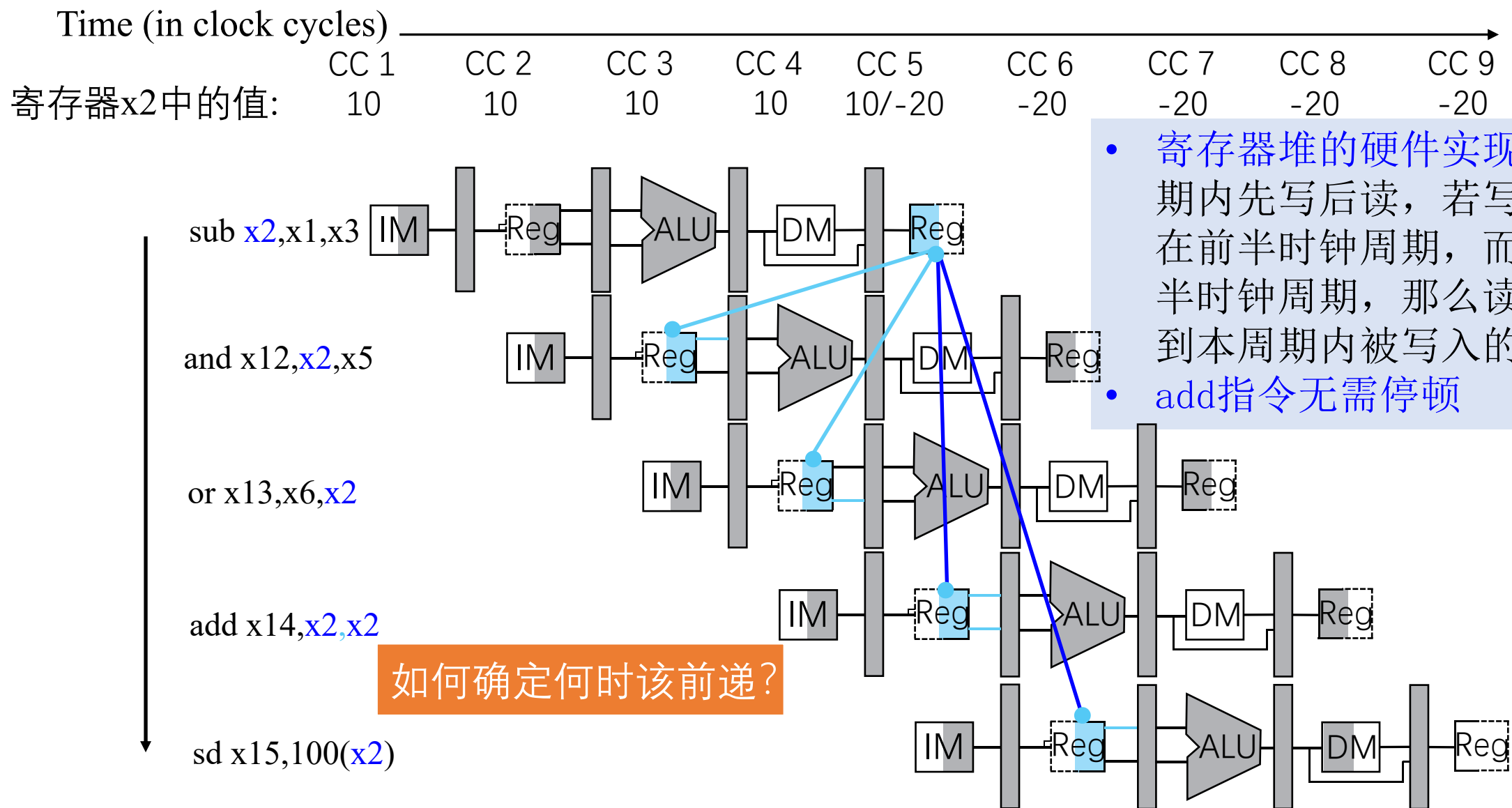
or x13, x6, x2

add x14, x2, x2

sd x15, 100(x2)

- 可通过前递来解决这些冒险

依赖关系与前递



数据冒险的检测方法

- 命名流水线寄存器中的具体寄存器
 - 例 ID/EX.RegisterRs1：ID/EX流水线寄存器中，存放寄存器Rs1编号的寄存器
- EX阶段，ALU操作数来源于立即数或寄存器：
 - ID/EX.RegisterRs1, ID/EX.RegisterRs2

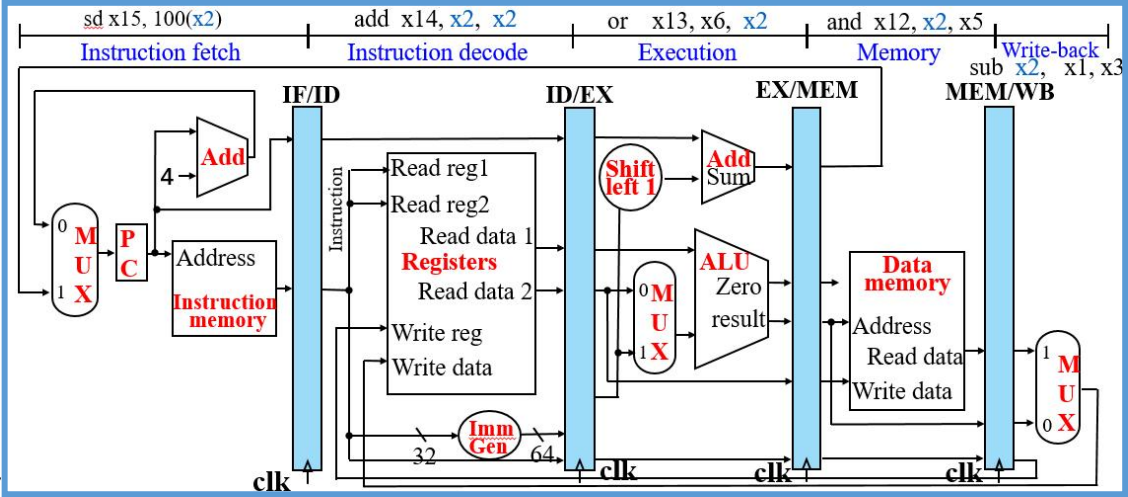
- EX冒险和MEM冒险

1a. EX/MEM.RegisterRd == ID/EX.RegisterRs1

1b. EX/MEM.RegisterRd == ID/EX.RegisterRs2

2a. MEM/WB.RegisterRd == ID/EX.RegisterRs1

2b. MEM/WB.RegisterRd == ID/EX.RegisterRs2



sub **x2**, x1,x3

and x12,**x2**,x5

sub **x2**, x1,x3

and x12,**x2**,x5

or x13,x6,**x2**

取指令	译码	执行	访存	写回	
	取指令	译码	执行	访存	写回

取指令	译码	执行	访存	写回		
	取指令	译码	执行	访存	写回	
		取指令	译码	执行	访存	写回

数据冒险的检测应考虑的其他方面

- 不是所有指令都写回寄存器

- 检查RegWrite信号是否有效

- EX冒险: EX/MEM.RegWrite $\neq 0$
- MEM冒险: MEM/WB.RegWrite $\neq 0$

- $R_d=0$?

- EX冒险: EX/MEM. RegisterRd $\neq$ 0
- MEM冒险: MEM/WB. RegisterRd $\neq$ 0

- ## ● 二次冒险

```
add x1,x1,x2
```

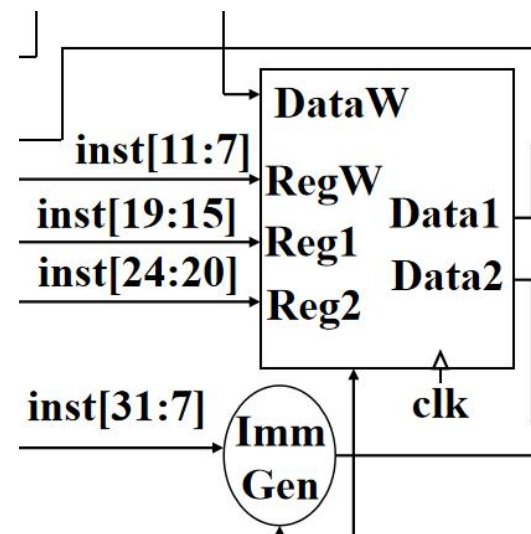
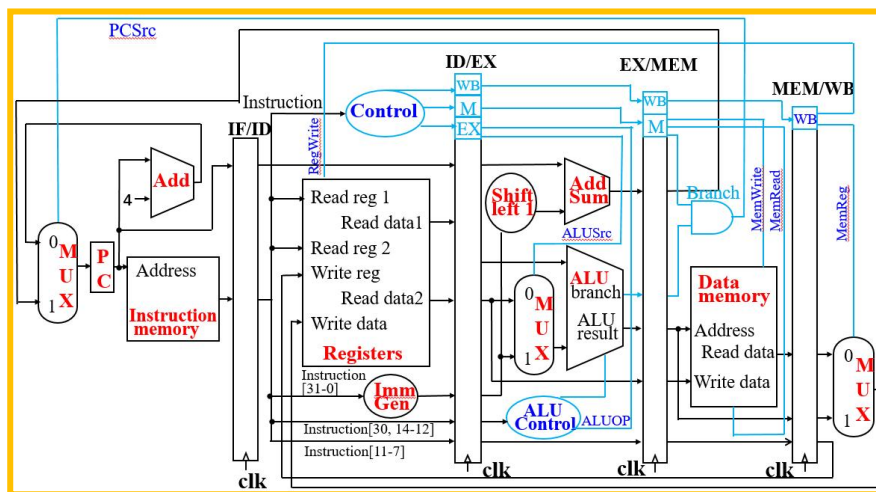
```
add x1,x1,x3
```

```
add x1,x1,x4
```

S型	imm[11:5]	rs2	rs1	func3	imm[4:0]	opcode
R型	func7	rs2	rs1	func3	rd	opcode

sd x1, 8(x2)

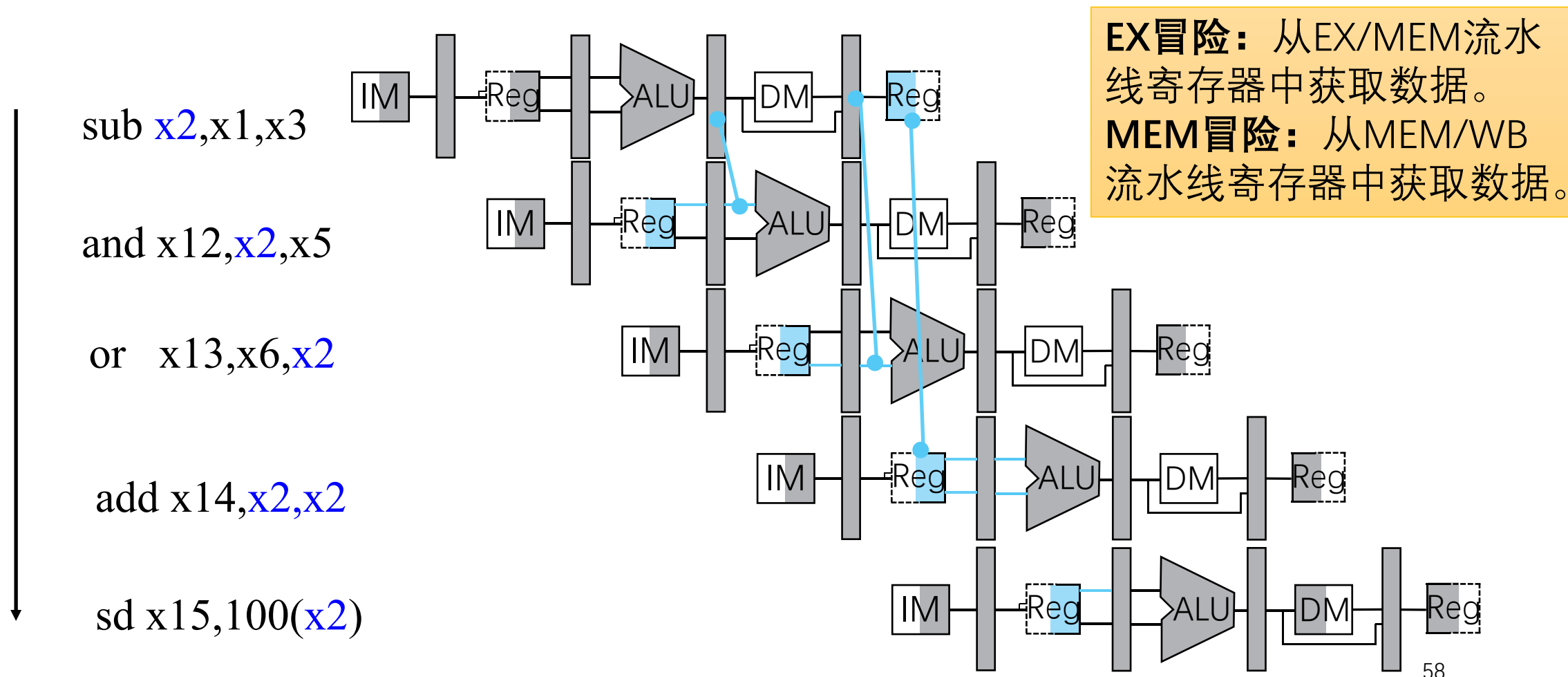
```
add x3, x8, x2
```



依赖关系的变化

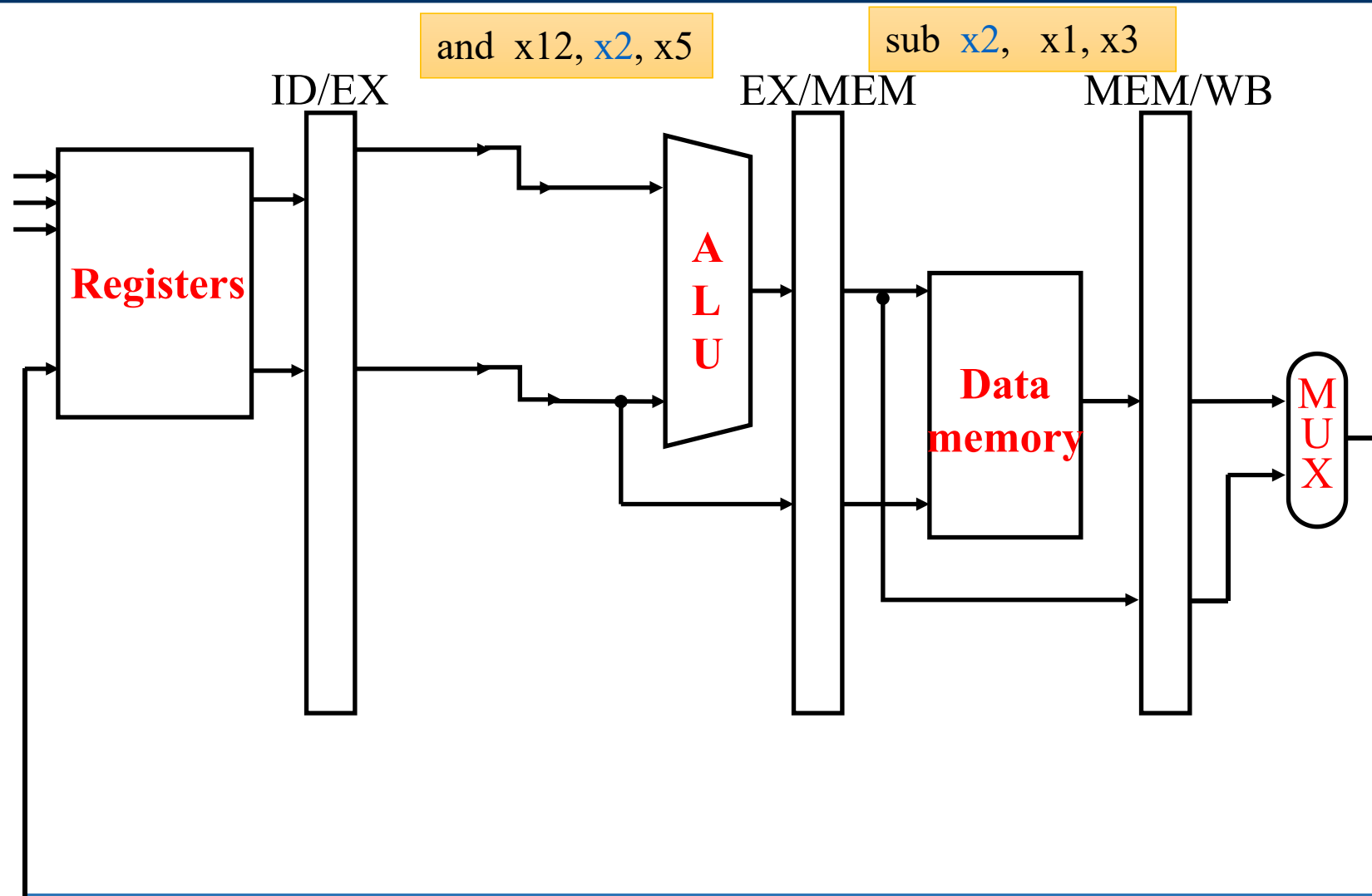
Time (in clock cycles)

	CC 1	CC 2	CC 3	CC 4	CC 5	CC 6	CC 7	CC 8	CC 9
Value of register x2:	10	10	10	10	10/-20	-20	-20	-20	-20



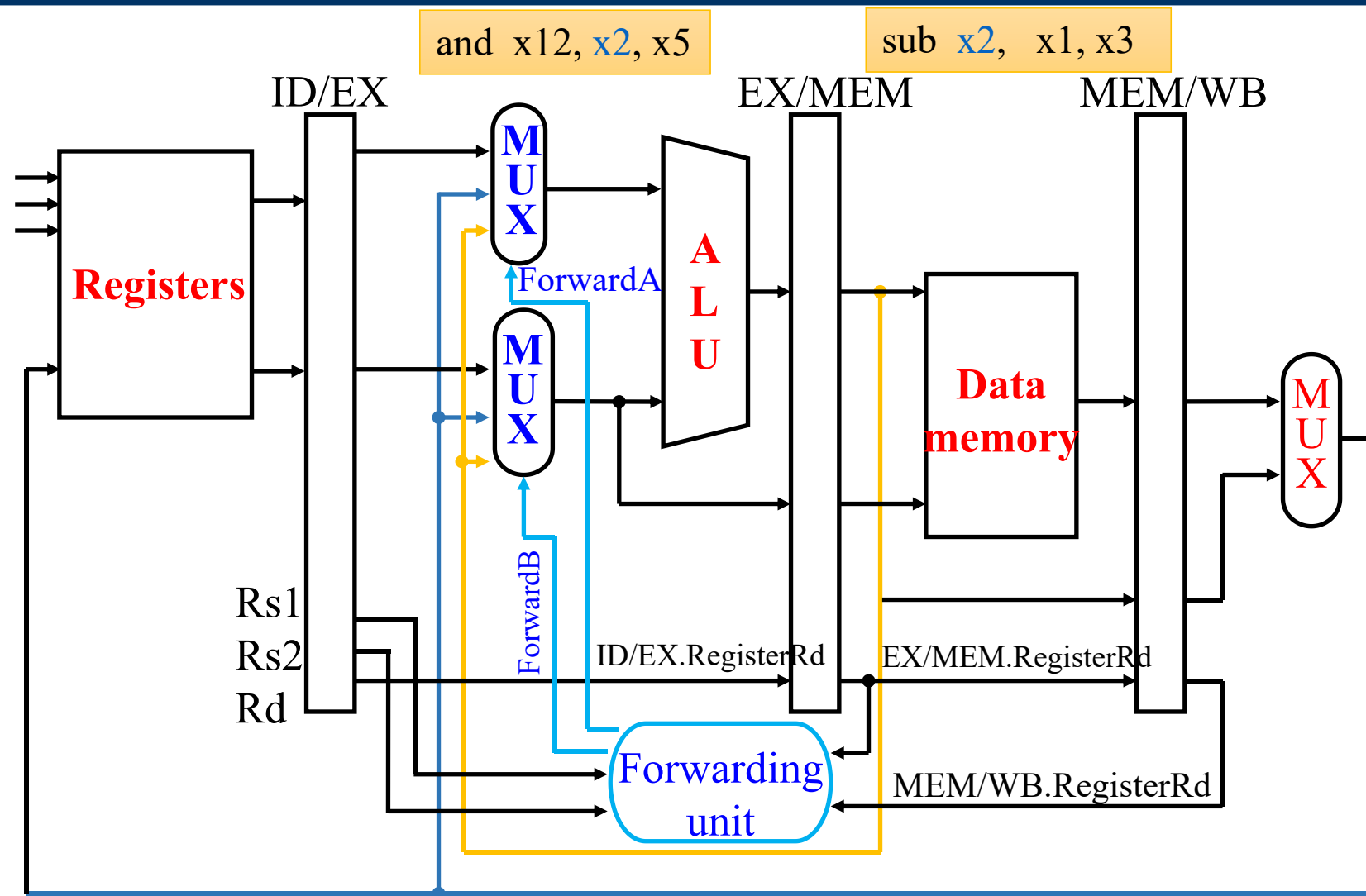
添加前递前

```
sub x2, x1, x3  
and x12, x2, x5  
or x13, x6, x2  
add x14, x2, x2  
sd x15, 100(x2)
```



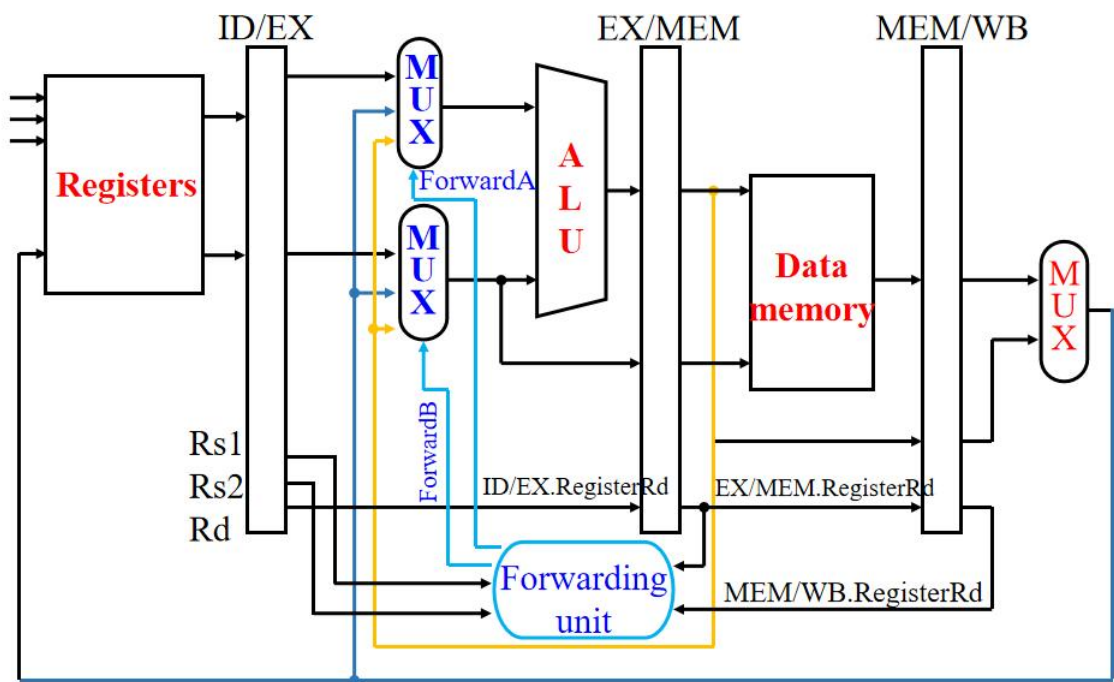
添加前递所需硬件

```
sub x2, x1, x3
and x12, x2, x5
or x13, x6, x2
add x14, x2, x2
sd x15, 100(x2)
```



加入前递之后，Rs1和Rs2字段也要添加到ID/EX流水线寄存器中

前递硬件中，多选器的控制值



ForwardB	数据来源	备注
00	ID/EX 寄存器	ALU的第二个操作数来自寄存器堆
10	EX/MEM 寄存器	ALU第二个操作数来自上一个ALU计算结果的前递
01	MEM/WB 寄存器	ALU的第二个操作数来自数据存储器或者更早的ALU计算结果的前递

ForwardA	数据来源	备注
00	ID/EX 寄存器	ALU的第一个操作数来自寄存器堆
10	EX/MEM 寄存器	ALU的第一个操作数来自上一个ALU计算结果的前递
01	MEM/WB 寄存器	ALU的第一个操作数来自数据存储器或者更早的ALU计算结果的前递

检测EX冒险的条件、相应的前递控制

if (EX/MEM.RegWrite **and** (EX/MEM.RegisterRd $\neq$ 0)
 and (EX/MEM.RegisterRd == ID/EX.RegisterRs1))

ForwardA = 10

if (EX/MEM.RegWrite **and** (EX/MEM.RegisterRd $\neq$ 0)
 and (EX/MEM.RegisterRd == ID/EX.RegisterRs2))

ForwardB = 10

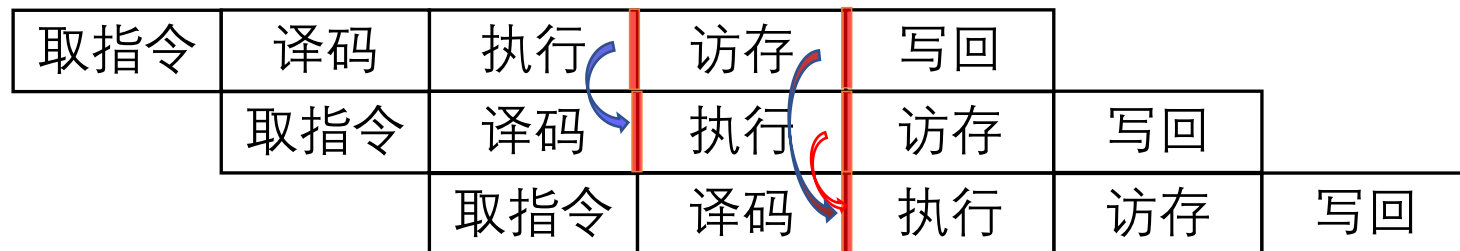
MEM冒险检测：两次数据冒险

- 考虑以下指令序列：

add **x1**,x1,x2

add **x1**,**x1**,x3

add x1,**x1**,x4



- 两组数据冒险都发生（EX数据冒险和MEM数据冒险）
 - 对于第三条指令而言，应该**前递最近的结果**
- MEM冒险前递的条件：只有EX冒险不发生的时候才前递
 - 理由参见上一条

检测MEM冒险的条件、相应的前递控制

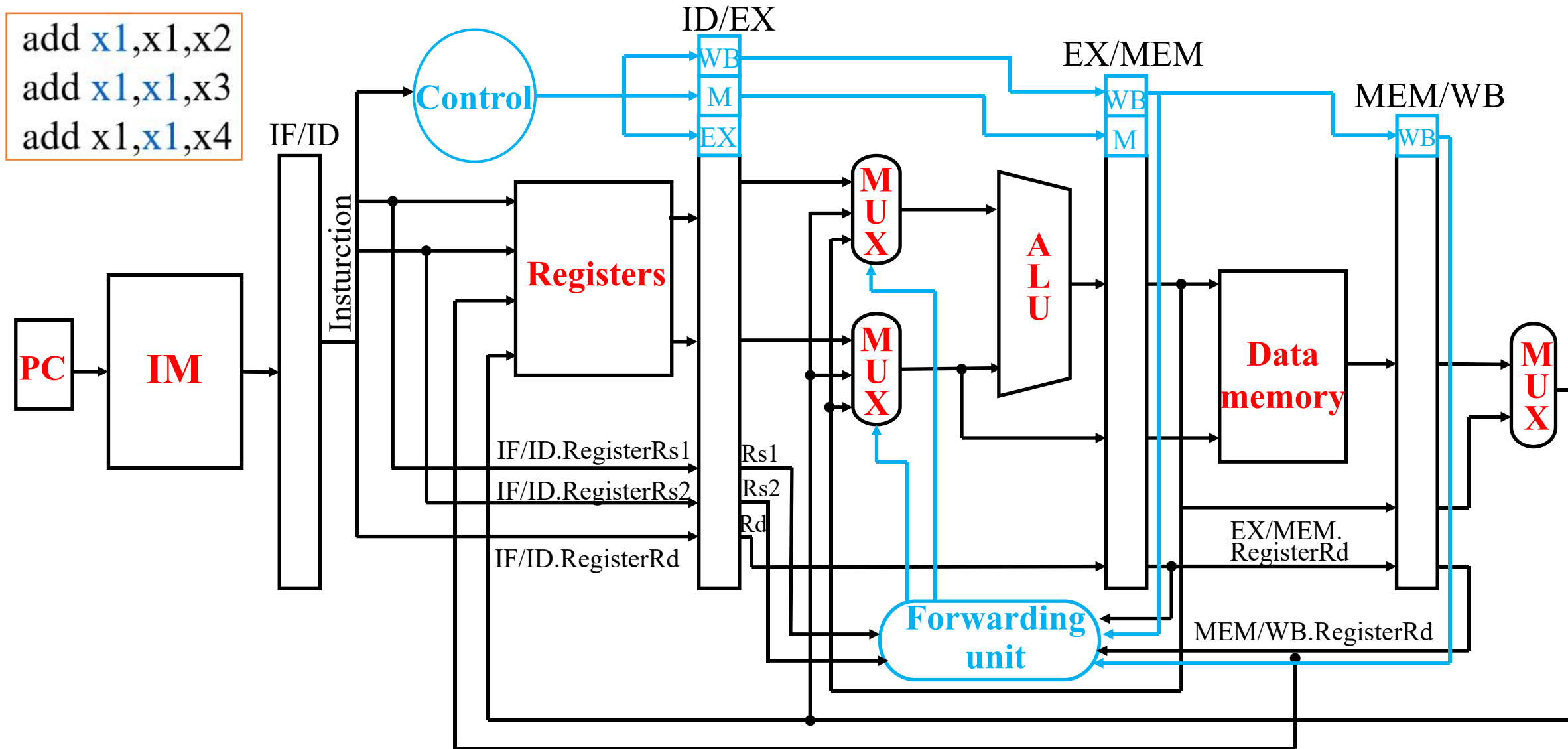
if (MEM/WB.RegWrite **and** (MEM/WB.RegisterRd $\neq$ 0) **and**
not (EX/MEM.RegWrite and (EX/MEM.RegisterRd $\neq$ 0) **and**
(EX/MEM.RegisterRd == ID/EX.RegisterRs1)) //无EX冒险
and (MEM/WB.RegisterRd == ID/EX.RegisterRs1))

ForwardA = 01

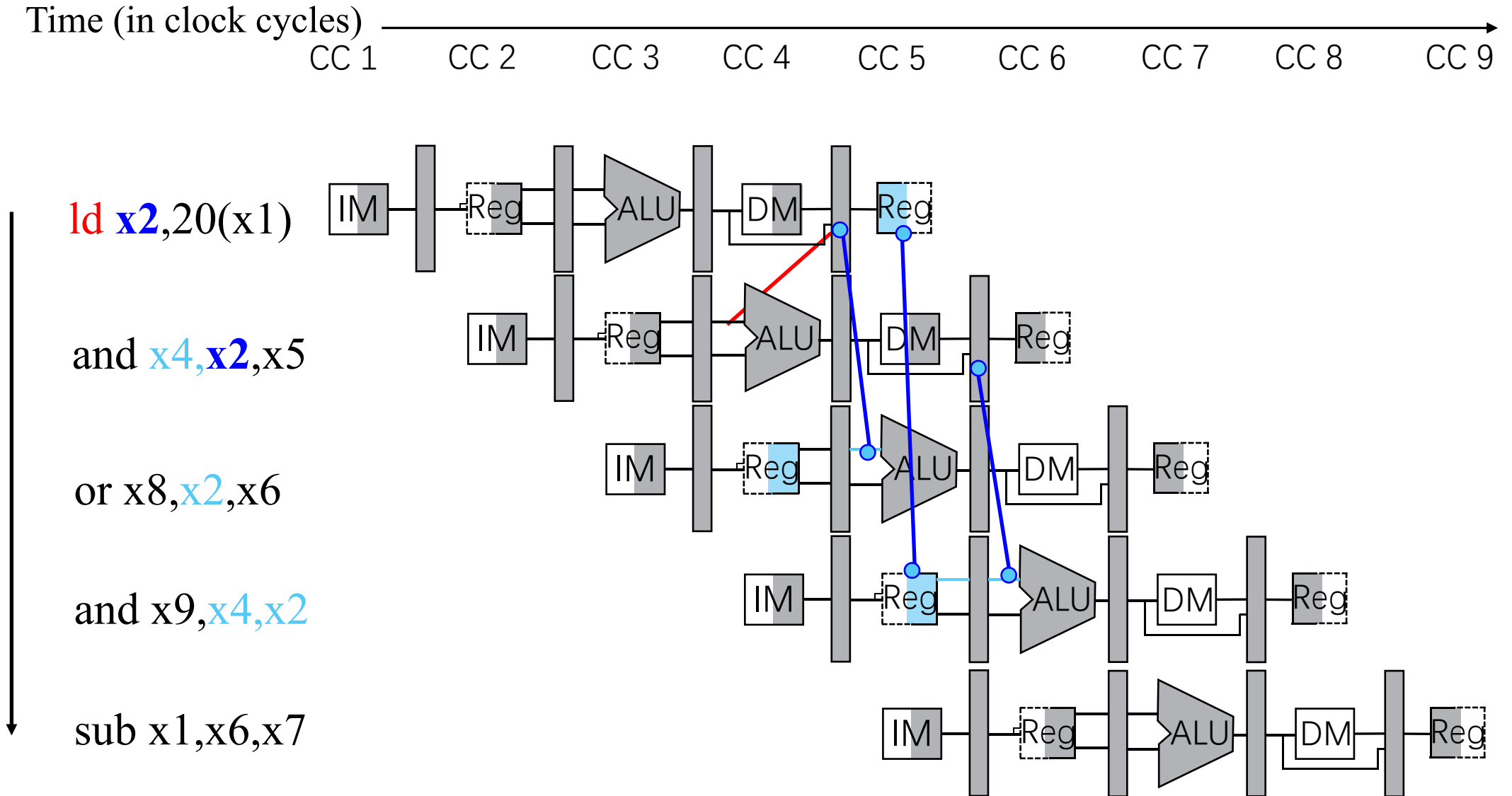
if (MEM/WB.RegWrite **and** (MEM/WB.RegisterRd $\neq$ 0) **and**
not(EX/MEM.RegWrite and (EX/MEM.RegisterRd $\neq$ 0) **and**
(EX/MEM.RegisterRd == ID/EX.RegisterRs2)) //无EX冒险
and (MEM/WB.RegisterRd == ID/EX.RegisterRs2))

ForwardB = 01

通过前递解决冒险的数据通路

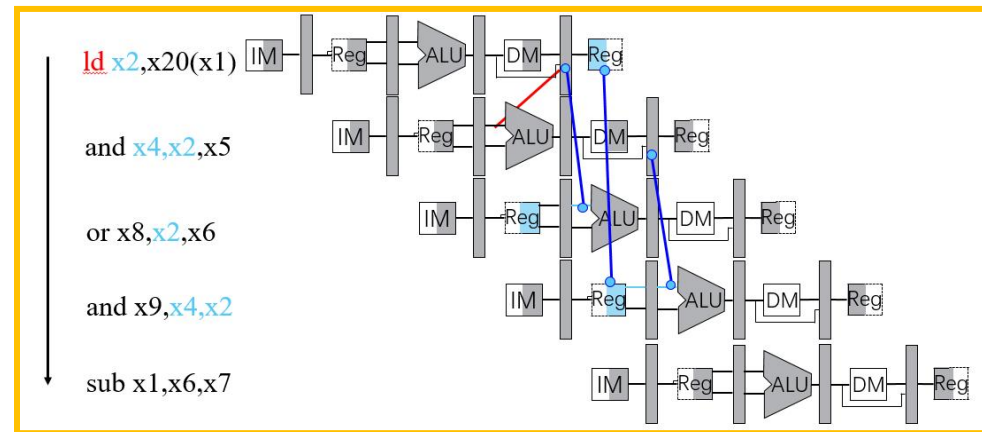


载入-使用(load-use)型数据冒险

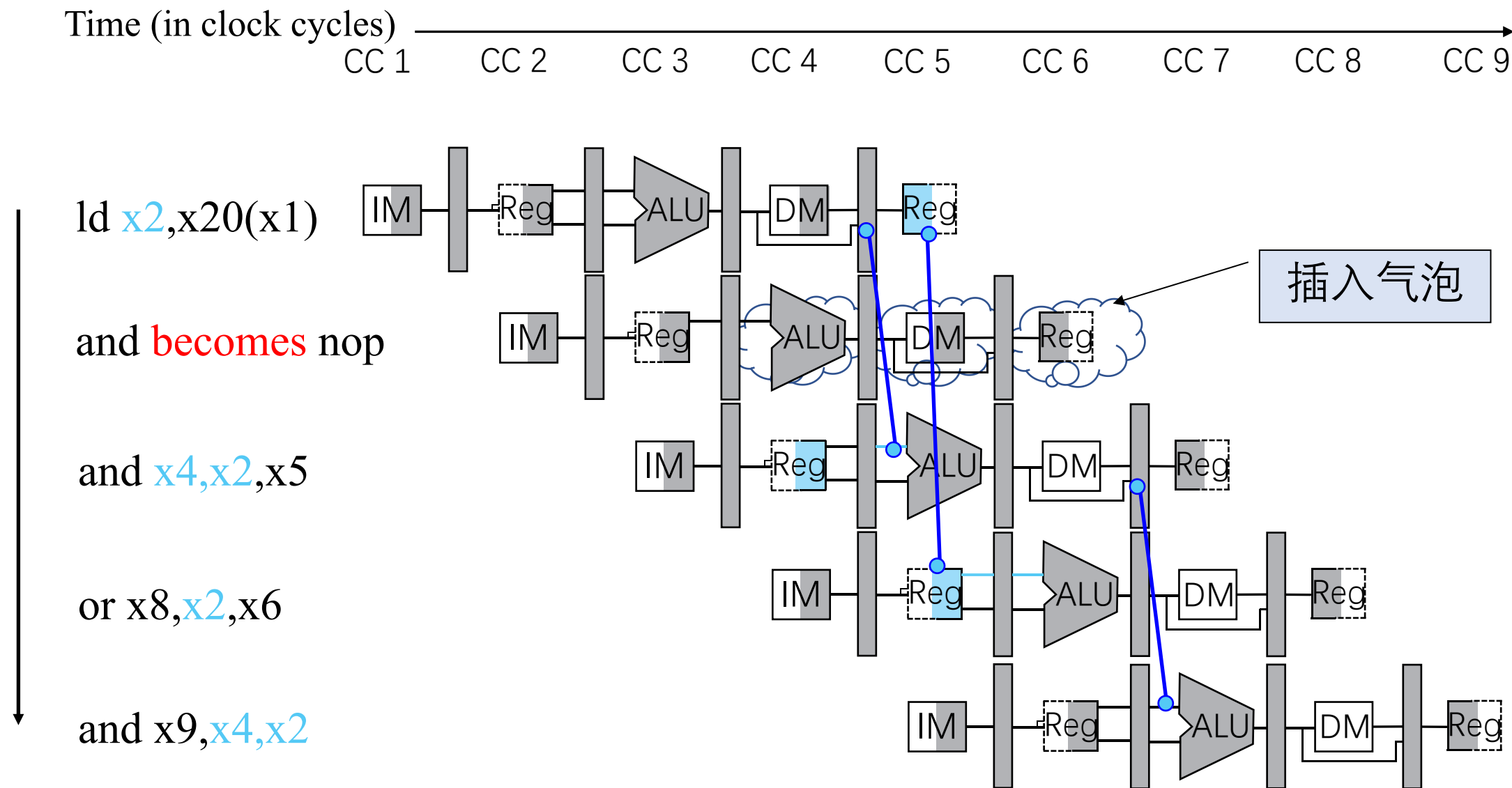


检测 载入-使用型数据冒险

- 在ID阶段进行检测
- ALU操作数寄存器字段名称分别是：
 - IF/ID.RegisterRs1, IF/ID.RegisterRs2
- 检测条件
 - ID/EX.MemRead and //是否为load指令？
 $((ID/EX.RegisterRd = IF/ID.RegisterRs1) \text{ or } (ID/EX.RegisterRd = IF/ID.RegisterRs2))$
- 如果检测到这种冒险， 停顿流水线（插入气泡）



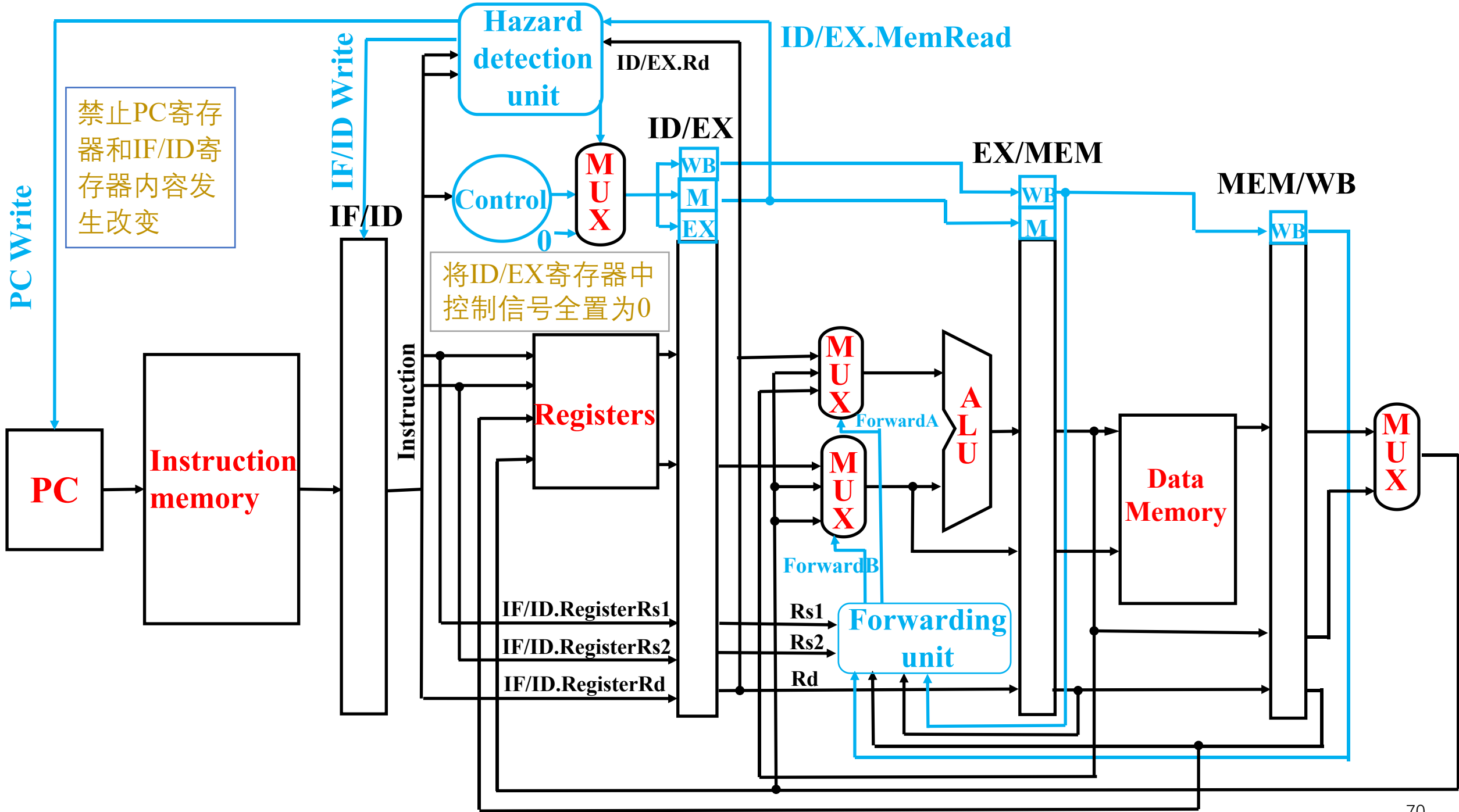
载入-使用型数据冒险



如何停顿流水线?

被停顿的指令正处于ID阶段:

- 将ID/EX寄存器中控制信号全置为0(RegWrite, MemWrite...)
 - 被停顿的指令在EX, MEM, WB 阶段执行空指令 nop
 - 不会有寄存器或者存储器被写入数据
- 禁止PC寄存器和IF/ID寄存器内容发生改变
 - ID阶段的寄存器会继续使用IF/ID寄存器中相同字段读寄存器
 - 下一条指令会重新取指
 - 1个时钟周期的停顿, 能够让ld指令的MEM阶段完成
 - 就可以把取到的数据前递到EX阶段



停顿与性能

- 停顿会导致性能下降
 - 但为了得到正确的结果，需要停顿
- 编译器能编排代码，尽量避免冒险和停顿
 - 这需要流水线结构的知识

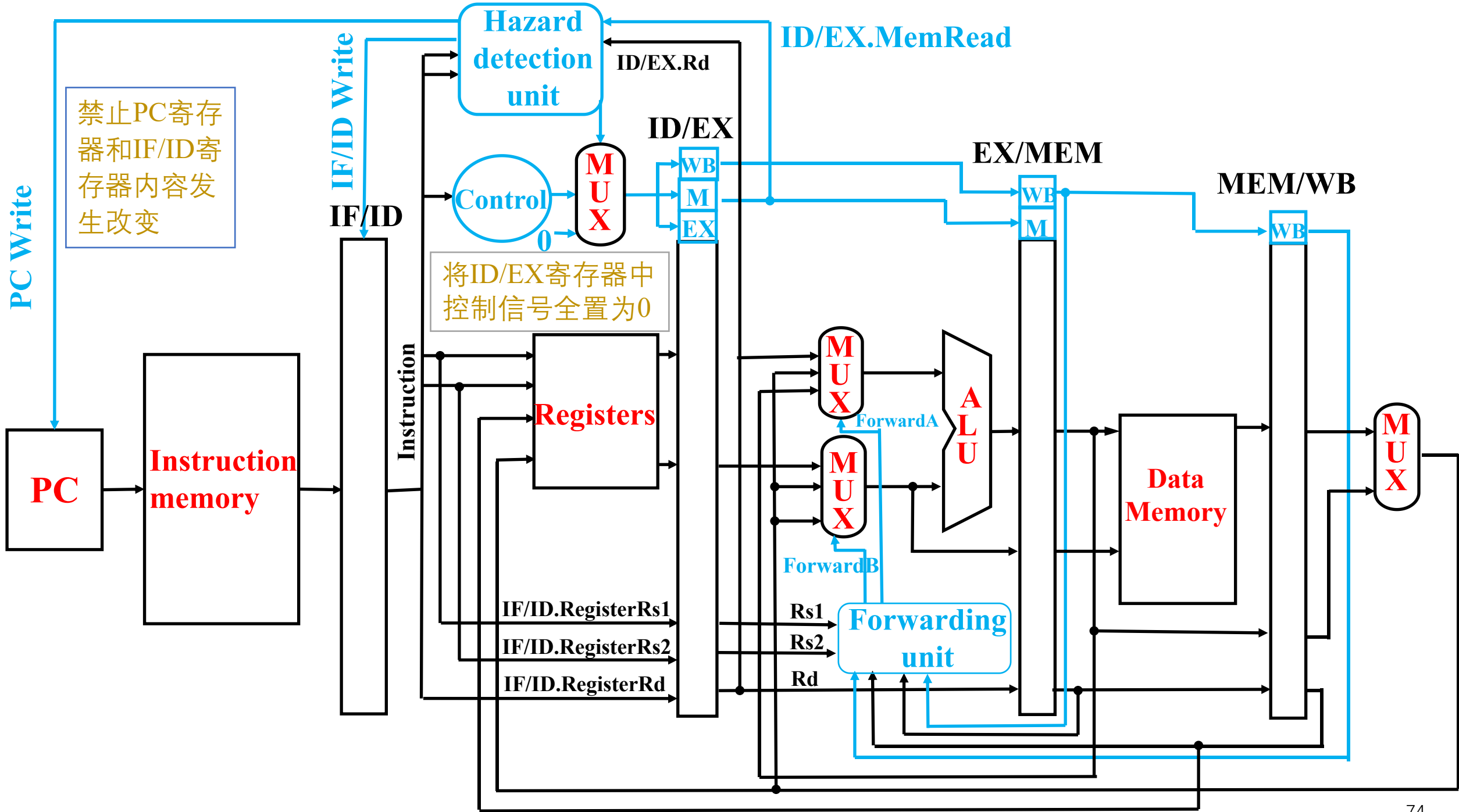
在本课程所讲的五段流水线处理器中，数据冒险可能会发生在哪些阶段？

- ☐ A Instruction **F**etch
- ☒ B Instruction **D**ecode
- ☒ C **E**Xecute
- ☒ D **M**EMory access
- ☐ E **W**rite **B**ack register

提交

第五章

- 流水线概述
- 流水线数据通路和控制
- 解决数据冒险的方法：前递与停顿
- 控制冒险
- 例外

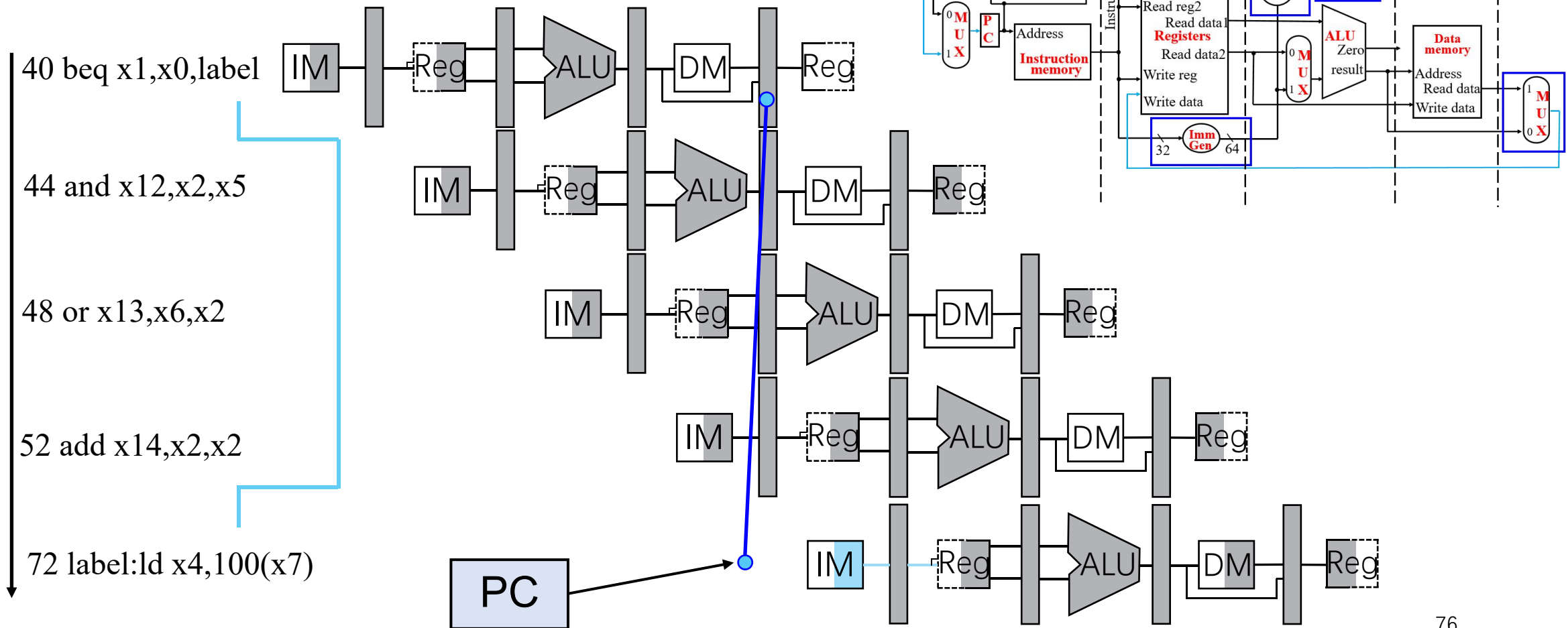


控制冒险

- 分支决定了控制流
 - 分支指令之后的IF阶段依赖于分支的结果
 - 流水线无法保证永远取到正确的指令
 - 此时，分支指令仍在ID阶段
- 在RISC-V流水线设计中
 - 尽可能早地完成寄存器比较、分支目标地址计算
 - 添加硬件，使得以上计算能在ID阶段完成

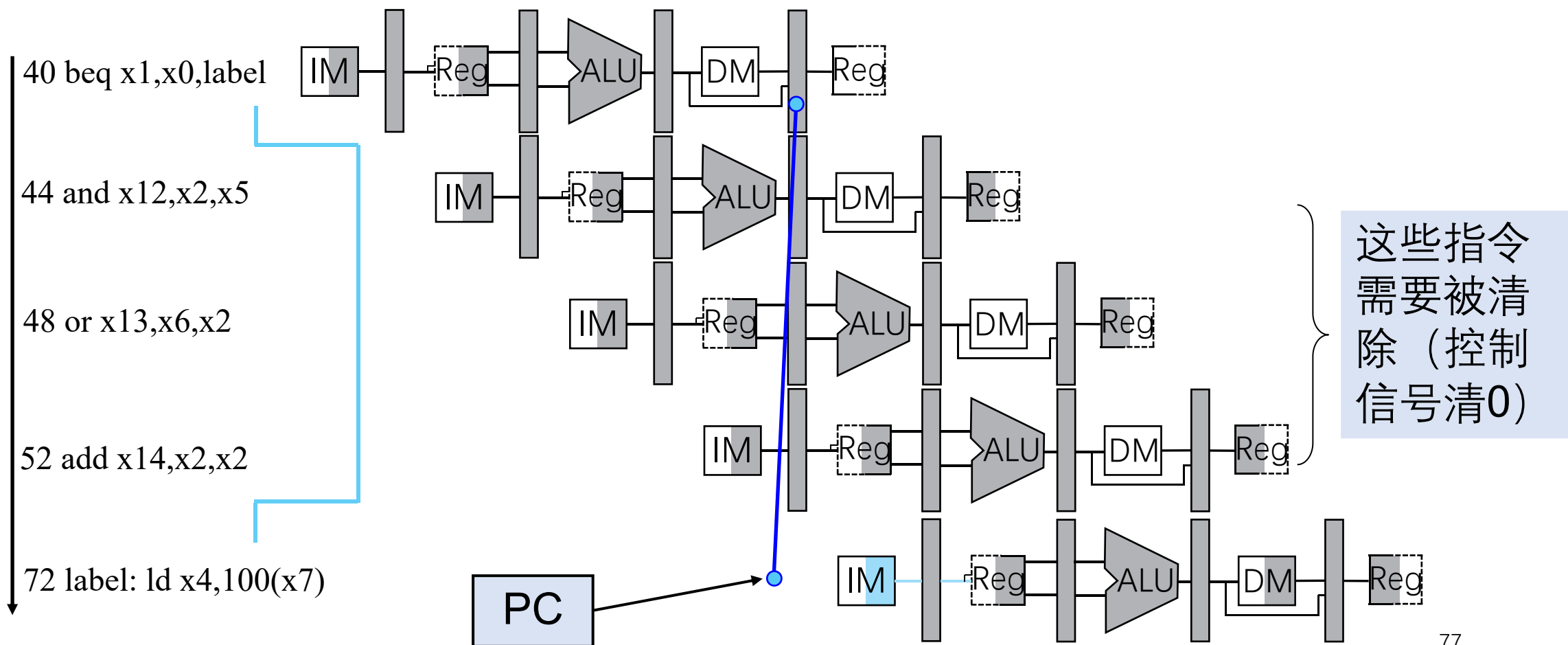
控制冒险

- **假设**分支结果在MEM阶段完成确认
 - 分支指令修改PC值发生在MEM阶段



假设默认分支不发生

- 如果发生跳转，则需要清除掉分支语句后续指令



缩短分支延迟

- 移动硬件，使得分支决定**提前**到ID阶段
 - 需要**提早判断**分支条件
 - 需要**提早计算**分支目标地址（IF/ID寄存器中有PC和Immediate）
- 例：分支跳转发生

36 sub x10, x4, x8

40 beq x1, x3, **label** //PC-relative branch to $40+16*2=72$

44 and x12, x2, x5

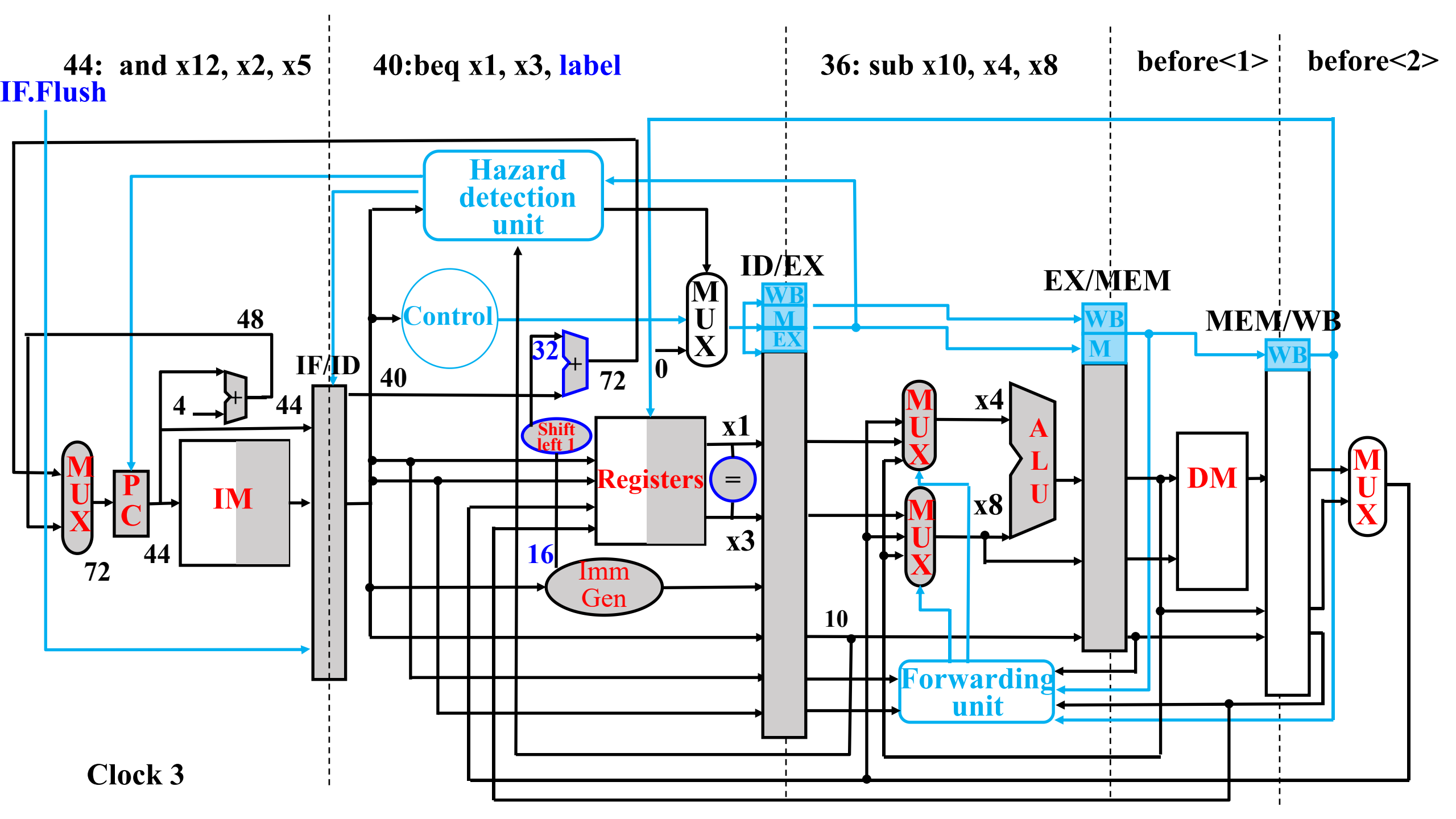
48 or x13, x2, x6

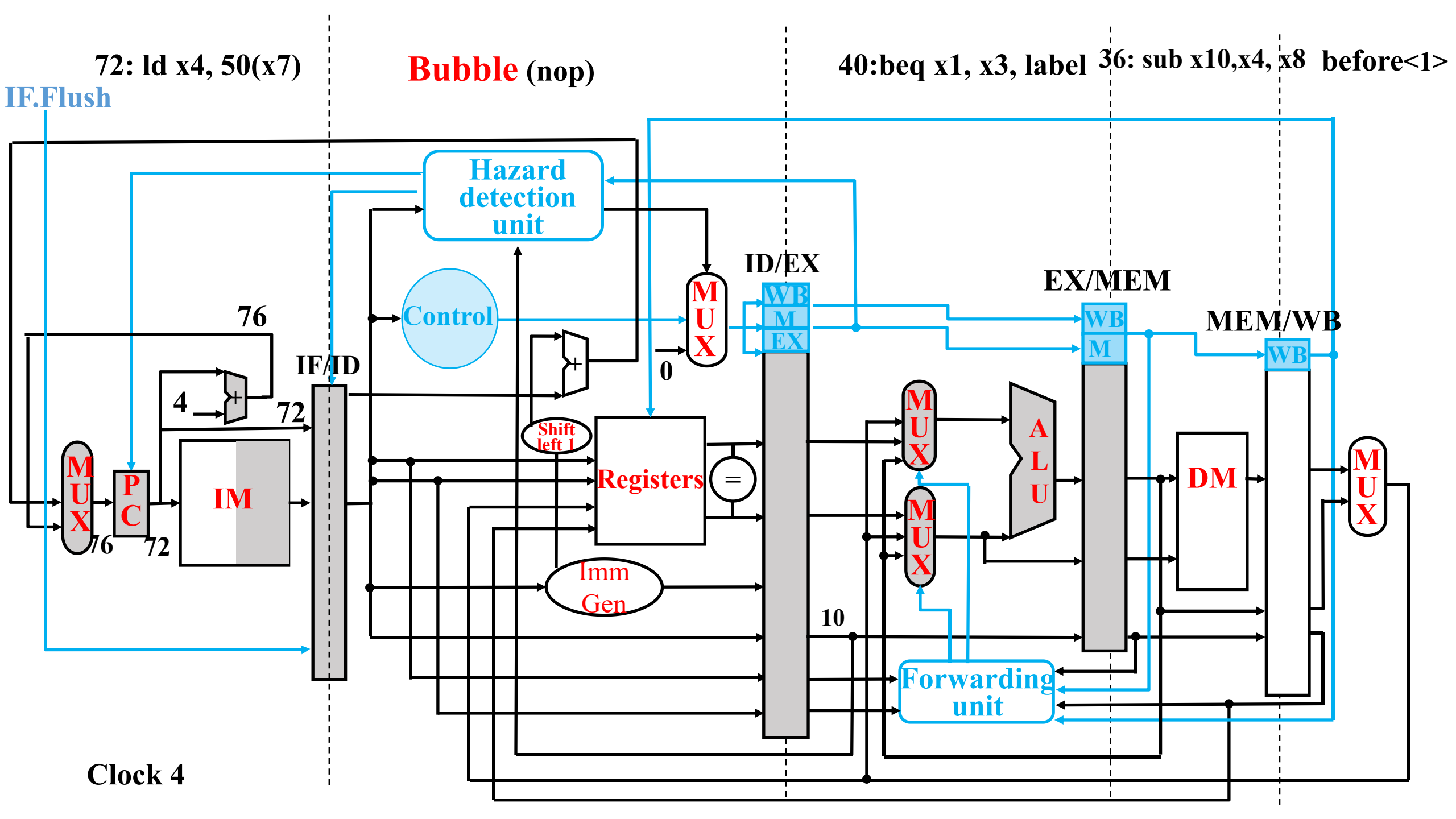
52 add x14, x4, x2

56 sub x15, x6, x7

...

72 label: ld x4, 50(x7)



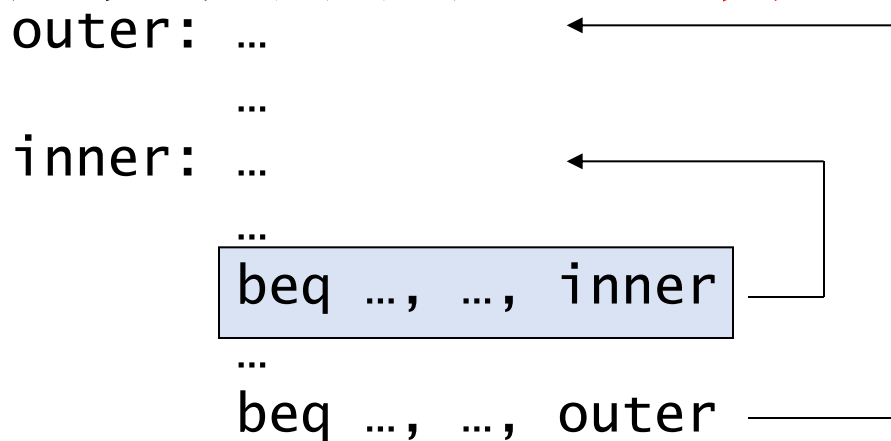


动态分支预测

- 流水线段数，分支预测错误的代价越大（从时钟周期数的角度）
- 使用动态分支预测
 - 一种实现方式：采用分支预测缓存或分支历史表（**Branch History Table**）
 - 是一块按照分支指令的**低位地址索引定位**的小容量存储器
 - 包含一个或多个位（bit）以表明一个分支最近是否发生了跳转
- 1-**Bit**预测机制：用1位表示最近是否发生了跳转
 - 查表，使用表中记录结果作为预测结果
 - 开始取指令
 - 如果预测错误，则清掉错误指令，并更改分支预测缓存中的记录

1-Bit预测机制的缺点

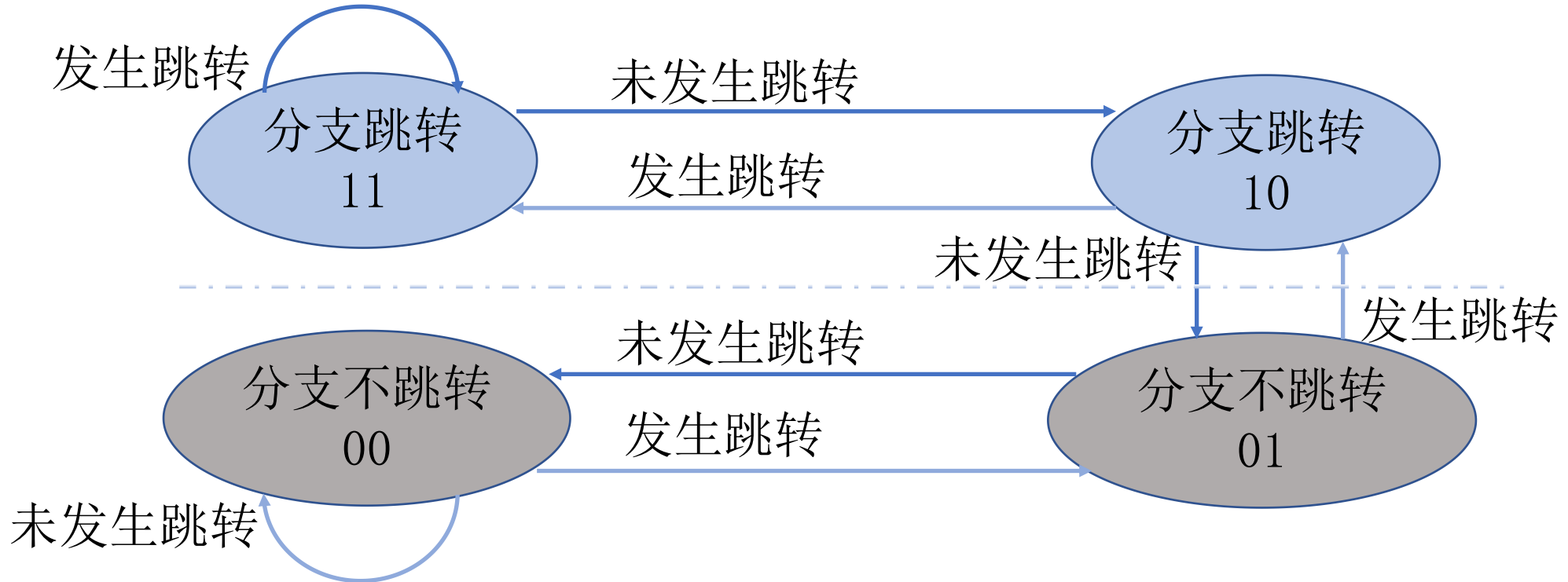
- 一个条件分支多数时间发生跳转，但一旦其不发生跳转时，会导致两次预测错误，而**不是**只造成**一次**错误



- 内层循环的最后一次迭代不发生跳转，预测发生跳转，预测**错误**，更改预测值为**不跳转**。
- 下一次外层循环开始后，内层循环第一次迭代**发生跳转**，预测不发生跳转，预测**错误**，更改预测值为**跳转**

2-Bit 预测机制

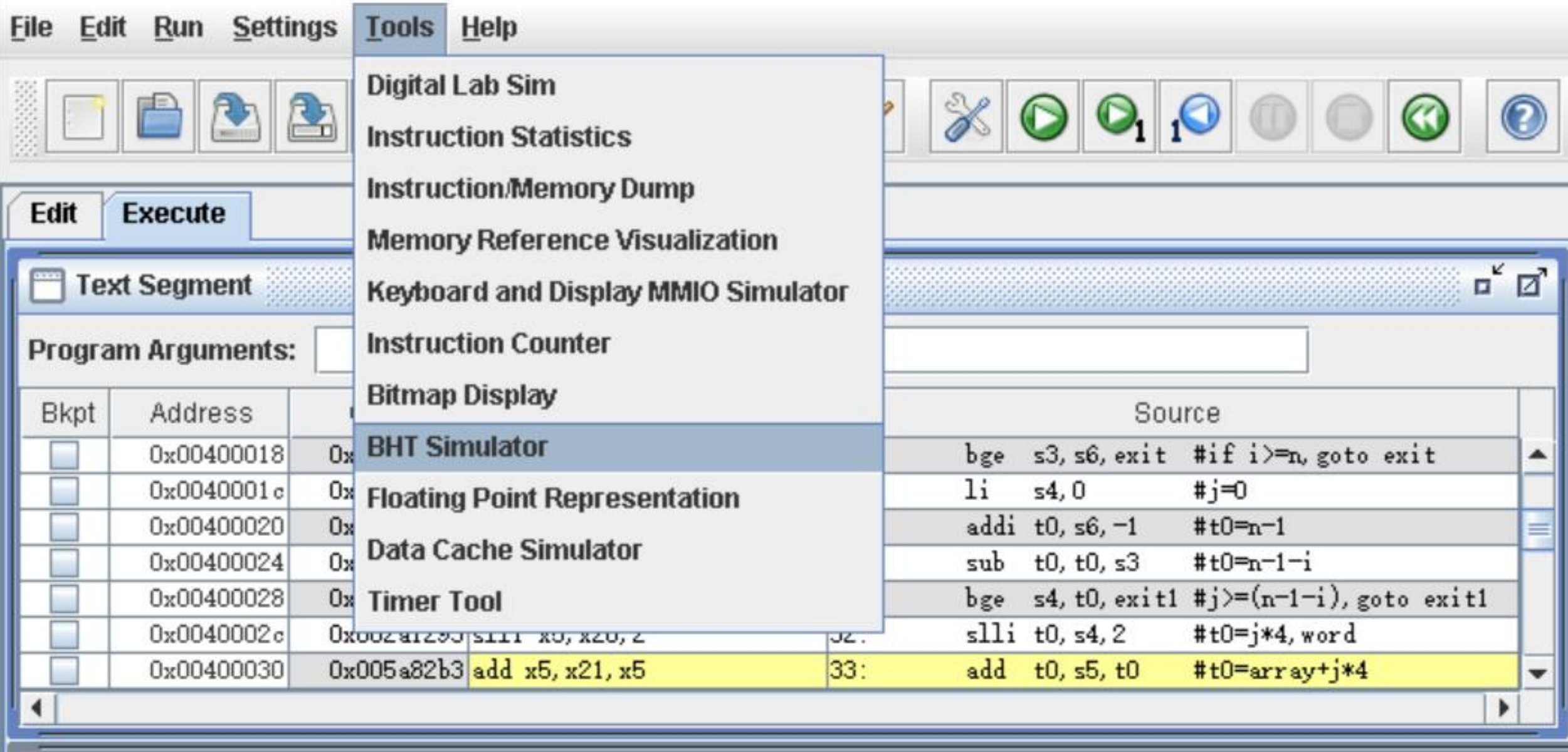
- 对于左侧两个状态，只有在发生了连续两次错误时预测结果才会被改变（跳转加1，不跳转减1）



在一个分支经常跳转或经常不跳转的情况下（大多数分支都是这样的），只会发生一次预测失败

分支目标计算

- 除了判断分支是否发生，还要计算分支目标地址
 - 发生跳转时需要一个时钟周期的代价来计算分支目标地址
- 使用分支目标缓存
 - 缓存分支目标PC值或目标指令
 - 根据当前分支指令的PC值来确定缓存的索引
 - 例如：可以利用指令地址的低六位
 - 如果缓存中有目标PC值或指令，且分支预测跳转，就能读取缓存结果，立即使用





Edit Execute

Text Segment

Program Arguments:

Bkpt	Address	Code	Basic	
☐	0x00400004	0x00050513	addi x10, x10, 0	
☐	0x00400008	0x000500593	addi x11, x0, 5	21: li a1, 5
☐	0x0040000c	0x000000993	addi x19, x0, 0	22: li s3, 0
☐	0x00400010	0x00a00ab3	add x21, x0, x10	23: mv s5, a0
☐	0x00400014	0x00b00b33	add x22, x0, x11	24: mv s6, a1
☐	0x00400018	0x0769d063	bge x19, x22, 0x00000060	26: bge s3, s6, exit
☐	0x0040001c	0x00000a13	addi x20, x0, 0	27: li s4, 0

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)
0x10010000	0x00000005	0x00000003	0x00000002	0x00000004
0x10010020	0x00000000	0x00000000	0x00000000	0x00000000
0x10010040	0x00000000	0x00000000	0x00000000	0x00000000
0x10010060	0x00000000	0x00000000	0x00000000	0x00000000
0x10010080	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000

0x10010000 (.data) ☒ Hexadecimal Address

Messages

Run I/O

Clear

BHT Simulator, Version 1.0 (Ingo Kofler)

Branch History Table Simulator

of BHT entries 16 BHT history size 1 Initial value NOT TAKE

Instruction

bge x19,x22,96

@ Address

0x400018

-> Index

6

Index	History	Prediction	Correct	Incorrect	Precision
0	NT	NOT TAKE	0	0	0.00
1	NT	NOT TAKE	0	0	0.00
2	NT	NOT TAKE	0	0	0.00
3	NT	NOT TAKE	0	0	0.00
4	NT	NOT TAKE	0	0	0.00
5	NT	NOT TAKE	0	0	0.00
6	NT	NOT TAKE	0	0	0.00
7	NT	NOT TAKE	0	0	0.00
8	NT	NOT TAKE	0	0	0.00
9	NT	NOT TAKE	0	0	0.00
10	NT	NOT TAKE	0	0	0.00
11	NT	NOT TAKE	0	0	0.00
12	NT	NOT TAKE	0	0	0.00
13	NT	NOT TAKE	0	0	0.00
14	NT	NOT TAKE	0	0	0.00
15	NT	NOT TAKE	0	0	0.00

Log

Instruction bge x19,x22,96 at address 0x400018, maps to index 6
 branches to address 0x4194520
 prediction is: do not take...

Tool Control

Disconnect from Program

Reset

Close

Control and Status

er	Value
0	0x0000000000000000
1	0x0000000000000000
2	0x000000007fffffc
3	0x0000000010008000
4	0x0000000000000000
5	0x0000000000000000
6	0x0000000000000000
7	0x0000000000000000
8	0x0000000000000000
9	0x0000000000000000
10	0x0000000010010000
11	0x0000000000000005
12	0x0000000000000000
13	0x0000000000000000
14	0x0000000000000000
15	0x0000000000000000
16	0x0000000000000000
17	0x0000000000000000
18	0x0000000000000000
19	0x0000000000000000
20	0x0000000000000000
21	0x0000000010010000
22	0x0000000000000005
23	0x0000000000000000
24	0x0000000000000000
25	0x0000000000000000
26	0x0000000000000000
27	0x0000000000000000
28	0x0000000000000000
29	0x0000000000000000
30	0x0000000000000000
31	0x0000000000000000
	0x00000000040001c



Edit Execute

Text Segment

Program Arguments:

Bkpt	Address	Code	Basic	
☐	0x00400008	0x00500593	addi x11, x0, 5	21: li a1, 5
☐	0x0040000c	0x00000993	addi x19, x0, 0	22: li s3, 0
☐	0x00400010	0x00a00ab3	add x21, x0, x10	23: mv s5, a0
☐	0x00400014	0x00b00b33	add x22, x0, x11	24: mv s6, a1
☐	0x00400018	0x0769d063	bge x19, x22, 0x00000060	26: bge s3, s6, exit
☐	0x0040001c	0x00000a13	addi x20, x0, 0	27: li s4, 0
☐	0x00400020	0xffffb0293	addi x5, x22, 0xffffffff	29: addi t0, s6, -1

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)
0x10010000	0x00000005	0x00000003	0x00000002	0x00000004
0x10010020	0x00000000	0x00000000	0x00000000	0x00000000
0x10010040	0x00000000	0x00000000	0x00000000	0x00000000
0x10010060	0x00000000	0x00000000	0x00000000	0x00000000
0x10010080	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000

0x10010000 (.data) ☒ Hexadecimal Address

Messages

Run I/O

Clear

BHT Simulator, Version 1.0 (Ingo Kofler)

Branch History Table Simulator

of BHT entries

16

BHT history size

1

Initial value

NOT TAKE

Instruction

bge x19,x22,96

@ Address

0x400018

-> Index

6

Index	History	Prediction	Correct	Incorrect	Precision
0	NT	NOT TAKE	0	0	0.00
1	NT	NOT TAKE	0	0	0.00
2	NT	NOT TAKE	0	0	0.00
3	NT	NOT TAKE	0	0	0.00
4	NT	NOT TAKE	0	0	0.00
5	NT	NOT TAKE	0	0	0.00
6	NT	NOT TAKE	1	0	100.00
7	NT	NOT TAKE	0	0	0.00
8	NT	NOT TAKE	0	0	0.00
9	NT	NOT TAKE	0	0	0.00
10	NT	NOT TAKE	0	0	0.00
11	NT	NOT TAKE	0	0	0.00
12	NT	NOT TAKE	0	0	0.00
13	NT	NOT TAKE	0	0	0.00
14	NT	NOT TAKE	0	0	0.00
15	NT	NOT TAKE	0	0	0.00

Log

Instruction bge x19,x22,96 at address 0x400018, maps to index 6
 branches to address 0x4194520
 prediction is: do not take...
 branch not taken, prediction was correct

Tool Control

Disconnect from Program

Reset

Close

Control and Status

er	Value
0	0x0000000000000000
1	0x0000000000000000
2	0x000000007fffffc
3	0x0000000010008000
4	0x0000000000000000
5	0x0000000000000000
6	0x0000000000000000
7	0x0000000000000000
8	0x0000000000000000
9	0x0000000000000000
10	0x0000000010010000
11	0x0000000000000005
12	0x0000000000000000
13	0x0000000000000000
14	0x0000000000000000
15	0x0000000000000000
16	0x0000000000000000
17	0x0000000000000000
18	0x0000000000000000
19	0x0000000000000000
20	0x0000000000000000
21	0x0000000010010000
22	0x0000000000000005
23	0x0000000000000000
24	0x0000000000000000
25	0x0000000000000000
26	0x0000000000000000
27	0x0000000000000000
28	0x0000000000000000
29	0x0000000000000000
30	0x0000000000000000
31	0x0000000000000000
	0x0000000004000020

Branch History Table Simulator

of BHT entries **16** BHT history size **1** Initial value **NOT TAKE**

Instruction

bge x20,x5,44

@ Address

0x400028

-> Index

10

Index	History	Prediction	Correct	Incorrect	Precision
0	NT	NOT TAKE	0	0	0.00
1	NT	NOT TAKE	0	0	0.00
2	NT	NOT TAKE	0	0	0.00
3	NT	NOT TAKE	0	0	0.00
4	NT	NOT TAKE	0	0	0.00
5	NT	NOT TAKE	0	0	0.00
6	NT	NOT TAKE	1	0	100.00
7	NT	NOT TAKE	0	0	0.00
8	NT	NOT TAKE	0	0	0.00
9	NT	NOT TAKE	0	0	0.00
10	NT	NOT TAKE	0	0	0.00
11	NT	NOT TAKE	0	0	0.00
12	NT	NOT TAKE	0	0	0.00
13	NT	NOT TAKE	0	0	0.00
14	NT	NOT TAKE	0	0	0.00
15	NT	NOT TAKE	0	0	0.00

Log

branch not taken, prediction was correct

instruction bge x20,x5,44 at address 0x400028, maps to index 10
 branches to address 0x4194432
 prediction is: do not take...



Edit Execute

Text Segment

Program Arguments:

Bkpt	Address	Code	Basic	Source
☐	0x00400014	0x00b00b33	add x22, x0, x11	24: mv s6, a1
☐	0x00400018	0x0769d063	bge x19, x22, 0x00000060	26: bge s3, s6, exit
☐	0x0040001c	0x00000a13	addi x20, x0, 0	27: li s4, 0
☐	0x00400020	0xffffb0293	addi x5, x22, 0xffffffff	29: addi t0, s6, -1
☐	0x00400024	0x413282b3	sub x5, x5, x19	30: sub t0, t0, s3
☐	0x00400028	0x025a5663	bge x20, x5, 0x0000002c	31: bge s4, t0, exit
☐	0x0040002c	0x002a1293	slli x5, x20, 2	32: slli t0, s4, 2

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)
0x10010000	0x00000005	0x00000003	0x00000002	0x00000004
0x10010020	0x00000000	0x00000000	0x00000000	0x00000000
0x10010040	0x00000000	0x00000000	0x00000000	0x00000000
0x10010060	0x00000000	0x00000000	0x00000000	0x00000000
0x10010080	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000

0x10010000 (.data)
☒ Hexadecimal Address

Messages Run I/O

Branch History Table Simulator

of BHT entries 16

BHT history size 1

Initial value

NOT TAKE

Instruction

bge x20,x5,44

@ Address

0x400028

-> Index

10

Index	History	Prediction	Correct	Incorrect	Precision
0	NT	NOT TAKE	0	0	0.00
1	NT	NOT TAKE	0	0	0.00
2	NT	NOT TAKE	0	0	0.00
3	NT	NOT TAKE	0	0	0.00
4	NT	NOT TAKE	0	0	0.00
5	NT	NOT TAKE	0	0	0.00
6	NT	NOT TAKE	1	0	100.00
7	NT	NOT TAKE	0	0	0.00
8	NT	NOT TAKE	0	0	0.00
9	NT	NOT TAKE	0	0	0.00
10	NT	NOT TAKE	1	0	100.00
11	NT	NOT TAKE	0	0	0.00
12	NT	NOT TAKE	0	0	0.00
13	NT	NOT TAKE	0	0	0.00
14	NT	NOT TAKE	0	0	0.00
15	NT	NOT TAKE	0	0	0.00

Log

instruction bge x20,x5,44 at address 0x400028, maps to index 10
 branches to address 0x4194432
 prediction is: do not take...
 branch not taken, prediction was correct

Execute

xt Segment

am Arguments:

Address	Code	Basic	Source
0x00400018	0x0769d063	bge x19, x22, 0x00000060	26: bge s3, s6, exit t
0x0040001c	0x00000a13	addi x20, x0, 0	27: li s4, 0
0x00400020	0xffffb0293	addi x5, x22, 0xffffffff	29: addi t0, s6, -1
0x00400024	0x413282b3	sub x5, x5, x19	30: sub t0, t0, s3
0x00400028	0x025a5663	bge x20, x5, 0x0000002c	31: bge s4, t0, exit t1
0x0040002c	0x002a1293	slli x5, x20, 2	32: slli t0, s4, 2
0x00400030	0x005a82b3	add x5, x21, x5	33: add t0, s5, t0

ta Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	V
0x10010000	0x00000005	0x00000003	0x00000002	0x00000004	
0x10010020	0x00000000	0x00000000	0x00000000	0x00000000	
0x10010040	0x00000000	0x00000000	0x00000000	0x00000000	
0x10010060	0x00000000	0x00000000	0x00000000	0x00000000	
0x10010080	0x00000000	0x00000000	0x00000000	0x00000000	
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000	
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000	



0x10010000 (.data)

☒ Hexadecimal Address

Pages

Run I/O

第五章

- 流水线概述
- 流水线数据通路和控制
- 解决数据冒险的方法：前递与停顿
- 控制冒险
- 例外

例外(exception)和中断(interrupt)

- 例外和中断是控制逻辑需要实现的任务之一
 - 除分支指令外，另一种改变指令执行控制流的方式
- 不同的ISA对于例外和中断的定义不同
 - Intel x86使用中断同时指代两者
 - 对于RISC-V

例外 指 意外的控制流变化，而这些变化**无须区分**产生原因是来自于处理器内部还是外部

中断 **仅指** 由处理器外部事件引发 的控制流变化

- 对例外进行时序**优化**是困难的

RISC-V中的例外和中断示例

事件类型	来源	RISC-V中的表示
系统重启	外部	例外
I/O设备请求	外部	中断
用户程序进行操作系统调用	内部	例外
未定义指令	内部	例外
硬件故障	皆可	皆可

RISC-V下操作系统如何处理例外

- 若执行某指令时出现硬件故障：保存发生例外的指令地址，将控制权转交给操作系统
 - 使用例外程序计数器(Exception Program Counter, EPC)保存发生例外的指令地址
 - 跳转到统一的入口地址，进行例外处理
 - 0000 0000 1C09 0000_{hex}
 - 保存例外发生的原因
 - 例外原因寄存器(Exception Cause Register, CAUSE)
 - CAUSE：大多数位未被使用。

另一种例外处理方式(x86等)

- 采用向量式中断（x86中统称为中断）
 - 例外原因决定后续控制流的起始地址
- 基址寄存器 + 例外原因（作为偏移量）
 - 未定义指令的偏移量： 00 0100 0000₂
 - 硬件故障的偏移量： 01 1000 0000₂
- 转到例外处理程序

例外处理程序的工作

- 如果可以重启程序的执行（I/O设备请求等）
 - 完成例外处理的所有操作
 - 使用SEPC寄存器中的内容重启程序的正常执行
- 否则（未定义指令或硬件故障等）
 - 停止当前程序的执行
 - 使用SEPC, SCAUSE等来报告错误

流水线实现中的例外

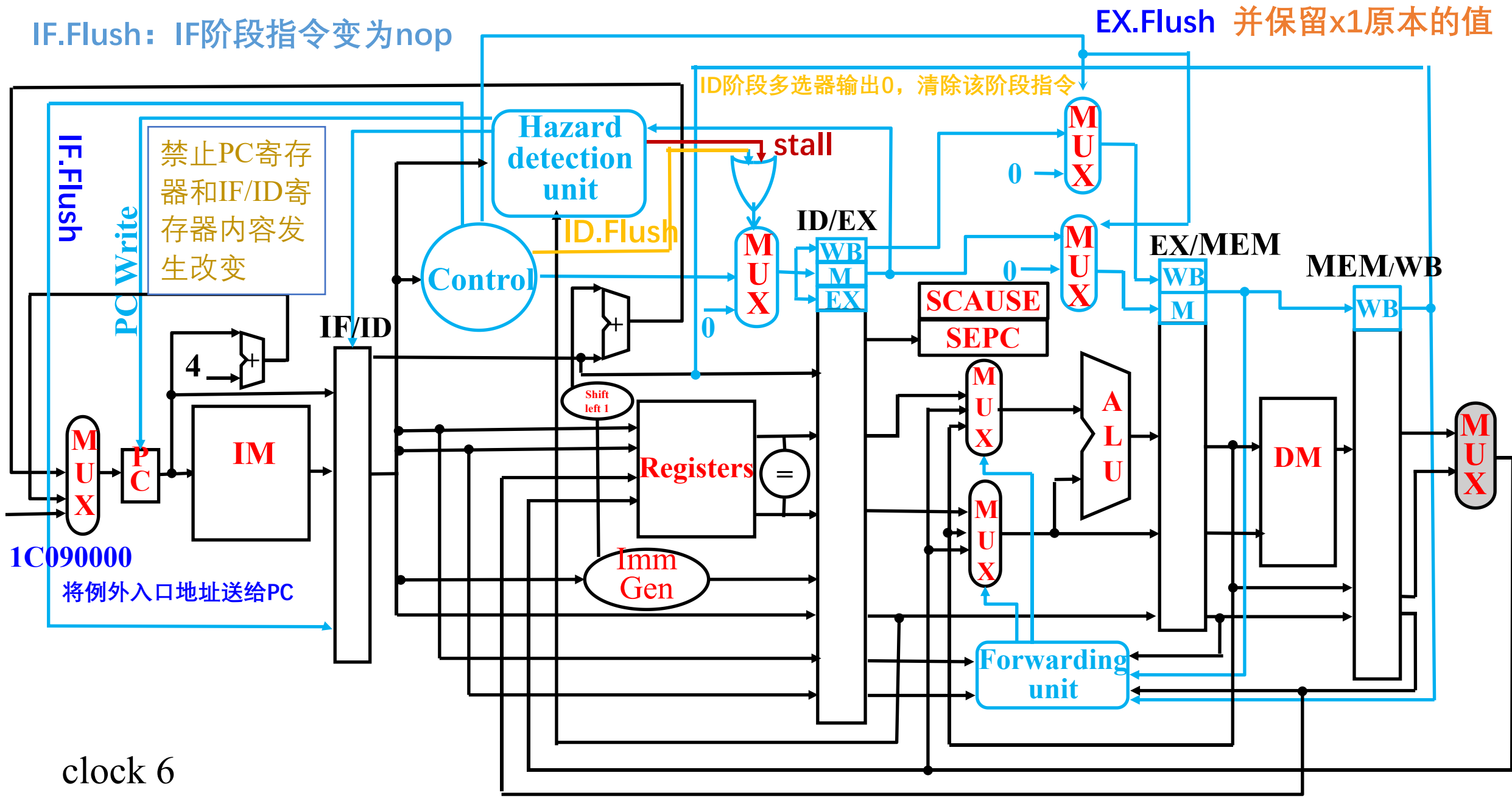
- 流水线实现中，将例外处理看作另一种控制冒险。
- 假设add指令执行阶段发生了硬件故障。

```
40      sub  x11, x2, x4
44      and  x12, x2, x5
48      or   x13, x2, x6
4C      add  x1, x2, x1
50      sub  x15, x6, x7
54      ld   x16, 100(x7)
...
```

- Handler

```
1C090000      sd  x26, 1000(x10)
1C090004      sd  x27, 1008(x10)
...
```


假设第四条指令 add x1, x2, x1 的EX阶段发生了硬件故障



54: ld x16, 100(x7)

50: sub x15, x6, x7

4C: add x1, x2, x1

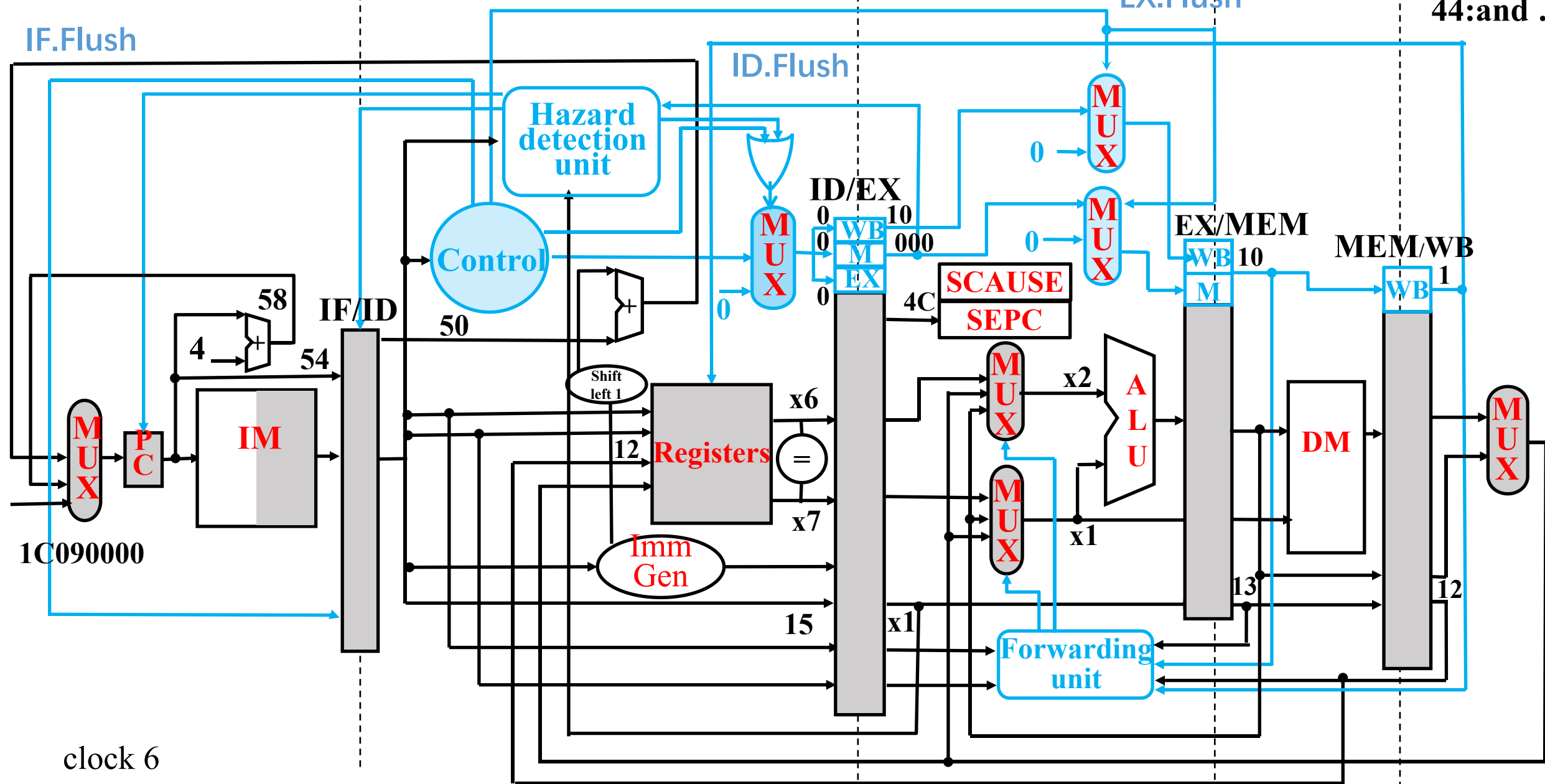
48: or x13, ...

44: and ...

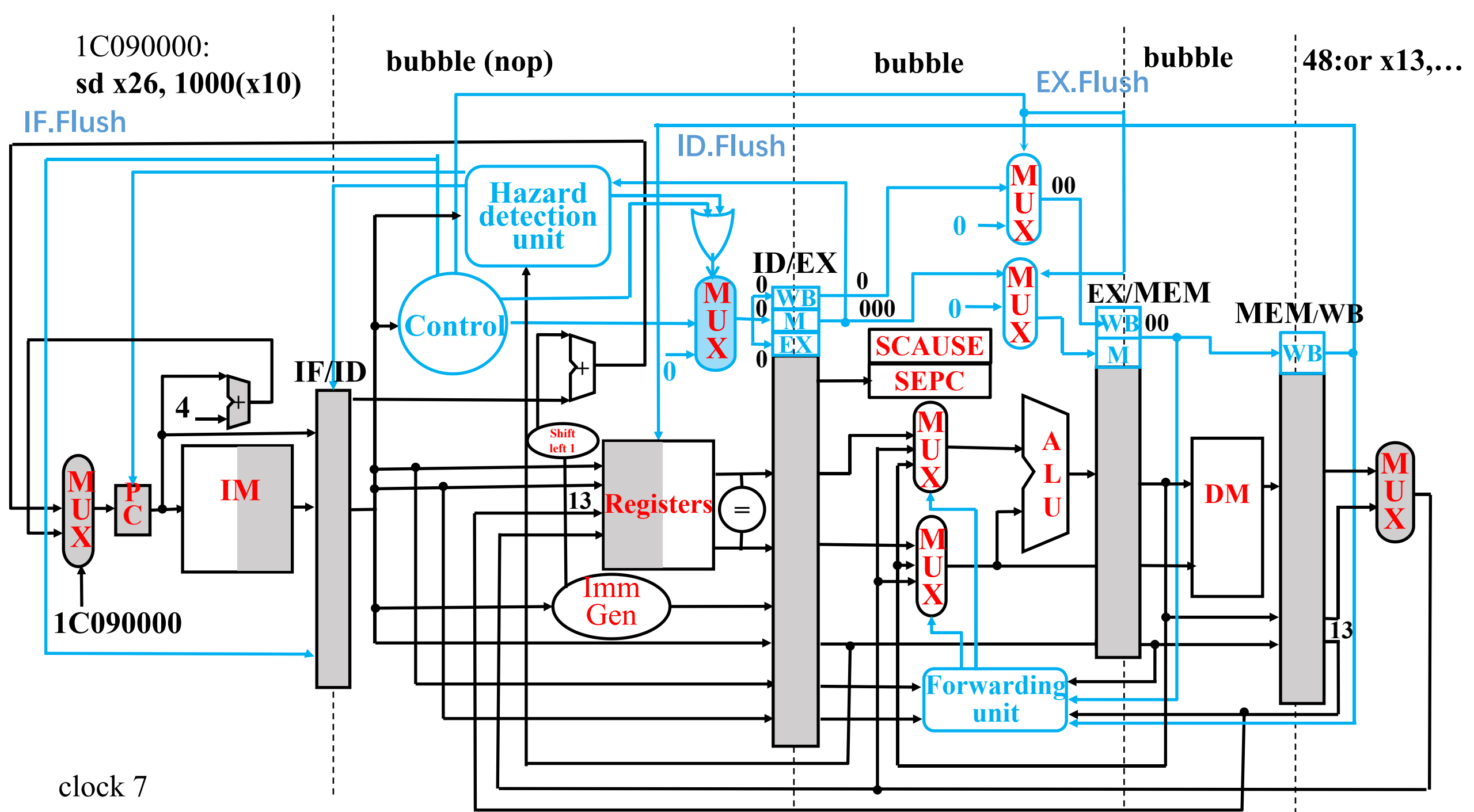
IF.Flush

ID.Flush

EX.Flush



clock 6



流水线实现中的例外处理小结

- IF阶段的指令：变为nop操作
- ID阶段的指令：增加新的逻辑部件，使译码阶段输出为0
- EX阶段的指令：需要保留x1原本的值
- 例外之前的指令，正常完成
- 设置SEPC和SCAUSE的寄存器值
- 转到例外处理程序

流水线处理器小结

- 用流水线方式实现并行加速
- 流水线数据通路和控制
 - 单个时钟周期内的数据通路 vs 多个时钟周期的数据通路
 - 流水线寄存器
- 三类冒险：结构冒险、**数据冒险、控制冒险**
- 消除冒险：增加硬件、前递与停顿、分支预测等
- 例外及其处理机制简介

学习本课程的收获

- 高级语言程序如何被翻译成机器语言？
- CPU原理：硬件如何执行最终的程序？**软件和硬件之间的接口是什么？**
- 什么因素决定了程序的性能？**程序员如何改进程序性能？**
- 硬件设计人员可以使用哪些技术来提高性能？
 - 现代计算机设计的基本概念和原理——**流水线**、层次结构等
- 哪些伟大思想奠定了现代计算技术的基础？

计算机体系结构中的7个伟大思想

面向摩尔定律的设计

1. 使用抽象简化设计
2. 加速经常性事件
3. 通过并行提高性能
4. 通过**流水线**提高性能
5. 通过**预测**提高性能
6. 存储层次
7. 通过冗余提高可靠性